

Restaurant Operating System (ROS)

Software Requirements Specification & System Architecture Document

Systems Analysis & Architecture Team

Version 1.0 — August 2026

Table of Contents

Document Control | ضبط الوثيقة

Document Identification

Field	Value
Document Title	Restaurant Operating System — Software Requirements Specification
Document ID	ROS-SRS-001
Version	1.0
Status	Baselined for Phase 1 Development
Classification	Confidential — Internal & Investor Distribution
Standard Basis	IEEE 830-1998 / ISO-IEC-IEEE 29148:2018
Architecture Notation	C4 Model, UML 2.5, BPMN 2.0
Primary Language	English (technical), Arabic (business summaries)
Total Requirement Count	612 functional, 148 non-functional

Revision History

Version	Date	Author	Description
0.1	2026-05-02	Business Analysis	Initial feature inventory captured from stakeholder workshop
0.2	2026-05-19	Business Analysis	Market and competitor analysis added
0.3	2026-06-08	Solution Architecture	Architecture baseline, modular

Version	Date	Author	Description
0.4	2026-06-27	Solution Architecture	monolith decision Domain model, bounded contexts, ERD
0.5	2026-07-11	Engineering	Offline synchronisation engine design
0.6	2026-07-22	Security	Threat model and compliance mapping
0.9	2026-07-30	All	Full internal review, requirement traceability closed
1.0	2026-08-04	All	Baselined

Approval Matrix

Role	Responsibility	Sign-off Required
Product Owner	Scope, priority, acceptance criteria	Yes
Solution Architect	Architecture, technology, integration	Yes
Engineering Lead	Feasibility, estimation, delivery plan	Yes
QA Lead	Testability, acceptance test coverage	Yes
Security Officer	Threat model, compliance, data protection	Yes
Finance / Compliance	Tax, invoicing, fiscal integration	Yes
Commercial Director	Packaging, pricing, go- to-market	Advisory

Distribution List

Product Owner, Solution Architect, Engineering Leads (Backend / Frontend / Mobile), QA Lead, DevOps Lead, Security Officer, Head of Customer Success, Commercial Director, Investment Committee.

Executive Summary (Arabic) — ملخص تنفيذي

الفكرة باختصار

المنتج ليس برنامج كاشير. المنتج **نظام تشغيل متكامل للمطاعم** — منصة واحدة تربط المبيعات بالمطبخ بالمخزون بالوصفات بالمشتريات بالموظفين بالنقدية بالتقارير، بحيث إن أي عملية بيع واحدة تُحدث أثراً تلقائياً في كل هذه الوحدات في نفس اللحظة.

الفرق الجوهرى بين "برنامج كاشير" و"نظام تشغيل" هو التالي: برنامج الكاشير يسجل أن الزبون دفع ١٢٠ جنيهاً. نظام التشغيل يسجل أن الزبون دفع ١٢٠ جنيهاً، وأن تكلفة ما أكله كانت ٣٩ جنيهاً، وأن ٢٠٠ جرام دجاج و ٥٠ جرام جبنة و ١ عبوة تغليف خرجت من مخزون الفرع، وأن المخزون المتبقى من الجبنة يكفي ١٧ طلباً فقط وينتهي غداً الساعة الثالثة، وأن الطلب تأخر ٤ دقائق في محطة الجريل، وأن الكاشير رقم ٧ أعطى خصماً بنسبة ١٥٪ للمرة الثالثة هذا الأسبوع في نفس الساعة.

المشكلة السوقية

المطاعم في المنطقة العربية تعاني من ثلاث مشاكل متكررة:

الأولى أن الأنظمة الموجودة مفصولة عن بعضها — نظام للكاشير، وإكسل للمخزون، وواتساب للمشتريات، ودفتر للحضور. النتيجة أن صاحب المطعم لا يعرف تكلفته الحقيقية إلا بعد نهاية الشهر، وغالباً لا يعرفها أصلاً.

الثانية أن الأنظمة الإنجليزية أولاً، والعربية فيها مجرد (Toast, Square, Lightspeed) الثانية أن الأنظمة العالمية السعودي بشكل أصيل ZATCA ترجمة ملصوقة، ولا تدعم الفاتورة الإلكترونية المصرية أو ربط

الثالثة أن الأنظمة الإقليمية الموجودة تركّز على نقطة البيع وتعامل المخزون والتكلفة كإضافة ثانوية، بينما الهامش الحقيقي في المطاعم يُربح أو يُخسر في المخزون والهدر وليس في شاشة الكاشير.

موقعنا التنافسي

وليس بالترجمة، **يعمل بدون إنترنت** بشكل كامل (Arabic-First) نبني نظاماً عربياً بالتصميم وليس جزئي، **يخصم المخزون من الوصفة** تلقائياً في كل عملية بيع، وقواعده الضريبية قابلة للتهيئة لكل دولة عبر حزم إعدادات منفصلة بدل تثبيتها داخل الكود.

نموذج الإيراد

بثلاث باقات — أساسية، احترافية، مؤسسية — مع رسوم تفعيل (SaaS) اشتراك شهري لكل فرع أولية، ورسوم إضافية لأجهزة نقاط البيع، وعمولة على الطلبات الواردة من تطبيقات التوصيل. متوسط الإيراد المستهدف لكل فرع بين ٦٥ و ٢٤٠ دولاراً شهرياً حسب الباقة.

الخلاصة الهندسية

على (Bounded Contexts) مقسم إلى وحدات ذات حدود واضحة **Modular Monolith** نبدأ بـ لنقطة البيع والمطبخ. لا نبدأ بالخدمات Flutter للإدارة و Next.js بواجهة Redis مع PostgreSQL المصغرة لأن الفريق في المرحلة الأولى صغير، والتعقيد التشغيلي للخدمات المصغرة يقتل السرعة قبل الوصول لأول مئة عميل. لكن الحدود بين الوحدات تُرسم من اليوم الأول بحيث يمكن فصل أي وحدة لاحقاً بدون إعادة كتابة.

Chapter 1 — Introduction | المقدمة

1.1 Purpose of This Document

This document is the authoritative specification for the Restaurant Operating System (hereafter “ROS” or “the System”). It exists to serve five distinct audiences simultaneously, and each section is written with awareness of which audience it primarily serves.

Engineering uses this document as the definitive source of behaviour. When a developer asks “what should happen when a cashier voids a line item that has already been fired to the kitchen and partially consumed inventory?”, the answer must be in this document, not in someone’s memory or a Slack thread. Ambiguity in a specification is not a stylistic flaw — it is a defect that costs three to ten times more to fix after implementation than before it.

Quality Assurance uses this document to derive test cases. Every functional requirement in this document is written to be independently verifiable. A requirement that cannot be tested is not a requirement; it is an aspiration, and it has been either rewritten or moved to the vision section.

Product Management uses this document to sequence delivery. The requirement priority classification (Must / Should / Could / Won’t, per MoSCoW) combined with the release roadmap in Chapter 30 defines what ships in which phase.

Commercial and Sales use Chapters 2, 3, and 30 to understand what the product does, who it is for, and what may honestly be promised to a prospect. Sales teams that promise features not in this document create technical debt denominated in customer trust.

Investors and Executive Sponsors use the Executive Summary, Chapter 2 (Business Context), Chapter 30 (Roadmap), and Chapter 31 (Risk Register) to evaluate the venture.

1.2 Product Scope Statement

The Restaurant Operating System is a multi-tenant, multi-branch, multi-country, bilingual SaaS platform that manages the complete operational lifecycle of restaurant and café businesses.

The System’s scope is defined by a single organising principle, stated here because it governs every design decision that follows:

Every sale is a transaction against the entire business, not merely against the cash drawer.

A single completed order must, atomically and without human intervention, produce all of the following effects:

1. A financial record in the sales ledger, denominated in the branch’s operating currency.
2. Kitchen production instructions routed to the correct preparation stations.
3. Inventory depletion for every ingredient in every recipe of every sold item, expanded recursively through sub-recipes.
4. Recognition of Cost of Goods Sold at the recipe’s current valuation.

5. A cash or electronic payment record attributed to a specific cashier, shift, and drawer.
6. A tax record formatted according to the branch's country configuration pack.
7. Attribution of the sale to an employee for performance measurement.
8. An immutable audit trail entry.
9. An update to any linked customer's loyalty balance and purchase history.

Any system that does not produce all nine effects from one action is a point-of-sale system with add-ons. This document specifies a system that produces all nine.

1.3 Product Perspective

ROS is a greenfield product. It is not a replacement for or extension of an existing internal system. It will, however, operate in an ecosystem of external systems with which it must integrate:

External System Class	Examples	Integration Direction
Payment Terminals	Network International, Geidea, PayTabs, Fawry	Bidirectional
Fiscal / Tax Authority	Egyptian Tax Authority (ETA), ZATCA (KSA), FTA (UAE)	Outbound submission, inbound acknowledgement
Delivery Aggregators	Talabat, HungerStation, Elmenus, Deliveroo, Careem	Inbound orders, outbound status
Accounting Systems	QuickBooks, Xero, Odoo, Zoho Books, local ERPs	Outbound journal export
Payroll Providers	Local payroll bureaus, HR platforms	Outbound timesheet export
Communication	SMS gateways, WhatsApp Business API, email	Outbound
Identity	Google Workspace, Microsoft Entra ID (enterprise tier)	Inbound SSO
Hardware	Thermal printers, cash drawers, barcode scanners, customer displays, kitchen printers, scales	Local device protocol

1.4 Definitions, Acronyms, and Abbreviations

This is the operative glossary. Terms defined here have precise meaning throughout the document and must not be used loosely.

Term	Definition
Tenant	A commercial customer of ROS. One tenant owns one or more brands. Data isolation boundary.

Term	Definition
Brand	A restaurant concept owned by a tenant. A tenant may own “Burger Co.” and “Pasta House” as separate brands with separate menus.
Branch	A physical location where sales occur. Belongs to exactly one brand. Has its own inventory, cash drawers, and staff roster.
Warehouse	A stock-holding location that does not sell to end customers. May be standalone or attached to a branch.
Central Kitchen (CK)	A production facility that manufactures semi-finished or finished goods and distributes them to branches. Also called Commissary.
Terminal	A registered physical or virtual device running the POS or KDS application. Uniquely identified and licensed.
Shift	A bounded period during which a specific employee operates a specific terminal with a specific cash drawer.
Business Day	An operational day, which may not align with the calendar day. A branch closing at 03:00 attributes those sales to the previous business day.
Order	A customer’s request, comprising one or more order lines, in one of the defined order types.
Order Line	A single menu item within an order, with quantity, modifiers, price, and preparation state.
Modifier	An adjustment to an order line — an addition, removal, or substitution — which may carry a price delta and a recipe delta.
Ticket	The kitchen-facing representation of an order or a subset of an order, routed to one preparation station.
Fire	The act of releasing an order or course to the kitchen for preparation.
Course	A grouping of order lines intended to be served together in sequence (starters, mains, desserts).
Recipe	The bill of materials for a sellable item:

Term	Definition
	which inventory items, in what quantities, in what units.
Sub-Recipe	A recipe that is an ingredient of another recipe. Sauces, doughs, marinades, stocks. Also called Prep Item.
Yield	The output quantity a recipe produces, accounting for preparation loss.
Theoretical Usage	The inventory quantity that <i>should</i> have been consumed, computed from sales × recipes.
Actual Usage	The inventory quantity that <i>was</i> consumed, computed from physical counts: Opening + Received – Closing.
Variance	Actual Usage minus Theoretical Usage. The single most important operational metric in the System.
Food Cost %	Cost of Goods Sold ÷ Net Sales, expressed as a percentage.
Contribution Margin	Selling price minus direct variable cost, per item or per order.
Waste	Inventory removed from stock without a corresponding sale, with a recorded reason.
Void	Cancellation of an order line before the order is finalised.
Refund	Return of value after an order has been finalised and paid.
Comp	An item given without charge, recorded at full cost.
Discount	A reduction in the selling price, recorded with a reason and, above a threshold, an approver.
Opening Float	Cash placed in a drawer before trading begins.
Cash Variance	Counted cash minus expected cash at shift close.
X Report	A mid-shift, non-resetting summary of activity to date.
Z Report	The terminal report that closes a shift or business day and resets counters.
Country Pack	A versioned configuration bundle containing a jurisdiction's tax rules, invoice format, currency, fiscal integration, and legal text.

Term	Definition
FIFO	First In, First Out. Stock consumption in order of receipt date.
FEFO	First Expired, First Out. Stock consumption in order of expiry date.
Batch / Lot	A quantity of an inventory item received or produced together, sharing an expiry date and cost.
UoM	Unit of Measure.
Outbox	A durable local queue of state changes awaiting synchronisation to the server.
HLC	Hybrid Logical Clock. A timestamp mechanism combining physical and logical time for distributed ordering.
Idempotency Key	A client-generated unique token that guarantees an operation is applied at most once.
RBAC	Role-Based Access Control.
ABAC	Attribute-Based Access Control.
SoD	Segregation of Duties.
RTO / RPO	Recovery Time Objective / Recovery Point Objective.

1.5 Requirement Identification Scheme

Every requirement in this document carries a stable identifier. Identifiers are never reused, even after a requirement is deleted; deleted requirements are marked [WITHDRAWN] and retained.

Format: <TYPE> - <MODULE> - <NUMBER>

Types:

Prefix	Meaning
FR	Functional Requirement — something the system does
NFR	Non-Functional Requirement — a quality the system exhibits
BR	Business Rule — a constraint from the business domain
UC	Use Case
DR	Data Requirement
IR	Integration Requirement

Prefix	Meaning
CR	Constraint

Module codes:

Code	Module
POS	Point of Sale
KDS	Kitchen Display System
MNU	Menu & Recipe Management
INV	Inventory Management
PRC	Purchasing & Suppliers
CST	Costing, Waste & Profitability
HRM	Employee Management
SEC	Security, Roles & Permissions
FIN	Cash & Financial Management
BRN	Branch & Central Kitchen
CRM	Customer & Loyalty
RPT	Reporting & Analytics
AUD	Audit & Compliance
OFF	Offline Operation & Synchronisation
LOC	Localisation & Country Packs
INT	External Integrations
PLT	Platform, Tenancy & Administration

Example: FR - POS - 042 is the forty-second functional requirement of the Point of Sale module.

1.6 Requirement Priority Classification

Priority	Meaning	Release Implication
M (Must)	The product is not viable without it.	Phase 1
S (Should)	Significant value; product ships without it under protest.	Phase 1 or 2
C (Could)	Desirable; included if capacity permits.	Phase 2 or 3
W (Won't, this time)	Explicitly deferred. Recorded to prevent re-litigation.	Backlog

1.7 Document Conventions

Requirements use RFC 2119 keywords with the following meanings:

- **SHALL / MUST** — an absolute requirement. Non-compliance is a defect.
- **SHOULD** — a strong recommendation. Deviation requires documented architectural justification.
- **MAY** — genuinely optional. Implementer's discretion.

Notes formatted like this convey engineering rationale:

Rationale: Explains *why* a requirement exists. Rationale is not a requirement and is not tested, but removing a requirement without reading its rationale is how systems acquire regressions.

Blocks formatted like this identify risk:

Risk: Identifies a specific way an implementation can go wrong, usually derived from observed industry failure.

1.8 Assumptions

ID	Assumption	Impact If False
A-01	Branches have at least intermittent internet connectivity (minimum: several hours per day).	Full offline architecture already mitigates; sync backlog grows but system functions.
A-02	Tenants operate legally registered businesses with tax registration numbers.	Fiscal integration cannot be completed; tenant restricted to non-fiscal mode.
A-03	Kitchen staff can read either Arabic or the branch's configured language.	Requires icon-driven KDS mode; specified as FR-KDS-031.
A-04	Payment card processing is performed by certified external terminals, not by ROS.	Would force PCI-DSS Level 1 scope onto ROS. This assumption is load-bearing — see Chapter 20.
A-05	Tenants accept cloud hosting; on-premise deployment is not offered in Phase 1.	Enterprise deals in regulated markets may require a private deployment SKU.
A-06	Recipe data is maintained by the tenant. ROS does not supply a recipe library.	Onboarding effort increases; mitigated by import tooling and templates.

1.9 Constraints

ID	Constraint	Type
CR-01	The POS application MUST operate for a minimum of 72 hours without server	Technical

ID	Constraint	Type
	connectivity.	
CR-02	Arabic and English MUST be first-class languages with full RTL/LTR layout support, not translations.	Product
CR-03	Country-specific tax logic MUST NOT be compiled into core application code.	Architectural
CR-04	Financial records, once posted, MUST be immutable. Corrections are made by compensating entries.	Regulatory
CR-05	The System MUST NOT store, process, or transmit primary account numbers (PAN) of payment cards.	Security
CR-06	Phase 1 engineering team is capped at 9 engineers.	Organisational
CR-07	The POS UI MUST be operable on a 10-inch touchscreen at 1280×800 without horizontal scrolling.	Hardware
CR-08	Audit records MUST be retained for a minimum of 7 years or the jurisdiction's statutory period, whichever is longer.	Regulatory

1.10 References

Reference	Description
ISO/IEC/IEEE 29148:2018	Requirements engineering standard
IEEE 830-1998	Recommended practice for SRS (superseded, used for structure)
C4 Model (Simon Brown)	Architecture diagramming notation
Domain-Driven Design (Evans, 2003)	Bounded context and aggregate modelling
Implementing Domain-Driven Design (Vernon, 2013)	Aggregate design rules applied in Chapter 7
Patterns of Enterprise Application Architecture (Fowler, 2002)	Pattern vocabulary in Chapter 24
Designing Data-Intensive Applications (Kleppmann, 2017)	Basis for the synchronisation design in Chapter 21

Reference	Description
OWASP ASVS 4.0	Application security verification standard
PCI-DSS v4.0	Payment card industry data security standard
ETA Egypt e-Receipt & e-Invoice specifications	Egyptian fiscal integration
ZATCA Fatoora Phase 2 specifications	Saudi fiscal integration
UAE FTA VAT legislation	UAE tax configuration

Chapter 2 — Business Context & Market Analysis | السياق التجاري

2.1 Why This Product Exists

The restaurant industry operates on margins that are simultaneously thin and volatile. A well-run independent restaurant in the MENA region operates at a net margin between 6% and 12%. A poorly-run one, indistinguishable from the outside, operates at negative margin and does not know it until the cash runs out.

The difference between these two restaurants is rarely the food. It is almost always the presence or absence of operational visibility, and specifically visibility into four numbers:

1. **Food Cost Percentage** — what fraction of revenue is consumed by ingredients.
2. **Variance** — the gap between what should have been consumed and what actually was.
3. **Labour Cost per Sales Hour** — whether staffing matches demand.
4. **Contribution Margin by Item** — which dishes actually make money.

An operator who knows these four numbers weekly can correct course. An operator who learns them from an accountant sixty days after month-end is managing a business that ended two months ago.

The systemic failure in the market is not the absence of software. Software is abundant. The failure is that the software is *disconnected*. The POS knows revenue. A spreadsheet knows purchases. Nobody knows recipes precisely enough to compute theoretical usage. Therefore variance — the single most diagnostic number in restaurant operations — is uncomputable, and the losses it would reveal (theft, over-portioning, unrecorded waste, receiving fraud, spoilage) remain invisible.

The core thesis: Variance is only computable when sales, recipes, and inventory live in one system with one clock. Every architectural decision in this document flows from the requirement to make variance computable, accurate, and available daily rather than monthly.

2.2 Quantifying the Problem

Consider a representative single-branch café with monthly revenue of USD 40,000 and a target food cost of 30%.

Scenario	Food Cost %	Monthly COGS	Monthly Delta
Target (controlled)	30.0%	12,000	baseline
Typical uncontrolled	34.5%	13,800	-1,800
Poorly controlled	38.0%	15,200	-3,200

A 4.5 percentage-point drift in food cost — which is entirely ordinary in an operation without recipe-linked inventory — costs this business USD 21,600 per year. That is frequently more than the owner’s own annual drawings.

The economic argument for ROS is therefore not “software convenience.” It is that a system costing USD 100–200 per month per branch, which recovers even one-third of that drift, returns 3× to 9× its cost. This is the sales narrative, and it is defensible because it is arithmetic rather than aspiration.

2.3 Market Segmentation

The System addresses four distinct segments. Their needs overlap but their buying behaviour, price sensitivity, and feature priorities differ materially.

Segment A — Independent Single Branch

Profile: One location, 5–20 staff, owner is present daily and often operating. Revenue USD 15k–60k/month.

Primary pain: Does not know true cost. Manages inventory by looking at shelves. Suspects theft but cannot prove it.

Buying behaviour: Price sensitive. Decides in days, not months. Buys on demonstration, not on RFP. Churns quickly if onboarding is hard.

Critical features: POS that just works, offline reliability, simple inventory, food cost per item, daily Z report.

Deal size: USD 65–95/month. Sales cost must be near zero — self-service onboarding is mandatory for this segment to be profitable.

Segment B — Small Chain (2–10 Branches)

Profile: Owner has stepped back from daily operation. An operations manager exists. Revenue USD 100k–600k/month aggregate.

Primary pain: Cannot compare branches. Cannot enforce consistent pricing or recipes. Suspects one branch is losing money but cannot isolate it.

Buying behaviour: Evaluates 2–3 vendors. Cares about migration effort. Wants a reference customer.

Critical features: Central menu and pricing, branch comparison reports, inter-branch transfers, consolidated purchasing, role-based access so branch managers see only their branch.

Deal size: USD 110–150/month per branch. This is the segment where the business becomes economically attractive.

Segment C — Multi-Brand Group / Franchise

Profile: Multiple brands, possibly multiple countries, central kitchen, franchise operations. 10–100+ locations.

Primary pain: Consolidation, standardisation, franchisee compliance, royalty calculation, central production planning.

Buying behaviour: Formal procurement. Security review. Contract negotiation. 3–9 month sales cycle. Requires SLA, uptime commitments, and often data residency guarantees.

Critical features: Multi-brand hierarchy, central kitchen production and distribution, franchise royalty reporting, API access, SSO, audit log export, custom reporting.

Deal size: USD 180–240/month per branch, plus platform fee. High retention. Long payback but high lifetime value.

Segment D — Cloud Kitchen / Delivery-Only

Profile: No dine-in. Multiple virtual brands from one kitchen. 100% of orders from aggregators.

Primary pain: Managing 4–8 aggregator tablets. Reconciling aggregator payouts. Attributing cost across virtual brands sharing one inventory.

Critical features: Aggregator integration (order injection), multi-brand from single inventory, payout reconciliation, prep-time analytics, station load balancing.

Deal size: USD 130–180/month. Fast-growing segment; high integration dependency.

2.4 Competitive Landscape

Competitor	Strength	Structural Weakness We Exploit
Foodics (KSA/regional)	Strong regional presence, good POS, ZATCA integration, payments bundling	Inventory and recipe depth is shallow relative to POS; heavy upsell model; limited Egypt fiscal depth
Marn (KSA)	Clean UX, ZATCA-native, competitive pricing	Narrow feature set beyond POS; limited multi-country
Toast (US)	Deepest feature set in the industry, excellent hardware integration	Not available or supported in MENA; no Arabic; no regional fiscal compliance;

Competitor	Strength	Structural Weakness We Exploit
Square (US/global)	Frictionless onboarding, strong SMB brand	hardware lock-in Weak inventory/recipe module; no Arabic-first design; no MENA fiscal compliance
Lightspeed (global)	Strong inventory, good reporting	Expensive; Arabic is a translation layer; poor offline story
Odoo POS (open source)	Cheap, extensible, ERP-integrated	Requires implementation partner; POS UX is weak; offline is unreliable; not restaurant-native
Local/regional bespoke systems	Cheap, locally supported, often installed on-premise	No cloud, no multi-branch consolidation, no updates, single point of failure, no offline sync (they're offline-only)
Excel + WhatsApp	Free, universally understood	The actual incumbent in Segments A and B. Underestimating it is the classic mistake.

Strategic note: The most common competitor in Segments A and B is not a software vendor. It is a spreadsheet plus a notebook plus a WhatsApp group. Any onboarding flow that is harder than the incumbent will lose to the incumbent regardless of feature superiority. This drives NFR-USA-003 (a new branch must be able to take its first order within 30 minutes of signup).

2.5 Differentiation Strategy

Four differentiators are defensible. Everything else is table stakes.

2.5.1 Arabic-First, Not Arabic-Translated

Most competitors implement Arabic by wrapping an English interface in a translation file and flipping the CSS direction. This produces interfaces where numbers are misaligned, dates are wrong, mixed-direction strings (an Arabic dish name containing a Latin brand name) render incorrectly, receipts break, and Arabic text is truncated because the layout was measured against English word lengths.

Arabic-first means: the design system is authored RTL and mirrored to LTR (not the reverse); typography is selected for Arabic legibility at small sizes on thermal printers and low-DPI POS screens; bidirectional text algorithm handling is tested as a first-class concern; numerals are configurable (Arabic-Indic vs Western); and Arabic is the default locale for the MENA market rather than an option buried in settings.

2.5.2 True Offline Operation

“Offline support” in most competitors means orders are queued and payments cannot be taken, or the system degrades to a cash-only calculator. In ROS, offline is a first-class operating mode: full order entry, kitchen routing, payment recording across all tender types, receipt printing, shift open and close, and inventory depletion all function without connectivity, with deterministic conflict resolution on reconnect.

Rationale: Internet reliability in the target markets is not a solved problem. A POS that stops working stops the business. This is the single most common reason restaurants abandon cloud POS systems and revert to local software. Chapter 21 specifies the synchronisation engine in full.

2.5.3 Recipe-Linked Inventory as a Default, Not an Upsell

In ROS, selling a burger deducts its ingredients. This is not a premium module. It is the core value proposition and is included from the entry tier, because a restaurant that does not experience recipe-linked depletion never experiences the product’s actual value and churns.

2.5.4 Configurable Country Packs

Tax logic, invoice layout, fiscal transmission, currency, rounding rules, receipt legal text, and reporting formats are data, not code. Adding a country is a configuration and certification exercise, not a development project. This makes international expansion a commercial decision rather than an engineering programme.

2.6 Commercial Model

2.6.1 Packaging

Capability	Starter	Professional	Enterprise
POS terminals included	1	3	Unlimited
KDS screens	1	4	Unlimited
Offline operation	Yes	Yes	Yes
Menu & modifiers	Yes	Yes	Yes
Recipes & food cost	Yes	Yes	Yes
Inventory & stock counts	Basic	Full	Full
Batch & expiry (FEFO)	No	Yes	Yes
Purchasing & suppliers	No	Yes	Yes
Waste management	Basic	Full	Full + anomaly detection

Capability	Starter	Professional	Enterprise
Employee scheduling	No	Yes	Yes
Multi-branch consolidation	n/a	Yes	Yes
Central kitchen production	No	No	Yes
Multi-brand	No	No	Yes
Fraud detection	No	Basic	Full
Custom roles	No	Yes	Yes
API access	No	Read-only	Full
SSO (SAML/OIDC)	No	No	Yes
Aggregator integration	Add-on	Yes	Yes
Support SLA	Email, 24h	Chat, 8h	Dedicated, 2h
Indicative price / branch / month	USD 65	USD 129	USD 199–240

2.6.2 Additional Revenue Lines

Line	Model	Notes
Onboarding & setup	One-time USD 150–2,500	Scales with menu/recipe complexity
Additional POS terminal	USD 20/month	
Additional KDS screen	USD 15/month	
Aggregator connector	USD 25/month per aggregator	
Payment processing	Revenue share with acquirer	Highest-margin line long-term
Hardware resale	15–25% margin	Working-capital intensive; consider partner model
Data & benchmarking reports	USD 40/month	Anonymised peer benchmarking; requires customer consent
Franchise royalty module	USD 300/month per brand	Enterprise only

2.6.3 Unit Economics Target

Metric	Target
Average Revenue Per Branch (ARPB)	USD 120/month
Gross margin	≥ 78%

Metric	Target
Customer Acquisition Cost (Segment B)	≤ USD 700
Payback period	≤ 8 months
Gross monthly churn	≤ 2.0%
Net revenue retention	≥ 108%

Risk: Churn in Segment A is structurally high because restaurant failure rates are high. Modelling Segment A churn at Segment B levels produces a business plan that will not survive contact with reality. Segment A should be treated as a low-touch, self-service channel that must be profitable at first-month margin, not at lifetime margin.

2.7 Business Objectives Traceable to Requirements

Objective	Success Metric	Primary Requirements
Make variance computable daily	≥ 80% of Professional-tier tenants run a stock count at least weekly	FR-INV-030..048, FR-CST-010..022
Eliminate connectivity as a failure mode	< 0.1% of business days report a POS outage attributable to connectivity	FR-OFF-001..044, NFR-REL-002
Reduce onboarding friction	Median time from signup to first order ≤ 30 minutes	NFR-USA-003, FR-PLT-020..029
Enable multi-country sale without code change	New country pack certified in ≤ 6 engineer-weeks	FR-LOC-001..038, CR-03
Provide defensible internal control	100% of sensitive actions produce audit entries	FR-AUD-001..026, FR-SEC-018
Make the product sellable to groups	Support ≥ 100 branches per tenant with sub-2s consolidated reporting	NFR-PERF-006, FR-BRN-001..031

Chapter 3 — Stakeholders, Actors & Personas | أصحاب المصلحة

3.1 Actor Catalogue

An actor is any person, role, or external system that interacts with ROS. Actors are the subjects of use cases and the anchor points of the permission model.

3.1.1 Human Actors

Actor ID	Actor	Scope	Primary Interface
ACT-01	Platform Administrator	Cross-tenant (ROS staff)	Admin Console
ACT-02	Tenant Owner	All brands, branches	Web Dashboard

Actor ID	Actor	Scope	Primary Interface
ACT-03	Operations Director	All branches of assigned brands	Web Dashboard
ACT-04	Brand Manager	One brand	Web Dashboard
ACT-05	Branch Manager	One branch	Web Dashboard + POS
ACT-06	Shift Supervisor	One branch, during shift	POS + limited Dashboard
ACT-07	Cashier	One terminal, during shift	POS
ACT-08	Waiter / Server	One branch, assigned tables	POS / Handheld
ACT-09	Kitchen Staff	One station	KDS
ACT-10	Head Chef	All stations, recipes	KDS + Dashboard
ACT-11	Storekeeper	One branch/warehouse inventory	Dashboard + Mobile
ACT-12	Purchasing Officer	Tenant-wide purchasing	Dashboard
ACT-13	Accountant	Financial data, read + export	Dashboard
ACT-14	Auditor	Read-only, all data, audit log	Dashboard
ACT-15	HR Officer	Employee records, attendance	Dashboard
ACT-16	Central Kitchen Manager	CK production and distribution	Dashboard + Production terminal
ACT-17	Franchisee	Own branches only, restricted config	Dashboard + POS
ACT-18	Customer	Self-service	Receipt QR, loyalty portal, QR menu

3.1.2 System Actors

Actor ID	System	Interaction
ACT-20	Payment Terminal	Payment authorisation request/response
ACT-21	Fiscal Authority Gateway	Invoice/receipt submission, UUID return
ACT-22	Delivery Aggregator	Order injection, status callback
ACT-23	Accounting System	Journal entry export

Actor ID	System	Interaction
ACT-24	SMS / WhatsApp Gateway	Notification dispatch
ACT-25	Thermal Printer	Receipt and ticket rendering
ACT-26	Scheduler	Time-triggered jobs (day close, reorder suggestions, report delivery)

3.2 Personas

Personas exist to keep design decisions anchored to real constraints. Each persona below is written as a description of operating conditions, not demographics.

Persona 1 — Mahmoud, Cashier

Twenty-three years old. Works a nine-hour shift. During the 13:00–15:00 rush he processes an order every 40 seconds. His hands are sometimes wet. The screen has fingerprints on it. He is being watched by a queue of eleven people.

What he needs: Fewer than four taps for the most common order. Large touch targets. No confirmation dialogs on the happy path. Immediate visual feedback. The ability to correct a mistake without calling a manager for anything that isn't financially sensitive.

What breaks him: A spinner. A modal that requires reading. A search that requires typing. An item that is three menu levels deep.

Design implication: POS interaction budget is specified as a hard requirement (NFR-USA-001): the median in-store order of ≤3 lines must be completable in ≤6 taps and ≤400ms of cumulative interface latency.

Persona 2 — Amal, Branch Manager

Thirty-four. Runs a 22-seat café with 11 staff. Arrives at 09:00, leaves at 23:00 on bad days. She is the person who counts the fridge, calls the supplier, covers a sick shift, and explains a variance to the owner.

What she needs: To know before noon whether yesterday was good. To be told what to order rather than having to work it out. To find out that the chicken is running low before the kitchen tells her at 20:00. To prove that a variance was waste, not theft.

What breaks her: Reports that require her to build them. Alerts that fire so often she ignores them. Any workflow that requires a laptop when she has a phone in her hand and flour on the counter.

Design implication: Mobile-capable manager workflows (FR-INV-040, FR-PRC-018) and a daily “morning brief” summary (FR-RPT-055).

Persona 3 — Youssef, Group Operations Director

Forty-one. Oversees 14 branches across two brands and two countries. He does not open the POS. He opens a dashboard on Monday morning and needs to know which three branches need his attention this week.

What he needs: Ranking, not listing. Exceptions, not totals. Comparability — a branch with higher revenue but worse food cost must be visible as a problem. Drill-down from anomaly to transaction to employee in under four clicks.

What breaks him: Reports that don't reconcile with the accountant's numbers. Currency confusion in consolidated views. Waiting for a report to generate.

Design implication: Pre-aggregated reporting layer (Chapter 19), branch scorecards (FR-RPT-031), and drill-through to source transaction (FR-RPT-042).

Persona 4 — Sameh, Head Chef

Forty-seven. Runs a kitchen with six stations. He knows his recipes but has never written them down with gram-level precision. He views the KDS as either an aid or an obstacle, with no middle ground.

What he needs: Tickets that arrive in the order he should cook them, not the order they were entered. Clear indication of what is late. The ability to bump an item without a mis-tap wiping a ticket.

What breaks him: A screen that requires him to look for more than one second. Recipe management that demands precision he cannot supply on day one.

Design implication: KDS is designed for glanceability at 2 metres (NFR-USA-006); recipe entry supports progressive precision — an item can be sold before its recipe is complete, with the system flagging incomplete recipes rather than blocking (BR-MNU-012).

Persona 5 — Nadia, Accountant (External)

Fifty-two. Handles books for eleven small businesses. Interacts with ROS twice a month. Cares about one thing: whether the numbers reconcile and whether she can export them.

What she needs: An export that maps to her chart of accounts. Immutable records. A clear audit trail for every adjustment.

Design implication: Export mapping configuration (FR-INT-020) and the immutability constraint CR-04.

3.3 Stakeholder Concerns Matrix

Stakeholder	Primary Concern	Chapters of Interest
Tenant Owner	Profitability visibility, theft prevention	13, 19, 20
Operations Director	Branch comparability, standardisation	17, 19

Stakeholder	Primary Concern	Chapters of Interest
Branch Manager	Daily operation, ordering, staffing	8, 11, 12, 14
Cashier	Speed, reliability	8, 21
Kitchen	Order clarity, timing	9
Accountant	Reconciliation, export, immutability	16, 22, 23
Auditor	Traceability, segregation of duties	15, 20
Regulator	Fiscal compliance	22, 23
ROS Engineering	Maintainability, deployability	5, 24, 25, 29
ROS Support	Diagnosability	20, 29
Investor	Scalability, defensibility, unit economics	2, 30, 31

Chapter 4 — System Scope & Context | نطاق النظام

4.1 Scope Boundary

4.1.1 In Scope

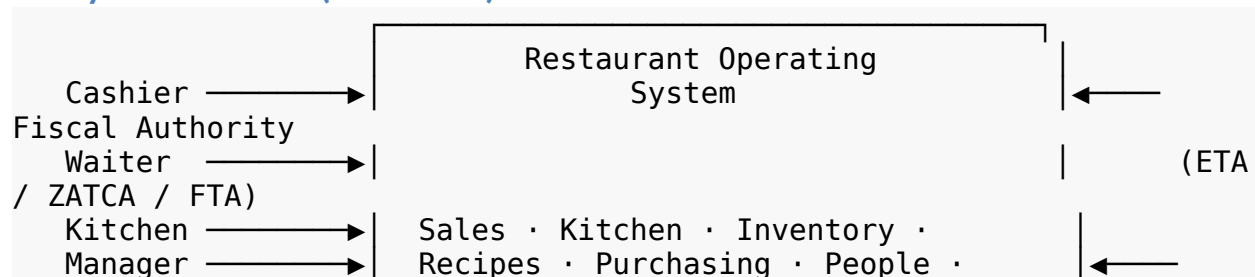
Domain	Included
Sales	Order capture, all order types, payments, receipts, discounts, voids, refunds, table management, split bills
Kitchen	Order routing, station display, timing, bumping, recall, expediting, load balancing
Menu	Categories, items, variants, modifiers, combos, pricing, availability, scheduling
Recipes	BOM, sub-recipes, yield, portion, costing, versioning
Inventory	Multi-location stock, UoM conversion, transfers, counts, batches, expiry, waste, adjustments
Purchasing	Requisition, quotation, PO, receiving, supplier invoice, three-way match, supplier scoring
Costing	Food cost, contribution margin, theoretical vs actual, variance analysis
People	Employees, roles, permissions, scheduling, attendance, overtime, performance

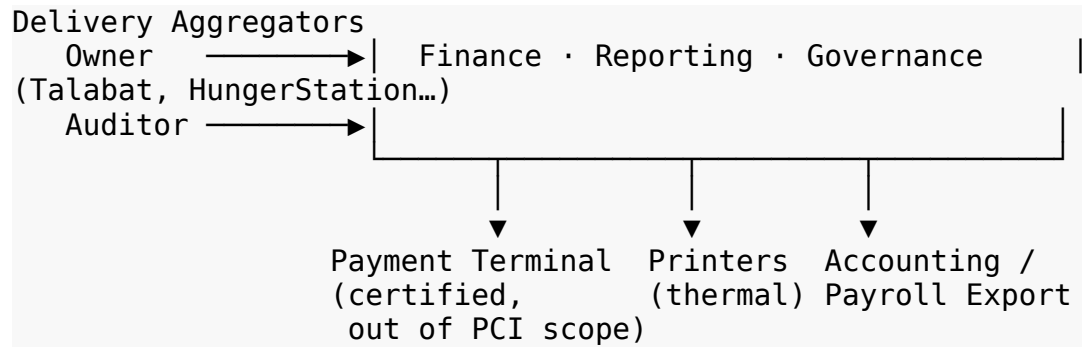
Domain	Included
Finance	Drawers, floats, shifts, variance, expenses, day close, tax computation
Organisation	Tenants, brands, branches, warehouses, central kitchens, transfers, consolidation
Customers	Profiles, order history, loyalty, promotions, segments
Reporting	Operational, financial, inventory, kitchen, employee, executive
Governance	Audit log, security, approvals, segregation of duties
Platform	Offline operation, synchronisation, localisation, country packs, integrations

4.1.2 Explicitly Out of Scope (Phase 1)

Excluded	Rationale	Reconsider At
Full general ledger accounting	Competing with accounting systems is a distraction; export is sufficient	Phase 4, enterprise demand
Payroll calculation	Jurisdiction-specific labour law is a product in itself	Phase 4
Own delivery driver dispatch & routing	Distinct problem domain; partner instead	Phase 3
Customer-facing ordering app (white label)	Aggregators dominate; low differentiation	Phase 3
Table reservation & waitlist	Adjacent, integrable	Phase 2
Hardware manufacturing	Capital intensive	Never
In-house payment processing / acquiring	Regulatory burden, PCI scope	Partner-only
Nutrition and allergen certification	Liability; provide data fields only, not certification	Phase 3 (data fields in Phase 2)
Franchise legal/contract management	Not software-solvable at our scale	Never

4.2 System Context (C4 Level 1)





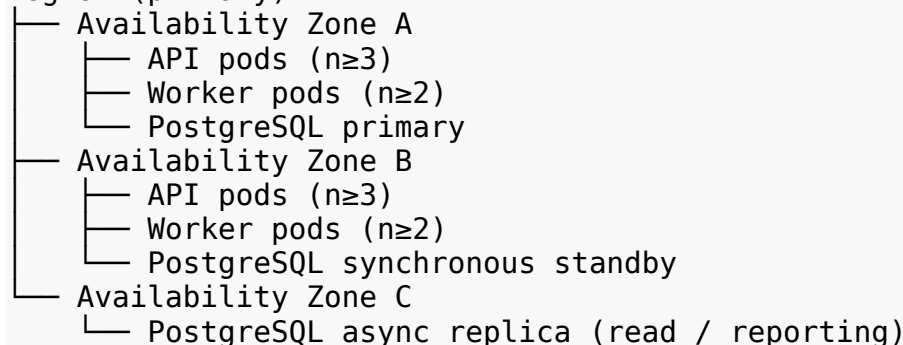
4.3 Container View (C4 Level 2)

Container	Technology	Responsibility	Deployment
POS Application	Flutter (Android/iOS/Windows)	Order capture, payment, printing, offline store	On-premise device
KDS Application	Flutter (Android/Web)	Ticket display, bumping, timing	On-premise device
Management Web App	Next.js + React + TypeScript	Configuration, reporting, back-office	Browser (CDN + SSR)
Mobile Manager App	Flutter	Approvals, alerts, stock count, quick reports	Device
Core API	NestJS + TypeScript	All business logic, modular monolith	Kubernetes / container platform
Sync Service	NestJS (dedicated module + workers)	Offline reconciliation, conflict resolution	Same cluster, isolated pool
Job Workers	BullMQ on Node	Async work: reports, fiscal submission, exports, notifications	Same cluster, separate pool
Reporting Engine	PostgreSQL materialised views + ClickHouse (Phase 3)	Pre-aggregation, analytics queries	Managed
Primary Datastore	PostgreSQL 16	Transactional data, source of truth	Managed, Multi-AZ
Cache / Queue	Redis 7	Session, cache, rate limiting, job queue	Managed
Object Storage	S3-compatible	Images, exports, invoice PDFs, backups	Managed
Search	PostgreSQL FTS → OpenSearch (Phase 3)	Product/customer/document search	Managed
Local POS Store	SQLite + Drift	Offline transactional	On-device

Container	Technology	Responsibility	Deployment
		store on device	

4.4 Deployment Topology

Region (primary)



Edge: CDN for web app assets + image delivery

Branch: POS/KDS devices on local Wi-Fi with local print server

Regional deployments are separate stacks (EG, KSA, UAE, EU) to satisfy data residency requirements. Tenants are pinned to a region at creation and cannot silently migrate. Cross-region reporting for multinational tenants is handled by an aggregation service that reads regional read-replicas and never moves raw transactional data across regions.

Chapter 5 — Architecture | معمارية النظام

5.1 Architectural Drivers

Architecture is the set of decisions that are expensive to reverse. The following drivers, in priority order, determine ROS's architecture. When two drivers conflict, the higher-priority one wins, and the trade-off is recorded.

Rank	Driver	Consequence
1	The POS must never stop	Offline-first client; server is not on the critical path for a sale
2	Financial and inventory data must be correct	Strong transactional consistency in the core; no eventual consistency for money or stock
3	Tenant data must be isolated	Enforced isolation at the data layer, not the application layer alone
4	A 9-engineer team must ship in 9 months	Modular monolith; boring technology; one language

Rank	Driver	Consequence
5	Country rules must change without deployment	where possible Externalised rule configuration; strategy pattern at every jurisdiction touchpoint
6	Reporting must not degrade transactions	Read/write separation; pre-aggregation; separate connection pools
7	Any module must be extractable later	Explicit module boundaries; no cross-module table access; in-process message bus with a network-ready contract

5.2 The Modular Monolith Decision

5.2.1 Decision

The Core API is built as a **modular monolith**: a single deployable artefact composed of modules with enforced boundaries, communicating through explicit interfaces and an in-process event bus.

5.2.2 Why Not Microservices

Microservices solve organisational scaling problems (many teams needing independent deployment) at the cost of severe operational and correctness complexity (distributed transactions, network partitions, versioned contracts, distributed tracing, service discovery, saga orchestration).

ROS has nine engineers. It does not have the organisational problem microservices solve. It does, however, have a correctness problem microservices would make dramatically harder: a single sale must atomically update sales, inventory, cost, cash, tax, and audit. In a monolith this is one database transaction. In a microservice architecture it is a saga with compensating transactions, and every failure mode of that saga becomes an inventory discrepancy that a restaurant owner will interpret as theft.

Rationale: Distributed systems are a tax paid in correctness. Pay it only when the organisational benefit exceeds it. At nine engineers it does not.

5.2.3 What “Modular” Requires

A monolith without discipline becomes a big ball of mud, at which point extraction is impossible and the decision above becomes a trap. The following rules are enforced mechanically, not by convention:

Rule	Enforcement
A module MUST NOT import from another	ESLint boundary rule + CI failure

Rule	Enforcement
module's internal directory	
A module MUST NOT query another module's tables	Per-module DB role with table-level grants in CI verification
Cross-module communication is via a published interface or a domain event	Architecture test suite
Every module publishes a versioned contract in <code>modules/<name>/contract/</code>	Code review checklist + CI check for contract file presence
Shared code lives in <code>shared/</code> and MUST NOT contain business logic	Static analysis rule (no imports of domain types in <code>shared/</code>)
Database migrations are owned by exactly one module	Migration file naming convention + CI ownership check

5.2.4 Extraction Path

When a module must become a service (candidates, in likely order: Reporting, Sync, Fiscal Integration, Notification):

1. The module already communicates only via its contract and events.
2. Replace the in-process event bus binding for that module with a network transport (message broker).
3. Replace direct interface calls with an HTTP/gRPC client that implements the same interface.
4. Split the schema; the module's tables move with it.
5. Deploy separately.

Because steps 1 and 2 are true from day one, extraction is a matter of days rather than a rewrite.

5.3 Bounded Contexts

Modules correspond to Domain-Driven Design bounded contexts. Within a context, a term has one unambiguous meaning; across contexts, the same word may mean different things, and translation happens explicitly at the boundary.

Context	Core Concern	Key Aggregates	Ubiquitous Language Notes
Identity & Tenancy	Who exists, who they are	Tenant, User, Session, Terminal	"User" = a login. Distinct from "Employee".
Organisation	Physical and legal structure	Brand, Branch, Warehouse, CentralKitchen	"Location" is abstract; Branch/Warehouse are concrete.
Catalogue	What can be sold	MenuItem, Category, Modifier, Combo,	"Item" here means <i>sellable</i> . In Inventory,

Context	Core Concern	Key Aggregates	Ubiquitous Language Notes
		PriceList	“Item” means <i>stockable</i> . Translation required.
Production Spec	How things are made	Recipe, SubRecipe, YieldProfile	“Ingredient” is a link to an Inventory Item, not a copy.
Sales	Commercial transactions	Order, OrderLine, Payment, Refund, Discount	“Order” is a commercial agreement, not a kitchen task.
Kitchen Ops	Producing the order	Ticket, Station, PrepTask	“Ticket” is the kitchen’s view of an Order — different lifecycle.
Inventory	Physical goods	StockItem, StockLevel, Batch, Movement, CountSession	“Item” here means <i>stockable</i> .
Procurement	Acquiring goods	Requisition, PurchaseOrder, GoodsReceipt, SupplierInvoice, Supplier	
Costing	Money consumed	CostSnapshot, VarianceReport, WasteRecord	
Workforce	People at work	Employee, Shift, Schedule, AttendanceRecord	“Employee” is a person in a job. Distinct from “User”.
Treasury	Cash and settlement	Drawer, CashSession, Expense, DayClose	
Customer	Demand-side relationships	Customer, LoyaltyAccount, Promotion	
Fiscal	Legal compliance of documents	TaxDocument, CountryPack, SubmissionRecord	
Analytics	Derived understanding	Fact tables, Dimensions, Scorecards	Read-only. Never source of truth.
Governance	Accountability	AuditEntry,	

Context	Core Concern	Key Aggregates	Ubiquitous Language Notes
		ApprovalRequest, Policy	
Sync	Distributed consistency	SyncBatch, DeviceState, ConflictRecord	

5.3.1 Context Map

Identity&Tenancy —(upstream, conformist)→ every context

Catalogue —(shared kernel: ItemId)→ Production Spec
 Production Spec —(customer/supplier)→ Inventory [recipe consumes stock items]
 Sales —(published events)→ Kitchen Ops
 Sales —(published events)→ Inventory [depletion]
 Sales —(published events)→ Costing
 Sales —(published events)→ Treasury
 Sales —(published events)→ Fiscal
 Sales —(published events)→ Customer
 Procurement —(customer/supplier)→ Inventory [receipt increases stock]
 Inventory —(published events)→ Costing
 Workforce —(published events)→ Treasury [shift → cash session]
 All contexts —(published events)→ Governance [audit]
 All contexts —(published events)→ Analytics [fact ingestion]

Relationship types used:

- *Conformist*: the downstream context accepts the upstream model as-is (all contexts to Identity).
- *Customer/Supplier*: the downstream context's needs influence the upstream contract (Inventory influences Production Spec).
- *Shared Kernel*: a small, jointly-owned model (identifiers and money value objects).
- *Anti-Corruption Layer*: used for all external integrations without exception.

5.4 Layered Architecture Within a Module

Every module has the same internal shape. Uniformity here is worth more than local optimisation, because it means any engineer can navigate any module.

```
modules/<context>/
├─ contract/          ← PUBLIC. Other modules may import only
this.
├─ events.ts          ← domain events published by this module
├─ commands.ts        ← commands this module accepts
├─ queries.ts         ← read interfaces exposed
└─ types.ts           ← DTOs crossing the boundary
```

— domain/	← PRIVATE. Pure business logic. Zero
framework imports.	
— entities/	
— value-objects/	
— aggregates/	
— events/	
— policies/	← business rules as first-class objects
— services/	← domain services (logic spanning
aggregates)	
— errors/	
— application/	← PRIVATE. Orchestration. Use cases.
— commands/	← command handlers (write)
— queries/	← query handlers (read)
— ports/	← interfaces the domain needs
(repositories, gateways)	
— sagas/	← long-running process coordination
— infrastructure/	← PRIVATE. Adapters implementing ports.
— persistence/	← repository implementations, ORM mappings
— messaging/	← event bus adapters
— external/	← third-party clients wrapped in ACLs
— migrations/	
— presentation/	← PRIVATE. Delivery mechanisms.
— http/	← controllers, DTO validation
— graphql/	← (Phase 3)
— jobs/	← scheduled/queued entry points

Dependency rule (Clean Architecture): dependencies point inward only. presentation → application → domain. infrastructure → application (implementing ports) and infrastructure → domain (mapping entities). The domain layer imports nothing from the outer layers, and imports no framework code whatsoever.

Rationale: The domain layer contains the rules that make the product correct — how a recipe expands, how variance is computed, how tax is applied. These rules must be testable without a database, without HTTP, and without mocking a framework. A domain layer that requires infrastructure to test will not be adequately tested, and the rules that matter most will be the ones least verified.

5.5 Communication Patterns

5.5.1 Synchronous — Direct Interface Call

Used when the caller requires the result to proceed and the operation must be in the same transaction.

Example: Sales must know an item's current price and tax class before it can create an order line. It calls `CatalogueQueryPort.getPricedItem(itemId, branchId, at)`.

5.5.2 Asynchronous In-Transaction — Domain Events

Used when a state change must cause other state changes atomically. Events are collected on the aggregate, and dispatched by the unit of work *within the same database transaction*.

Example: `OrderCompleted` causes inventory depletion, COGS recognition, and cash posting. All four must succeed or all must fail. There is no acceptable state in which a sale is recorded and inventory is not depleted.

5.5.3 Asynchronous Out-of-Transaction — Transactional Outbox

Used when a state change must cause an effect outside the database, which cannot participate in the transaction.

Example: fiscal submission to the tax authority, sending a receipt SMS, pushing an order to a delivery aggregator, invalidating a CDN cache.

The outbox pattern: the effect is recorded as a row in an outbox table inside the same transaction as the state change. A relay process reads unprocessed outbox rows and performs the effect, marking success. This guarantees at-least-once delivery with no dual-write inconsistency.

```
BEGIN;  
  INSERT INTO orders ...;  
  INSERT INTO stock_movements ...;  
  INSERT INTO outbox (topic, payload, ...) VALUES ('fiscal.submit',  
{...});  
COMMIT;  
-- Relay picks up outbox row, calls tax authority, marks processed
```

Risk: Without the outbox, a common implementation is: commit the transaction, then call the external service. If the process dies between those steps, the sale exists but was never submitted to the tax authority — a compliance breach that is invisible until an audit. The outbox eliminates this class of bug entirely and is therefore mandatory (FR-PLT-041).

5.5.4 Event Catalogue (Core Subset)

Event	Publisher	Principal Subscribers
<code>order.opened</code>	Sales	Kitchen Ops, Analytics
<code>order.line.fired</code>	Sales	Kitchen Ops
<code>order.line.voided</code>	Sales	Kitchen Ops, Inventory, Governance
<code>order.completed</code>	Sales	Inventory, Costing, Treasury, Fiscal, Customer, Analytics
<code>order.refunded</code>	Sales	Inventory, Costing, Treasury, Fiscal, Governance
<code>discount.applied</code>	Sales	Governance, Analytics
<code>ticket.bumped</code>	Kitchen Ops	Sales, Analytics

Event	Publisher	Principal Subscribers
stock.received	Procurement	Inventory, Costing
stock.moved	Inventory	Costing, Analytics
stock.counted	Inventory	Costing, Governance
waste.recorded	Inventory	Costing, Governance, Analytics
recipe.version.published	Production Spec	Costing, Catalogue
shift.opened / shift.closed	Workforce	Treasury, Analytics
cash.variance.detected	Treasury	Governance, Analytics
purchase_order.approved	Procurement	Governance, Analytics
day.closed	Treasury	Analytics, Fiscal, Reporting
sync.conflict.resolved	Sync	Governance

Every event carries a mandatory envelope:

```
{
  "eventId": "01J8XZ...",           // ULID, globally unique
  "eventType": "order.completed",
  "eventVersion": 2,
  "occurredAt": "2026-08-04T11:02:33.412Z",
  "recordedAt": "2026-08-04T11:02:33.610Z",
  "tenantId": "...",
  "branchId": "...",
  "actorId": "...",
  "actorType": "user|system|device",
  "correlationId": "...",           // ties an entire causal chain
  "causationId": "...",            // the event/command that caused
  this
  "idempotencyKey": "...",
  "payload": { }
}
```

Rationale: correlationId and causationId are what make production incidents diagnosable. Given a customer complaint about an incorrect stock level, an engineer must be able to trace backwards from the stock movement to the order to the terminal to the cashier to the sync batch. Without causation chaining, that investigation takes hours; with it, it takes one query.

5.6 Architecture Decision Records

ADR-001 — Modular Monolith over Microservices

Status: Accepted · **Date:** 2026-06-08

Context: Nine engineers, nine-month deadline, transactional correctness requirements spanning six domains.

Decision: Single deployable Core API with enforced module boundaries.

Consequences: *Positive* — atomic cross-domain transactions; single deployment; simple local development; low operational cost. *Negative* — the whole API scales as one unit; a memory leak in reporting affects order processing (mitigated by separate worker pools and connection pools); requires active boundary enforcement or degrades to a mud ball.

Revisit when: Engineering exceeds ~30 people, or a single module's scaling profile diverges by more than 5× from the rest.

ADR-002 — PostgreSQL as Primary Datastore

Status: Accepted · **Date:** 2026-06-08

Context: Need transactional integrity, complex relational queries, JSON flexibility for country packs and modifiers, full-text search, and time-series-ish reporting, without operating four datastores.

Decision: PostgreSQL 16 for all transactional and initial analytical workloads.

Alternatives rejected: MongoDB (no multi-document ACID guarantees at the required granularity in earlier versions; schema drift risk in financial data); MySQL (weaker JSON, weaker CTEs, no partial indexes historically); a document store plus a relational store (two consistency models is one too many).

Consequences: Analytical queries will eventually contend with transactional load; mitigated by read replicas and materialised views, and by introducing ClickHouse in Phase 3 when the reporting volume justifies it.

ADR-003 — Shared Database with Row-Level Tenant Isolation

Status: Accepted · **Date:** 2026-06-14

Context: Multi-tenant SaaS with tenants from 1 to 200+ branches. Isolation is a hard requirement; operational simplicity is a survival requirement.

Decision: Shared schema with a mandatory `tenant_id` column on every tenant-scoped table, enforced by PostgreSQL Row-Level Security policies, with a schema-per-tenant escape hatch for Enterprise tenants who contractually require it.

Consequences: Isolation depends on RLS being correctly configured and session context being reliably set — this is a critical risk requiring dedicated testing (see 6.4). Migrations are simple (one schema). Noisy-neighbour risk requires per-tenant rate limiting.

ADR-004 — Offline-First POS with Local SQLite

Status: Accepted · **Date:** 2026-06-21

Decision: POS holds a full local transactional store; server is a synchronisation target, not a dependency for sales.

Consequences: Business logic that must run offline is duplicated between the Dart client and the TypeScript server — a genuine and permanent cost. Mitigation: the duplicated logic is confined to a narrow, precisely specified surface (pricing, tax computation, recipe expansion, discount rules), and is validated by a shared, language-neutral conformance test suite in JSON (see 21.9).

ADR-005 — Country Rules as Data, Not Code

Status: Accepted · **Date:** 2026-06-27

Decision: Tax computation, invoice formatting, rounding, and fiscal submission are driven by versioned Country Pack configuration objects plus a small set of registered strategy implementations.

Consequences: Country packs need their own validation, testing, and release process. A malformed pack could produce incorrect tax on live sales — therefore packs are versioned, digitally signed, and validated against a conformance suite before activation (FR-LOC-031).

ADR-006 — Event-Driven Internal Integration with Transactional Outbox

Status: Accepted · **Date:** 2026-07-02

Decision: In-process domain events inside the transaction boundary; transactional outbox for all external effects.

Consequences: All external effects are at-least-once, so all external integrations must be idempotent. Consumers must be written accordingly; this is a standing engineering rule, not an optional practice.

ADR-007 — Flutter for POS and KDS

Status: Accepted · **Date:** 2026-07-05

Context: POS runs on Android tablets, Windows all-in-ones, and occasionally iPads. KDS runs on Android displays and browsers.

Decision: Flutter for both.

Alternatives rejected: Native per platform (3× cost with 9 engineers); React Native (weaker offline database story with Drift/SQLite equivalents, weaker rendering consistency on low-end Android hardware, which is the majority of the target device fleet); web-only PWA (offline printing and cash-drawer control are unreliable).

Consequences: Flutter/Dart is a second language in the stack. Accepted. Printer and drawer integration requires platform channels — a known cost, budgeted.

ADR-008 — Money as Integer Minor Units

Status: Accepted · **Date:** 2026-06-10

Decision: All monetary amounts are stored and computed as 64-bit integers in the currency's minor unit, paired with an ISO-4217 currency code. Floating point is prohibited for money at every layer, including the client and the reporting layer.

Consequences: Currencies with non-decimal or zero-decimal minor units (KWD/BHD/OMR: 3 decimals; JPY: 0) require the exponent to be carried with the currency, not assumed to be 2. Percentage-based computations (tax, discount, service charge) require an explicit, specified rounding policy — see 16.7.

ADR-009 — ULID for Distributed Identifiers

Status: Accepted · **Date:** 2026-06-21

Context: Offline devices must generate identifiers for entities before the server has ever seen them. Sequential integers are impossible; UUIDv4 destroys index locality on high-write tables.

Decision: ULIDs (128-bit, lexicographically sortable, timestamp-prefixed), stored as BYTEA(16) or native UUID.

Consequences: Identifiers leak approximate creation time — acceptable, and in fact useful for debugging. Index locality is preserved because ULIDs are time-ordered, avoiding the write amplification of random UUIDs on B-tree indexes.

ADR-010 — Append-Only Financial and Inventory Records

Status: Accepted · **Date:** 2026-06-30

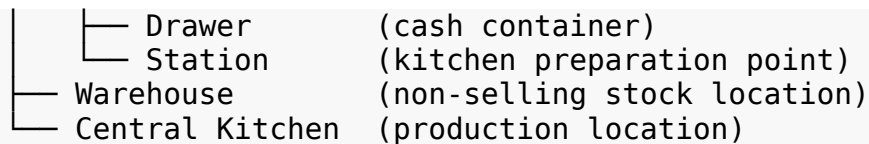
Decision: Orders, payments, stock movements, and audit entries are never updated or deleted. Corrections are new records that reference the original.

Consequences: Tables grow monotonically — requires partitioning strategy (Chapter 25). Reads that need “current state” require either aggregation or a maintained projection (stock_levels is a projection over stock_movements), and the projection must be reconcilable against the ledger by a scheduled job (FR-INV-051).

Chapter 6 — Multi-Tenancy & Platform | تعدد المستأجرين

6.1 Tenancy Hierarchy

Platform	
└─ Tenant	(commercial customer; billing and
isolation boundary)	
└─ Brand	(restaurant concept; menu and identity
boundary)	
└─ Branch	(selling location; inventory and cash
boundary)	
└─ Terminal	(POS/KDS device)



FR-PLT-001 [M] — The System SHALL model tenants, brands, branches, warehouses, and central kitchens as distinct entities with the hierarchy above.

FR-PLT-002 [M] — A tenant SHALL be able to own multiple brands, and a brand SHALL be able to own multiple branches.

FR-PLT-003 [M] — Every tenant-scoped record SHALL carry an immutable `tenant_id`. Records SHALL NOT be transferable between tenants.

FR-PLT-004 [S] — A branch MAY be reassigned between brands within the same tenant by a Tenant Owner, with a full audit record and an explicit warning regarding menu and pricing implications.

BR-PLT-001 — Inventory is held at a location (branch, warehouse, or central kitchen), never at brand or tenant level. Two brands operating from the same physical kitchen must be modelled as two branches sharing a warehouse, not as one location with two brands.

Rationale: Cloud kitchens routinely run 5–8 virtual brands from one kitchen. If inventory were held at brand level, a shared bag of onions would have to be split five ways, which is fictional. Holding stock at location and attributing consumption to brands via recipes is the only model that reflects physical reality.

6.2 Isolation Model

6.2.1 Data Isolation

FR-PLT-010 [M] — The System SHALL enforce tenant data isolation at the database layer using PostgreSQL Row-Level Security, independent of application-layer filtering.

Every tenant-scoped table carries:

```
tenant_id UUID NOT NULL REFERENCES tenants(id)
```

and a policy:

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;  
ALTER TABLE orders FORCE ROW LEVEL SECURITY;
```

```
CREATE POLICY tenant_isolation ON orders  
  USING (tenant_id = current_setting('app.tenant_id')::uuid)  
  WITH CHECK (tenant_id = current_setting('app.tenant_id')::uuid);
```

The application sets the session variable at the start of every request, inside the transaction:

```
SET LOCAL app.tenant_id = '...';
```

FR-PLT-011 [M] — The application database role SHALL NOT have BYPASSRLS. A separate, restricted migration/admin role SHALL be used for schema changes and platform-level operations.

FR-PLT-012 [M] — Any request that reaches the data layer without a resolved tenant context SHALL fail closed with an error, never defaulting to an unfiltered query.

Risk: The catastrophic failure mode of shared-schema multi-tenancy is cross-tenant data leakage. Every reported incident of this class in the industry traces to the same root cause: application-layer filtering that was omitted on one query path. RLS as a second, independent layer means an omitted WHERE `tenant_id = ?` returns zero rows rather than another customer's data. This defence-in-depth is not optional and is verified by the test suite in 6.4.

6.2.2 Isolation Testing

FR-PLT-013 [M] — The CI pipeline SHALL execute a cross-tenant isolation test suite that, for every table containing `tenant_id`, attempts to read and write records belonging to Tenant B while the session context is Tenant A, and fails the build on any success.

This suite is generated, not hand-written — it enumerates tables from the information schema so that a newly added table without a policy fails the build automatically.

FR-PLT-014 [M] — The CI pipeline SHALL fail if any table with a `tenant_id` column lacks an enabled and forced RLS policy.

6.2.3 Compute and Rate Isolation

FR-PLT-015 [M] — The System SHALL apply per-tenant rate limits on API requests, report generation, and export jobs.

Resource	Starter	Professional	Enterprise
API requests / minute	300	1,200	6,000
Concurrent report generations	1	3	10
Export rows / day	100,000	1,000,000	10,000,000
Sync batch size	500 ops	2,000 ops	5,000 ops
Webhook deliveries / hour	0	5,000	50,000

FR-PLT-016 [M] — Long-running analytical queries SHALL execute against a read replica using a separate connection pool, so that a heavy report cannot exhaust the transactional pool.

FR-PLT-017 [S] — The System SHALL apply a statement timeout of 5 seconds on the transactional pool and 120 seconds on the analytical pool.

6.2.4 Enterprise Isolation Tier

FR-PLT-018 [C] — For Enterprise tenants with contractual isolation requirements, the System SHALL support deployment into a dedicated schema or a dedicated database instance, using identical application code and migrations.

6.3 Tenant Lifecycle

State	Meaning	System Behaviour
provisioning	Being created	No login permitted
trial	Evaluation period	Full features, watermarked receipts, 30-day limit
active	Paying	Full features per plan
past_due	Payment failed	Full features, in-app banner, grace period 14 days
restricted	Grace expired	POS read-only: existing orders closable, no new orders; full data export available
suspended	Prolonged non-payment or ToS violation	Login blocked; data retained; export available on request
terminating	Cancellation requested	Read-only; export enabled; 30-day countdown
purged	Data destroyed	Only billing records and audit metadata retained

FR-PLT-020 [M] — Tenant self-service signup SHALL create a working branch with a starter menu template, a default role set, and a first user in a single flow.

FR-PLT-021 [M] — The System SHALL NOT delete tenant data upon downgrade or suspension. Data beyond the plan's limits SHALL become read-only rather than being destroyed.

FR-PLT-022 [M] — At any lifecycle state except purged, a tenant SHALL be able to export all of their data in machine-readable format (CSV per entity plus a JSON manifest) within 24 hours of request.

FR-PLT-023 [M] — Termination SHALL be a two-step process: an initiation that begins a 30-day reversible countdown, and a final confirmation. Data purge SHALL occur no earlier than 30 days after initiation.

Rationale: Requirement FR-PLT-021 is a commercial decision expressed as an engineering requirement. Restaurants have seasonal cash-flow problems. A system that destroys a lapsed customer's data guarantees they never return. A system that preserves it converts a churned customer into a dormant one, and dormant customers reactivate at rates between 8% and 15%.

6.4 Configuration Hierarchy and Resolution

Settings resolve through a cascade. A value set at a lower level overrides a higher level, unless the higher level marks it locked.

```
Platform Default
  ↓ overridden by
Country Pack
  ↓ overridden by
Tenant Setting
  ↓ overridden by
Brand Setting
  ↓ overridden by
Branch Setting
  ↓ overridden by
Terminal Setting
```

FR-PLT-025 [M] — The System SHALL implement a hierarchical settings resolver with the precedence above.

FR-PLT-026 [M] — A setting SHALL be markable as locked at any level, preventing override at lower levels. Locked settings SHALL be visibly indicated in the UI at lower levels with the locking level named.

FR-PLT-027 [S] — The System SHALL provide a settings inspector showing, for any effective value, which level supplied it and what the value would be at each level.

Rationale: The most common support ticket in configurable multi-level systems is “why is this price/tax/permission different at this branch?” A settings inspector converts that ticket from a 30-minute investigation into a 10-second answer, and it is one of the highest-ROI features in the entire back office.

FR-PLT-028 [M] — Settings that affect financial computation (tax class, rounding policy, service charge) SHALL be versioned with effective dates, and historical transactions SHALL be interpreted with the setting version in force at their transaction time, never the current version.

Chapter 7 — Domain Model | نموذج المجال

7.1 Modelling Principles

Aggregates. An aggregate is a cluster of entities with one root, treated as a single unit of consistency. Rules:

1. Reference other aggregates by identity only, never by object reference.
2. One transaction modifies one aggregate. Cross-aggregate consistency is achieved through domain events, within the same transaction where atomicity is required.
3. Invariants that must always hold are enforced inside the aggregate boundary. Invariants that may be briefly violated live between aggregates.

4. Keep aggregates small. A large aggregate causes lock contention and is usually a sign that a consistency boundary has been drawn too widely.

Value Objects. Immutable, compared by value, no identity. Money, Quantity, UnitOfMeasure, DateRange, TaxRate, Address, PhoneNumber, PercentageDiscount.

Entities. Have identity that persists across state changes.

7.2 Shared Kernel Value Objects

Money

```
final class Money {  
    private constructor(  
        readonly amount: bigint,          // minor units, e.g. 1250 = 12.50  
EGP        readonly currency: Currency,  // ISO 4217 + exponent  
    ) {}  
  
    static of(amount: bigint, currency: Currency): Money;  
    static zero(currency: Currency): Money;  
  
    plus(other: Money): Money;           // throws CurrencyMismatchError  
    minus(other: Money): Money;  
    times(factor: Decimal, rounding: RoundingMode): Money;  
    allocate(ratios: Decimal[]): Money[]; // penny-safe distribution  
    isZero(): boolean;  
    isNegative(): boolean;  
}
```

BR-CORE-001 — Arithmetic between different currencies SHALL raise an error. There is no implicit conversion.

BR-CORE-002 — `allocate` SHALL distribute an amount across ratios such that the sum of the results exactly equals the input, distributing remainder minor units to the largest ratios first.

Rationale: `allocate` exists because splitting a bill of 100.00 three ways produces $33.33 + 33.33 + 33.33 = 99.99$. One minor unit vanishes. Over a year of split bills this produces a reconciliation discrepancy the accountant cannot explain. Allocation must be exact by construction, not by rounding.

Quantity

```
final class Quantity {  
    private constructor(  
        readonly value: Decimal,          // arbitrary precision, 6 dp  
        readonly unit: UnitOfMeasure,  
    ) {}  
  
    convertTo(target: UnitOfMeasure, ctx: ConversionContext): Quantity;  
    plus(other: Quantity): Quantity;     // converts if compatible, else
```



```
throws
    times(factor: Decimal): Quantity;
}
```

BR-CORE-003 — Quantities SHALL be stored with 6 decimal places of precision.

Rationale: A recipe may specify 0.5 g of saffron per portion. At 4 dp this is representable; at 2 dp it rounds to zero and the ingredient silently disappears from costing. Six decimals covers every realistic culinary case including yeast, spices, and food colouring.

BR-CORE-004 — Conversion between units of different dimensions (mass ↔ volume) SHALL require an item-specific density factor and SHALL fail if none is configured for that item.

UnitOfMeasure

Dimension	Base Unit	Derived Units
Mass	gram	kilogram, milligram, pound, ounce
Volume	millilitre	litre, centilitre, gallon, fluid ounce
Count	piece	dozen, case, pack, tray
Length	millimetre	centimetre, metre
Time	second	minute, hour

Item-specific packaging units (e.g. “carton of 12 × 1L”) are modelled as `PackagingUnit` with a conversion factor to the item’s base unit, defined per item and optionally per supplier.

7.3 Aggregate Catalogue

#	Aggregate Root	Context	Contained Entities	Key Invariants
1	Tenant	Identity	Subscription, TenantSettings	Unique slug; exactly one owner user
2	User	Identity	Credential, MfaEnrolment, Session	Unique email within tenant
3	Terminal	Identity	DeviceFingerprint, LicenceAssignment	Bound to exactly one branch
4	Brand	Organisation	BrandTheme, DefaultSettings	Belongs to one tenant
5	Branch	Organisation	OperatingHours, Stations, PrintRouting	Belongs to one brand; one timezone; one

#	Aggregate Root	Context	Contained Entities	Key Invariants
6	Warehouse	Organisation	—	base currency Belongs to one tenant
7	MenuItem	Catalogue	Variants, ModifierGroupLinks, Images	≥1 variant; every variant priced in every active price list
8	ModifierGroup	Catalogue	Modifiers	min ≤ max; if required then min ≥ 1
9	Combo	Catalogue	ComboSlots, SlotOptions	Every slot has ≥1 option
10	PriceList	Catalogue	PriceEntries, ValidityWindow	No overlapping windows of same priority for same scope
11	Recipe	Production Spec	RecipeLines, YieldProfile, Version	No circular sub-recipe references; yield > 0
12	StockItem	Inventory	UomConversions, StorageRequirements, SupplierLinks	Exactly one base unit
13	StockLevel	Inventory	BatchAllocations	Projection; reconcilable to movements
14	Batch	Inventory	—	Expiry ≥ production date; quantity ≥ 0
15	StockMovement	Inventory	—	Immutable; signed quantity; balanced counterpart for transfers
16	CountSession	Inventory	CountLines, Recount	Cannot post twice; blind-count option locks expected qty

#	Aggregate Root	Context	Contained Entities	Key Invariants
17	Supplier	Procurement	Contacts, PriceListEntries, PaymentTerms	Unique code per tenant
18	PurchaseRequisition	Procurement	RequisitionLines	≥1 line
19	PurchaseOrder	Procurement	POLines, ApprovalChain	Approved before sending; total matches lines
20	GoodsReceipt	Procurement	ReceiptLines, BatchAssignments	Cannot exceed PO qty beyond tolerance
21	SupplierInvoice	Procurement	InvoiceLines, MatchResult	Three-way match before payment approval
22	Order	Sales	OrderLines, LineModifiers, Discounts, Payments, ServiceCharges	Total = Σ lines + charges – discounts + tax; state machine enforced
23	Ticket	Kitchen Ops	TicketLines	Derived from Order; independent lifecycle
24	Station	Kitchen Ops	RoutingRules, CapacityConfig	Belongs to one branch
25	Employee	Workforce	EmploymentTerms, Certifications	May link to at most one User
26	Schedule	Workforce	ScheduledShifts	No overlapping shifts for one employee
27	AttendanceRecord	Workforce	ClockEvents, BreakPeriods	Clock-out after clock-in
28	CashSession	Treasury	CashMovements, Denominations, Reconciliation	One open session per drawer
29	DayClose	Treasury	SessionSummaries, VarianceReport	All sessions closed before day close
30	Expense	Treasury	Attachments,	Requires

#	Aggregate Root	Context	Contained Entities	Key Invariants
			ApprovalChain	approval above threshold
31	WasteRecord	Costing	WasteLines	Requires reason; approval above value threshold
32	Customer	Customer	Addresses, LoyaltyAccount, Consents	Unique phone within tenant
33	Promotion	Customer	Conditions, Effects, UsageLimits	Cannot stack unless explicitly permitted
34	CountryPack	Fiscal	TaxRules, InvoiceTemplate, FiscalConfig	Signed and versioned
35	TaxDocument	Fiscal	DocumentLines, SubmissionAttempts	Immutable after submission
36	ApprovalRequest	Governance	Steps, Decisions	Requester ≠ approver
37	AuditEntry	Governance	—	Append-only; hash-chained
38	ProductionOrder	Central Kitchen	ProductionLines, OutputBatches	Output ≤ theoretical yield + tolerance
39	DistributionOrder	Central Kitchen	DistributionLines	Sum of distributions ≤ produced quantity
40	SyncBatch	Sync	Operations, ConflictRecords	Idempotent by batch key

7.4 Core Entity Specifications

7.4.1 Order Aggregate

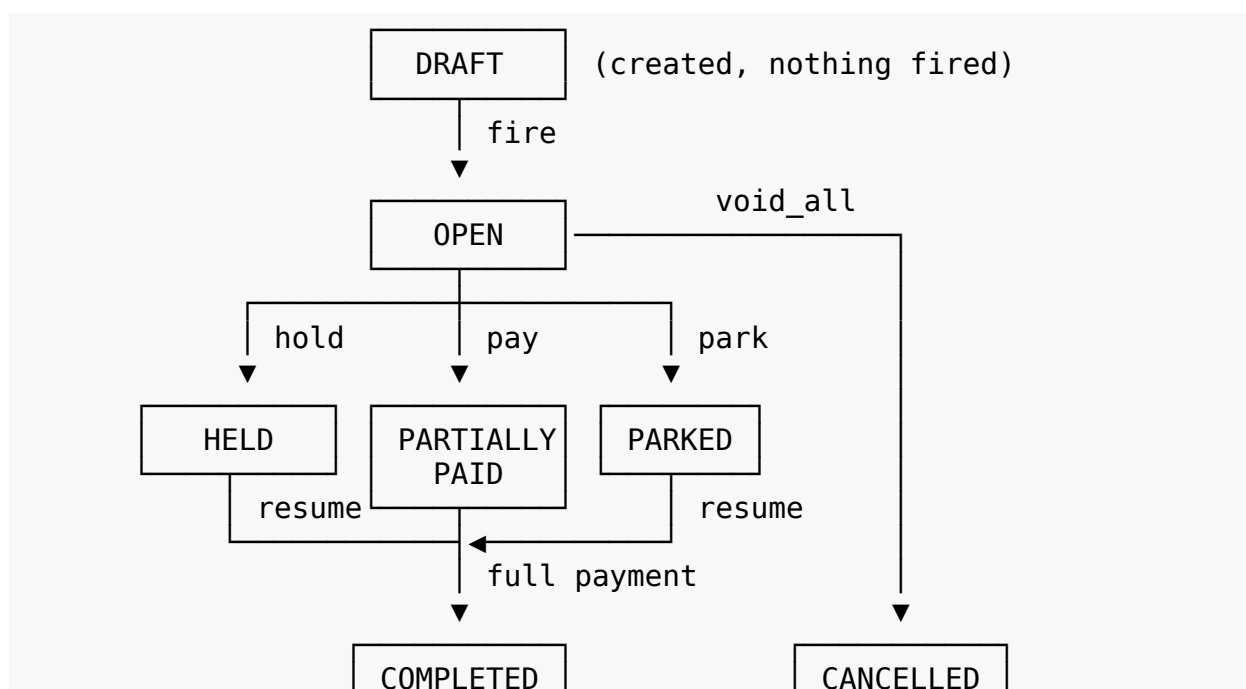
The Order is the most important aggregate in the system. Everything else exists to support it or to be affected by it.

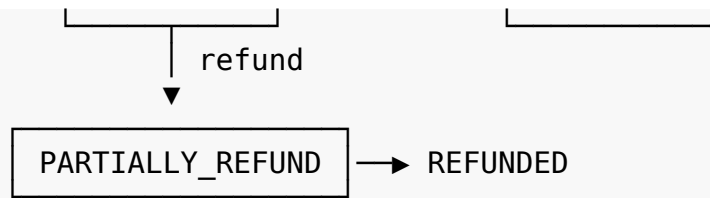
Attribute	Type	Constraints	Notes
id	ULID	PK	Generated client-side
tenant_id	UUID	NOT NULL, FK	RLS key

Attribute	Type	Constraints	Notes
branch_id	UUID	NOT NULL, FK	
terminal_id	UUID	NOT NULL, FK	Originating device
order_number	VARCHAR(20)	NOT NULL	Human-readable; unique per branch per business day
business_day	DATE	NOT NULL	May differ from calendar date
order_type	ENUM	NOT NULL	dine_in, takeaway, delivery, drive_thru, pickup, aggregator
channel	ENUM	NOT NULL	pos, kiosk, qr, aggregator, phone, api
state	ENUM	NOT NULL	See state machine below
table_id	UUID	NULL	Required when dine_in
guest_count	SMALLINT	NULL	
customer_id	ULID	NULL	
opened_by	UUID	NOT NULL	Employee
served_by	UUID	NULL	May differ from opener
closed_by	UUID	NULL	
cash_session_id	ULID	NULL	Set at payment
currency	CHAR(3)	NOT NULL	
subtotal	BIGINT	NOT NULL	Minor units, before discount and tax
discount_total	BIGINT	NOT NULL DEFAULT 0	
service_charge_total	BIGINT	NOT NULL DEFAULT 0	
tax_total	BIGINT	NOT NULL DEFAULT 0	
rounding_adjustment	BIGINT	NOT NULL DEFAULT 0	Cash rounding per country pack
grand_total	BIGINT	NOT NULL	
paid_total	BIGINT	NOT NULL DEFAULT	

Attribute	Type	Constraints	Notes
tip_total	BIGINT	0 NOT NULL DEFAULT 0	
cogs_total	BIGINT	NULL	Populated on completion
opened_at	TIMESTAMPTZ	NOT NULL	
first_fired_at	TIMESTAMPTZ	NULL	
completed_at	TIMESTAMPTZ	NULL	
origin_device_time	TIMESTAMPTZ	NOT NULL	Device's clock at creation
hlc	VARCHAR(40)	NOT NULL	Hybrid logical clock for sync ordering
sync_state	ENUM	NOT NULL	local, pending, synced, conflicted
idempotency_key	VARCHAR(64)	UNIQUE per tenant	Prevents duplicate submission
aggregator_ref	VARCHAR(64)	NULL	External order ID
notes	TEXT	NULL	
metadata	JSONB	NOT NULL DEFAULT '{}'	Extension point

Order state machine:





BR-POS-001 — An Order in state COMPLETED SHALL NOT be modified. Corrections are made by creating a Refund referencing it.

BR-POS-002 — An Order SHALL NOT transition to COMPLETED while $\text{paid_total} + \text{discount_total_of_comps} < \text{grand_total}$.

BR-POS-003 — An Order SHALL NOT transition to CANCELLED if any line has been fired to the kitchen and bumped, unless a user with the `order.cancel_after_production` permission approves and a reason is recorded.

7.4.2 OrderLine Entity

Attribute	Type	Notes
id	ULID	
order_id	ULID	FK to parent
sequence	SMALLINT	Display order
menu_item_id	ULID	
variant_id	ULID	Size/variant selected
item_name_snapshot	JSONB	{ "ar": "...", "en": "... " } — captured at sale time
quantity	DECIMAL(12,3)	Supports fractional (0.5 kg of grilled meat)
unit_price	BIGINT	Before modifiers
modifier_total	BIGINT	Sum of modifier price deltas
line_discount	BIGINT	
line_subtotal	BIGINT	
tax_class_id	UUID	Snapshot of class at sale time
tax_amount	BIGINT	
line_total	BIGINT	
unit_cost_snapshot	BIGINT	Recipe cost at sale time
recipe_version_id	ULID	Which recipe version was consumed
course	SMALLINT	1 = starter, 2 = main, ...
seat_number	SMALLINT	For seat-level ordering and splitting
state	ENUM	pending, fired,

Attribute	Type	Notes
		preparing, ready, served, voided, comped
fired_at	TIMESTAMP TZ	
ready_at	TIMESTAMP TZ	
void_reason_id	UUID	Required when voided
voided_by	UUID	
is_comp	BOOLEAN	Given free; cost still recognised
notes	TEXT	Kitchen instruction

BR-POS-004 — item_name_snapshot, unit_price, tax_class_id, unit_cost_snapshot, and recipe_version_id SHALL be captured at the time of sale and SHALL NOT be recomputed from current master data.

Rationale: Snapshotting is the difference between a report that can be reproduced and one that cannot. If a manager renames “Chicken Burger” to “Grilled Chicken Sandwich” and raises its price, last month’s report must still show what was actually sold at the price actually charged with the cost actually incurred. Systems that join to live master data produce reports that silently change over time, which destroys trust in every number the system produces.

7.4.3 StockMovement Entity

The immutable ledger of all inventory change. Every quantity in the system is derivable from this table.

Attribute	Type	Notes
id	ULID	
tenant_id	UUID	
location_id	UUID	Branch, warehouse, or CK
stock_item_id	ULID	
batch_id	ULID NULL	Required when item is batch-tracked
movement_type	ENUM	See table below
quantity	DECIMAL(16,6)	Signed: negative = out
unit_id	UUID	Always the item’s base unit
unit_cost	BIGINT	Cost per base unit at movement time
total_cost	BIGINT	quantity × unit_cost
balance_after	DECIMAL(16,6)	Running balance (denormalised for audit)
reference_type	VARCHAR(32)	order, goods_receipt,

Attribute	Type	Notes
		transfer, count, waste, production
reference_id	ULID	
counterpart_movement_id	ULID NULL	For transfers: the paired movement
occurred_at	TIMESTAMPTZ	Business time
recorded_at	TIMESTAMPTZ	System time
performed_by	UUID	
reason_code_id	UUID NULL	Required for waste and adjustments
notes	TEXT	

Movement types:

Type	Sign	Trigger
purchase_receipt	+	Goods receipt posted
purchase_return	-	Return to supplier
sale_depletion	-	Order completed
sale_reversal	+	Order refunded / line voided after firing
transfer_out	-	Inter-location transfer dispatch
transfer_in	+	Inter-location transfer receipt
production_input	-	Consumed by production order
production_output	+	Produced by production order
waste	-	Waste recorded
count_adjustment	±	Stock count posted
manual_adjustment	±	Manual correction (restricted permission)
opening_balance	+	Initial stock load
expiry_writeoff	-	Automated expiry removal

BR-INV-001 — StockMovement records SHALL be append-only. No UPDATE or DELETE is permitted. Reversals are new movements with opposite sign referencing the original.

BR-INV-002 — For every `transfer_out` there SHALL be exactly one `transfer_in` with equal absolute quantity, linked by `counterpart_movement_id`. In-transit stock is visible as the interval between the two.

BR-INV-003 — The sum of all movements for an (item, location) pair SHALL equal the `stock_levels` projection for that pair. A reconciliation job SHALL verify this daily and raise an alert on any divergence.

7.4.4 Recipe Aggregate

Attribute	Type	Notes
<code>id</code>	ULID	Logical recipe identity, stable across versions
<code>tenant_id</code>	UUID	
<code>scope</code>	ENUM	tenant, brand, branch — allows local variation
<code>scope_id</code>	UUID	
<code>recipe_type</code>	ENUM	menu_item, sub_recipe, production_item
<code>target_id</code>	ULID	MenuItemVariant or StockItem produced
<code>version</code>	INTEGER	Incremented on publish
<code>status</code>	ENUM	draft, published, superseded, archived
<code>yield_quantity</code>	DECIMAL(16,6)	Output quantity
<code>yield_unit_id</code>	UUID	
<code>yield_percentage</code>	DECIMAL(5,2)	Loss factor, default 100.00
<code>prep_time_seconds</code>	INTEGER	Used for KDS timing
<code>computed_cost</code>	BIGINT	Cached; recomputed on ingredient cost change
<code>cost_computed_at</code>	TIMESTAMPTZ	
<code>effective_from</code>	TIMESTAMPTZ	
<code>published_by</code>	UUID	
<code>instructions</code>	JSONB	Localised preparation steps

RecipeLine:

Attribute	Type	Notes
<code>recipe_version_id</code>	ULID	
<code>sequence</code>	SMALLINT	
<code>component_type</code>	ENUM	stock_item or sub_recipe
<code>component_id</code>	ULID	

Attribute	Type	Notes
quantity	DECIMAL(16,6)	Per recipe yield, not per portion
unit_id	UUID	
wastage_percentage	DECIMAL(5,2)	Expected trim/loss for this component
is_optional	BOOLEAN	Excluded from availability blocking
substitute_group_id	UUID NULL	Alternatives permitted

BR-MNU-001 — A recipe SHALL NOT reference itself, directly or transitively, through sub-recipes. Cycle detection SHALL run on save and reject the change with the offending path displayed.

BR-MNU-002 — Publishing a new recipe version SHALL NOT alter the cost or consumption of previously completed orders. Historical orders reference recipe_version_id.

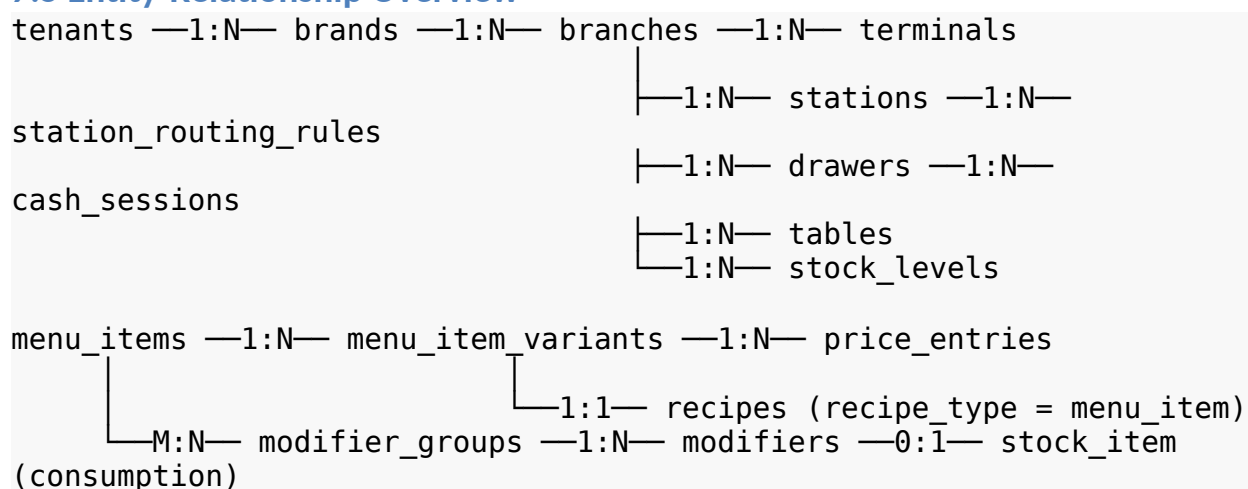
BR-MNU-003 — Recipe cost SHALL be computed as:

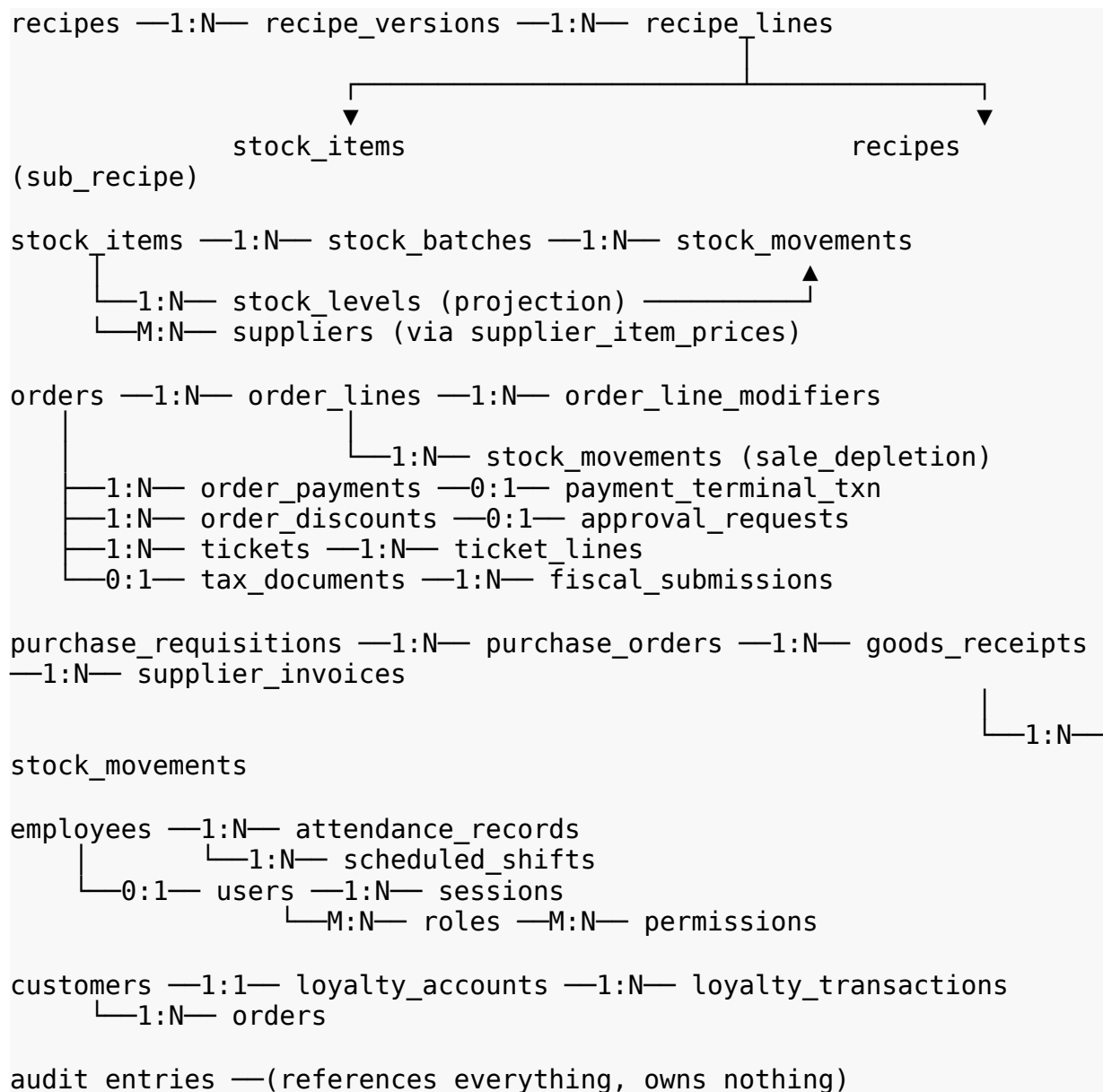
```
cost(recipe) = Σ over lines of:
    quantity_in_base_unit
    × (1 + wastage_percentage / 100)
    × cost_per_base_unit(component)
    ÷ (yield_percentage / 100)
```

```
where cost_per_base_unit(stock_item) = current valuation per the
chosen costing method
    cost_per_base_unit(sub_recipe) = cost(sub_recipe) /
yield_quantity(sub_recipe)
```

Sub-recipe expansion is recursive with a depth limit of 10, exceeded only by malformed data.

7.5 Entity-Relationship Overview





Chapter 8 — Point of Sale | نقطة البيع

8.1 Module Purpose

The Point of Sale is where the business meets its customer and where the System's data originates. Every downstream number — food cost, variance, labour productivity, tax liability — is derived from what happens on this screen. Data quality at the POS determines data quality everywhere.

Two design constraints govern the entire module:

1. **Speed is a correctness requirement.** A POS that is slow will be worked around. Staff will batch orders, take them on paper, and enter them later in bulk. When that happens, kitchen routing breaks, timing data becomes fiction, and inventory depletes at the wrong moment. Every second of friction converts into data corruption downstream.
2. **The POS must function with no server.** Specified in full in Chapter 21. Every requirement in this chapter is written on the assumption that it must be satisfiable offline unless explicitly marked [ONLINE - ONLY].

8.2 Order Capture

8.2.1 Order Creation

FR-POS-001 [M] — The System SHALL allow creation of an order in the following types: dine-in, takeaway, delivery, drive-through, pickup, and aggregator-injected.

FR-POS-002 [M] — The System SHALL assign a human-readable order number, unique within a branch within a business day, generated locally without requiring server connectivity.

Order number format: <branch_code>-<business_day_seq> where the sequence is drawn from a locally-held block. Each terminal is issued a block of numbers on sync (default 500) and requests a new block when 80% consumed. Offline terminals that exhaust their block fall back to <terminal_code>-<local_seq> and are reconciled on sync.

Rationale: Sequential numbering is legally required in several jurisdictions and expected by customers everywhere. Allocating blocks rather than assigning centrally is the only design that satisfies both sequential numbering and offline operation.

FR-POS-003 [M] — For dine-in orders, the System SHALL require table assignment before firing to the kitchen.

FR-POS-004 [S] — The System SHALL support assigning order lines to seat numbers within a dine-in order, to enable seat-level splitting and service.

FR-POS-005 [M] — The System SHALL allow multiple orders to be open simultaneously on one terminal and SHALL allow switching between them without data loss.

FR-POS-006 [M] — The System SHALL allow an order to be parked (set aside) and resumed by the same or another authorised user on the same or another terminal within the branch.

FR-POS-007 [M] — The System SHALL record opened_by, served_by, and closed_by employee identities on every order.

8.2.2 Item Selection

FR-POS-010 [M] — The System SHALL present the menu as a configurable grid of categories and items, with per-branch layout, colour coding, and item images.

FR-POS-011 [M] — The System SHALL support item lookup by: category navigation, text search (Arabic and English, diacritic- and hamza-insensitive), barcode scan, PLU code, and a configurable favourites/quick-keys panel.

FR-POS-012 [M] — Arabic search SHALL normalise ا | آ | إ to a single form, ة to ي, ه to ي, and SHALL ignore tashkeel, so that a cashier typing “شاورما” finds “شَاوْرَمَا”.

Rationale: Arabic text entry on a POS during a rush is error-prone and orthographic variation is the norm. Naïve string matching means the search box does not work, which means staff stop using it, which means the menu grid must contain every item, which means it becomes unnavigable.

FR-POS-013 [M] — The System SHALL support items with multiple variants (size, preparation) and SHALL require variant selection before the line is added.

FR-POS-014 [M] — The System SHALL support fractional quantities for items sold by weight, with input from a connected scale where available.

FR-POS-015 [M] — The System SHALL support open-price items where the price is entered at sale time, restricted by permission and bounded by configurable min/max.

FR-POS-016 [M] — The System SHALL support open-description items (“miscellaneous”) mapped to a configured revenue category and tax class.

8.2.3 Modifiers

FR-POS-020 [M] — The System SHALL present modifier groups attached to an item immediately upon selection, in configured order, enforcing minimum and maximum selection counts.

FR-POS-021 [M] — Modifiers SHALL support three semantic kinds:

Kind	Effect on Price	Effect on Recipe
Addition (“extra cheese”)	+ delta	Adds ingredient quantity
Removal (“no onion”)	0 or – delta	Removes ingredient quantity
Substitution (“chicken instead of beef”)	± delta	Replaces one component with another

FR-POS-022 [M] — Modifier price deltas SHALL be configurable per price list, per branch, and per order type. A modifier may be free for dine-in and charged for delivery.

FR-POS-023 [S] — The System SHALL support nested modifier groups to a depth of 2 (e.g. “Add sauce” – “which sauce” – “how much”).

FR-POS-024 [M] — Removal modifiers SHALL reduce ingredient consumption in the inventory depletion calculation. A “no cheese” burger SHALL NOT deplete cheese.

Rationale: This is a differentiator that competitors handle poorly. Over a month, an outlet selling 3,000 sandwiches with a 12% “no sauce” rate consumes measurably less sauce than the theoretical usage predicts. If modifiers are not recipe-aware, that gap appears as unexplained variance and the manager investigates a theft that never occurred.

FR-POS-025 [S] — The System SHALL support free-text kitchen notes per line, with a configurable per-branch option to disable them.

FR-POS-026 [C] — The System SHALL support a library of predefined note chips (“no ice”, “well done”, “separate packaging”) to reduce free-text usage.

8.2.4 Combos and Meal Deals

FR-POS-030 [S] — The System SHALL support combo items composed of slots, where each slot offers a choice among configured options, with optional price deltas for premium choices.

FR-POS-031 [S] — Combo pricing SHALL support fixed price, sum-of-components-minus-discount, and component-price-override strategies.

FR-POS-032 [S] — Combo revenue SHALL be allocable to component items for reporting purposes, using a configurable allocation basis (equal, by list price, by cost).

Rationale: Without allocation, a combo shows as a single line and the sales report cannot answer “how many burgers did we sell?” because half of them were inside meals. Restaurateurs consider this a defect, correctly.

8.2.5 Firing and Course Management

FR-POS-035 [M] — The System SHALL support firing an order to the kitchen either automatically upon line entry (fast-casual mode) or explicitly by user action (table-service mode), configurable per branch and per order type.

FR-POS-036 [S] — The System SHALL support courses, allowing lines to be grouped and fired independently in sequence.

FR-POS-037 [S] — The System SHALL support a “hold and fire” instruction where a course is sent to the kitchen but marked not to start until released.

FR-POS-038 [M] — Adding a line to an already-fired order SHALL create an amendment ticket to the kitchen clearly marked as an addition, not a reprint of the whole order.

Rationale: Reprinting an entire ticket when one item is added is the single most common cause of duplicate production in kitchens using printer-based systems. The kitchen sees a ticket that looks new and makes everything on it again.

8.3 Pricing, Discounts and Charges

8.3.1 Price Resolution

FR-POS-040 [M] — Price for an item SHALL be resolved by the following precedence, evaluated at order time:

1. Manual price override (permission-gated, audited)
2. Active promotion for the customer and context
3. Time-based price list active at the order timestamp (happy hour)
4. Order-type-specific price list (delivery pricing)

5. Branch price list
6. Brand price list
7. Base price

FR-POS-041 [M] — Price resolution SHALL be deterministic and SHALL be identical when computed offline on the device and online on the server. The resolution algorithm is part of the shared conformance test suite (21.9).

FR-POS-042 [M] — The System SHALL record which price list and which rule produced the price for each line, for audit and reporting.

8.3.2 Discounts

FR-POS-045 [M] — The System SHALL support discounts at line level and order level, expressed as percentage or fixed amount.

FR-POS-046 [M] — Every discount SHALL require selection of a reason from a configurable list.

FR-POS-047 [M] — The System SHALL enforce configurable approval thresholds:

Threshold Dimension	Configuration
Maximum percentage without approval	Per role, per branch
Maximum absolute amount without approval	Per role, per branch
Maximum discounts per shift per employee	Per role
Discount permitted after payment started	Boolean, per role

FR-POS-048 [M] — Approval SHALL be obtainable by manager PIN entry on the terminal, by manager card swipe, or by remote approval request to the manager’s mobile app, without abandoning the order.

FR-POS-049 [M] — The System SHALL record for every discount: amount, percentage, reason, applying employee, approving employee, timestamp, and order context.

FR-POS-050 [S] — The System SHALL support “comp” (complimentary) as distinct from discount: a comped item is given free, the revenue is zero, but the cost is still recognised and inventory is still depleted.

Rationale: Conflating comps and discounts destroys the ability to analyse either. A discount is a pricing decision; a comp is a service-recovery cost. They belong in different lines of the P&L and are abused in different ways.

FR-POS-051 [M] — Discount stacking SHALL be controlled by configuration: promotions may be marked exclusive, and the System SHALL apply the most favourable single discount when exclusivity is set.

8.3.3 Service Charges and Tips

FR-POS-055 [S] — The System SHALL support automatic service charge as a percentage, configurable per branch, per order type, and conditional on guest count.

FR-POS-056 [S] — The System SHALL support tip entry at payment, on the payment terminal, and post-payment adjustment where the acquirer supports it.

FR-POS-057 [S] — The System SHALL support tip pooling configuration and SHALL produce a tip distribution report per shift.

FR-POS-058 [M] — The System SHALL correctly apply the country pack's rule on whether service charge is itself taxable, as this differs by jurisdiction.

8.4 Payment

FR-POS-060 [M] — The System SHALL support the following tender types: cash, card (via integrated terminal), card (manual/external terminal), digital wallet, gift card, loyalty points, store credit, voucher, bank transfer, on-account (credit customer), and aggregator-settled.

FR-POS-061 [M] — The System SHALL support split payment across multiple tenders on a single order, with running balance display.

FR-POS-062 [M] — The System SHALL support splitting a bill by: equal division among N parties, by seat, by selected items, and by arbitrary amount.

FR-POS-063 [M] — For cash payment, the System SHALL compute change due and SHALL apply the country pack's cash rounding rule, recording any rounding adjustment as a separate ledger amount.

FR-POS-064 [M] — Integrated card payment SHALL be initiated from the POS, and the System SHALL handle terminal timeout, decline, partial approval, and communication failure without leaving the order in an indeterminate state.

FR-POS-065 [M] — Payment operations SHALL be idempotent. A retried payment request bearing the same idempotency key SHALL NOT create a second charge.

Risk: The most damaging POS defect class is double-charging a customer because the cashier tapped "Pay" again after a slow response. Idempotency keys generated per payment attempt, plus a terminal-side transaction reference check before initiating a new authorisation, are both required.

FR-POS-066 [M] — The System SHALL NOT store, log, or transmit card PANs, CVV, or magnetic stripe data. Only the last 4 digits, card scheme, authorisation code, and terminal reference SHALL be retained.

FR-POS-067 [S] — The System SHALL support on-account sales to registered credit customers with a configurable credit limit, blocking sales that would exceed it without override permission.

8.5 Voids, Refunds and Cancellations

FR-POS-070 [M] — The System SHALL distinguish four correction operations:

Operation	When	Inventory Effect	Financial Effect
Line void (pre-fire)	Before kitchen	None	None
Line void (post-fire)	After kitchen started	Depletion stands; waste record prompted	Removed from bill
Order cancel	Before payment	Per line rules above	Order marked cancelled
Refund	After payment	Configurable: return to stock or write off	Negative financial record

FR-POS-071 [M] — Post-fire voids SHALL prompt the user to classify the disposition of the produced item: returned to stock (if applicable), wasted, or given to staff. The classification SHALL create the corresponding inventory record.

Rationale: This is where uncontrolled systems bleed. An item is made, voided from the bill, and physically consumed by someone — but the inventory system was never told, so it appears as variance. Forcing disposition classification at the moment of the void is the only reliable way to capture it, because nobody will record it later.

FR-POS-072 [M] — Refunds SHALL reference the original order and SHALL NOT permit a refund exceeding the original amount, in aggregate across all refunds against that order.

FR-POS-073 [M] — Refunds SHALL require a reason and, above a configurable threshold, manager approval.

FR-POS-074 [M] — Refunds SHALL be returned to the original tender type by default; refunding to a different tender SHALL require elevated permission and SHALL be flagged in the fraud detection report.

FR-POS-075 [M] — All voids, cancellations, and refunds SHALL generate audit entries containing the actor, approver, reason, amount, and full before/after state.

8.6 Table Management

FR-POS-080 [S] — The System SHALL provide a graphical floor plan editor allowing definition of areas, tables, shapes, capacities, and positions.

FR-POS-081 [S] — The floor plan SHALL display live table state: available, seated, ordered, food served, bill requested, payment in progress, needs cleaning.

FR-POS-082 [S] — The System SHALL support merging tables, splitting a table's order, and transferring an order between tables, each with an audit record.

FR-POS-083 [S] — The System SHALL display time-since-seated and time-since-last-course per table, to support service pacing.

FR-POS-084 [C] — The System SHALL support server section assignment, restricting or highlighting tables per assigned server.

8.7 Shift Operations at the POS

FR-POS-090 [M] — A cashier SHALL be required to open a shift, declaring an opening float, before processing sales.

FR-POS-091 [M] — The System SHALL support mid-shift cash operations: pay-in (adding cash to drawer), pay-out (removing cash for an expense), and safe drop (removing excess cash to the safe), each with reason and amount.

FR-POS-092 [M] — Safe drops SHALL be enforceable by a configurable drawer limit that triggers a prompt or a block when exceeded.

FR-POS-093 [M] — The System SHALL produce an X report (non-resetting mid-shift summary) on demand for authorised users.

FR-POS-094 [M] — Shift close SHALL require a physical cash count. The System SHALL support both blind count (expected amount hidden until after entry) and open count, configurable per branch.

FR-POS-095 [M] — Blind count SHALL be the default configuration.

Rationale: If the cashier can see the expected amount before counting, a shortage can be concealed by “counting” the expected number. Blind count is a basic internal control and its absence is a finding in any operational audit.

FR-POS-096 [M] — Shift close SHALL compute and record cash variance, and SHALL require a reason and manager acknowledgement when variance exceeds a configurable tolerance.

FR-POS-097 [M] — Denomination-level counting SHALL be supported, with the System computing the total from denomination counts.

8.8 Receipts and Documents

FR-POS-100 [M] — The System SHALL print a customer receipt containing all elements mandated by the branch’s country pack, including tax registration number, invoice sequence, tax breakdown, and any required QR code.

FR-POS-101 [M] — Receipt layout SHALL be template-driven per country pack and per brand, supporting logo, custom header/footer text, and language selection.

FR-POS-102 [M] — The System SHALL support bilingual receipts with configurable layout: Arabic only, English only, or both with defined ordering.

FR-POS-103 [M] — The System SHALL support digital receipt delivery by SMS, WhatsApp, email, and QR code, in addition to or instead of printing.

FR-POS-104 [S] — The System SHALL support receipt reprint, with reprints clearly marked as duplicates and logged.

FR-POS-105 [M] — Kitchen tickets SHALL print in the kitchen’s configured language, which may differ from the customer receipt language.

FR-POS-106 [M] — The System SHALL handle printer failure gracefully: queue the job, alert the user, permit continuation of the sale, and retry automatically, never blocking the transaction.

8.9 POS Non-Functional Requirements

NFR-USA-001 [M] — A three-line dine-in order with no modifiers SHALL be completable in no more than 6 user interactions from an empty screen.

NFR-PERF-001 [M] — Item addition to an order SHALL render within 100 ms at the 95th percentile on the reference device (Android 11, 4 GB RAM, quad-core 1.8 GHz).

NFR-PERF-002 [M] — Payment finalisation, including local persistence and print job dispatch, SHALL complete within 800 ms at p95 offline, and 1,500 ms at p95 online.

NFR-PERF-003 [M] — POS cold start SHALL complete within 6 seconds on the reference device.

NFR-USA-002 [M] — All primary touch targets SHALL be at least 48 × 48 dp with at least 8 dp separation.

NFR-USA-004 [M] — The POS SHALL be fully operable by a user who cannot read, using a configurable image-based menu grid.

NFR-REL-001 [M] — The POS SHALL survive abrupt power loss with no loss of committed orders. Every state transition SHALL be persisted to durable local storage before the UI acknowledges it.

8.10 Use Case: UC-POS-01 — Process a Dine-In Order

Field	Content
Actor	Waiter, Cashier
Preconditions	Shift is open; terminal is authenticated; menu is loaded
Trigger	Customer is seated and ready to order

Main flow:

1. Waiter selects table from floor plan. System creates an order in DRAFT, assigns order number, records opened_by.
2. Waiter selects guest count. System records it.
3. Waiter selects menu items. For each item with required modifier groups, System presents modifier selection and enforces min/max.
4. System resolves price per FR-POS-040, computes line tax per the branch’s country pack, and displays a running total.
5. Waiter assigns courses (optional).

6. Waiter fires course 1. System transitions order to OPEN, creates tickets, routes each line to its station per FR-KDS-010, records `first_fired_at`, and publishes `order.line.fired`.
7. Kitchen prepares and bumps. System receives `ticket.bumped`, updates line states to ready.
8. Waiter fires subsequent courses.
9. Customer requests bill. Waiter prints the pre-bill (non-fiscal).
10. Waiter or cashier initiates payment. System presents tender selection.
11. Payment is captured. System validates `paid_total ≥ grand_total`.
12. System transitions the order to COMPLETED, publishes `order.completed`.
13. Subscribers execute atomically: inventory depletion via recipe expansion; COGS recognition; cash session posting; tax document generation; loyalty accrual; audit entry.
14. System prints the fiscal receipt and, where required, queues fiscal submission via the outbox.
15. System releases the table to `needs_cleaning`.

Alternate flows:

- **3a.** Item is unavailable (86'd): System displays it as disabled with the reason; waiter may request manager override where permitted.
- **6a.** No connectivity: all steps proceed against the local store; outbound events are queued in the local outbox.
- **11a.** Card declined: System returns to tender selection with the balance intact and logs the decline.
- **11b.** Partial payment: order enters PARTIALLY_PAID; remaining balance displayed.
- **13a.** An ingredient's stock goes negative: depletion is recorded regardless (physical reality already occurred) and a negative-stock alert is raised. The sale is never blocked by inventory state.

Rationale for 13a: Blocking a sale because the system's stock count is wrong is a defect, not a control. The food was sold; the record must reflect it. Negative stock is a signal that a receipt was not entered, and it should be surfaced loudly — but never at the customer's expense.

Postconditions: Order is COMPLETED and immutable; inventory reflects consumption; cash session balance updated; audit trail complete; tax document generated.

Chapter 9 — Kitchen Display System | نظام عرض المطبخ

9.1 Module Purpose

The KDS is the operational bridge between an order and the food. It replaces printed tickets, which have three failure modes: they are lost, they cannot be updated after printing, and they produce no timing data.

The KDS's job is to answer, for each cook, one question at a glance from two metres away: **what should I make next?**

9.2 Stations and Routing

FR-KDS-001 [M] — The System SHALL support definition of preparation stations per branch, with configurable name, display colour, and capacity.

Standard station types: Grill, Fryer, Cold/Salad, Hot line, Beverage, Barista, Dessert, Bakery, Shawarma/Rotisserie, Packaging, Expediter (Pass).

FR-KDS-010 [M] — The System SHALL route each order line to one or more stations, resolved by the following precedence:

1. Explicit line-level station override
2. Modifier-driven routing rule (a “make it crispy” modifier may reroute)
3. Menu item’s assigned station for this branch
4. Category default station
5. Fallback station configured for the branch

FR-KDS-011 [M] — A single order line SHALL be routable to multiple stations when the item requires multi-station preparation (a burger requiring grill and packaging).

FR-KDS-012 [S] — The System SHALL support a Prep-Time-Aware coordination mode in which items on one order are released to their stations at staggered times, calculated backwards from a target completion time, so that all components finish together.

```
target_ready = fire_time + max(preptime of all lines)
release_time(line) = target_ready - prep_time(line)
```

Rationale: Without staggering, the salad is ready in 90 seconds and sits wilting for six minutes while the steak cooks. This is the difference between a kitchen system and a ticket printer, and it is a feature customers can taste.

FR-KDS-013 [S] — The System SHALL support an Expediter (Pass) display showing complete orders, with per-station completion state, for final assembly and quality check.

9.3 Display and Interaction

FR-KDS-020 [M] — The KDS SHALL display tickets as cards containing: order number, order type, elapsed time, table or customer reference, item lines with quantity and modifiers, and preparation notes.

FR-KDS-021 [M] — Modifiers SHALL be visually distinguished from item names, with removals (- no onion) rendered differently from additions (+ extra cheese).

FR-KDS-022 [M] — Tickets SHALL be colour-coded by elapsed time against a configurable target: within target (neutral), approaching (amber), exceeded (red), critically exceeded (red, flashing).

FR-KDS-023 [M] — The System SHALL support the following ticket sort orders, configurable per station: oldest first (FIFO), by target completion time, by order type priority, and by course sequence.

FR-KDS-024 [M] — Staff SHALL be able to mark an individual item ready (“bump item”) or an entire ticket ready (“bump all”).

FR-KDS-025 [M] — The System SHALL provide a recall function restoring the most recently bumped tickets, retained for a configurable period (default 30 minutes).

FR-KDS-026 [M] — Bump actions SHALL require a deliberate interaction (long-press, double-tap, or a dedicated confirm zone) to prevent accidental bumping from splashes or brushes against the screen.

Rationale: Kitchen screens get touched by accident constantly — an elbow, a sleeve, a splash of water. Single-tap bumping causes tickets to vanish and food not to be made. Every kitchen that has used a single-tap KDS has this complaint.

FR-KDS-027 [S] — The System SHALL support order priority flags (rush, VIP, remake) with distinct visual treatment.

FR-KDS-028 [S] — Amendments to a fired order SHALL appear as a visually distinct update on the existing ticket, with an audible and visual alert, never as a new ticket.

FR-KDS-029 [M] — Cancelled lines SHALL be struck through and highlighted on the station display, with an alert, and SHALL remain visible for a configurable period so the cook stops preparing.

FR-KDS-030 [S] — The System SHALL support “all-day” counts: an aggregate view showing total outstanding quantity per item across all active tickets at this station.

FR-KDS-031 [S] — The KDS SHALL support an icon-and-image mode for kitchens where staff literacy in the configured language cannot be assumed.

9.4 Timing and Performance Measurement

FR-KDS-040 [M] — The System SHALL record the following timestamps per ticket and per line: created, routed, first viewed, started, ready, bumped, and served.

FR-KDS-041 [M] — The System SHALL compute and report: average preparation time by item, by station, by hour, by employee, and by order type.

FR-KDS-042 [M] — The System SHALL define and report “ticket time” as bump time minus fire time, and “order time” as last-line-ready minus order-open.

FR-KDS-043 [S] — The System SHALL identify the bottleneck station per hour, defined as the station with the highest ratio of queue depth to throughput.

FR-KDS-044 [S] — The System SHALL support configurable per-item target preparation times, defaulting to the recipe’s `prep_time_seconds`.

FR-KDS-045 [C] — The System SHALL provide a capacity warning when queued items at a station exceed configured throughput capacity for the next 15 minutes.

9.5 Kitchen Non-Functional Requirements

NFR-USA-006 [M] — All KDS text conveying item identity SHALL be legible at 2 metres on a 21-inch display at 1920×1080, implying a minimum effective font size of 24 pt for item names.

NFR-PERF-004 [M] — A fired order SHALL appear on the target station display within 1 second at p95 on the local network.

NFR-REL-002 [M] — The KDS SHALL continue to display and accept bump actions for already-received tickets during a network outage, buffering bump events locally.

NFR-REL-003 [M] — Where the KDS and POS are on the same local network, order routing SHALL function via local peer discovery even when the internet connection is down.

Rationale: A cloud-round-trip requirement for kitchen routing means the kitchen stops when the internet stops, which is unacceptable. Local-network routing with cloud sync as a secondary path is the required topology (Chapter 21.6).

9.6 Use Case: UC-KDS-01 — Prepare and Complete a Multi-Station Order

Field	Content
Actor	Kitchen Staff (Grill, Fryer), Expediter
Preconditions	Stations configured; KDS terminals authenticated
Trigger	<code>order.line.fired</code> received

Main flow:

1. System evaluates routing rules per line. Burger → Grill; Fries → Fryer; both → Packaging; ticket summary → Expediter.
2. If staggered release is enabled, System computes release times and holds shorter-prep items.
3. Grill display shows the burger line with elapsed timer starting.
4. Grill staff long-press to mark started (optional configuration), then long-press to bump when done.
5. System records `ready_at` for that line and publishes `ticket.bumped`.
6. Fryer completes similarly.
7. Expediter display shows the order as fully prepared when all station lines are bumped, and highlights it for assembly.
8. Expediter bumps the order. System sets all order lines to `served`, computes order time, and notifies the POS.
9. POS floor plan updates the table state to “food served”.

Alternate flows:

- **3a.** Elapsed time exceeds target: card turns amber then red; a delayed-order alert is raised on the manager's dashboard after a configurable threshold.
- **4a.** Wrong bump: staff uses recall to restore the ticket within the retention window.
- **5a.** Network outage between KDS and server: bump is stored locally and published on reconnection with its original timestamp preserved.
- **6a.** Line is voided at the POS after firing: the line is struck through on the station display with an audible alert, and the POS prompts for waste disposition.

Chapter 10 — Menu & Recipe Management | إدارة المنيو والوصفات

10.1 Module Purpose

This module is where the sellable world (menu) meets the physical world (inventory). It is the module that makes ROS an operating system rather than a cash register: because a menu item is linked to a recipe, and a recipe is composed of stock items, a sale becomes a physical event with a knowable cost.

10.2 Menu Structure

FR-MNU-001 [M] — The System SHALL support a menu hierarchy of: Menu – Category – Sub-category (optional) – Item – Variant.

FR-MNU-002 [M] — A tenant SHALL be able to define multiple menus (e.g. Breakfast, Main, Late Night, Ramadan, Delivery) and assign each to branches, order types, and time windows.

FR-MNU-003 [M] — Where multiple menus are active simultaneously for a context, resolution SHALL follow explicit priority ordering, and the System SHALL warn on configuration that produces ambiguity.

FR-MNU-004 [M] — Menu items SHALL support the following attributes:

Attribute	Purpose
Names (localised, per language)	Display on POS, KDS, receipt, aggregator — each independently overridable
Description (localised)	Customer-facing menu, QR menu
Kitchen name (localised)	Short name for KDS/ticket, may differ from customer name
Images	POS grid, QR menu, aggregator
Category assignment	Navigation and reporting
Tax class	Fiscal treatment
Revenue account code	Accounting export
Station routing	KDS
Preparation time	KDS staggering, customer estimates

Attribute	Purpose
Availability rules	Time, branch, channel
Allergens & dietary tags	Customer information
Barcode / PLU	Scanning
Sort order & colour	POS grid
Is combo / is open-price / is weighed	Behavioural flags

FR-MNU-005 [M] — Item names SHALL be independently configurable for each surface (POS, KDS, customer receipt, aggregator listing), because the appropriate name differs by context.

Rationale: A customer receipt should read “Grilled Chicken Sandwich with Garlic Sauce.” The KDS should read “GRL CHKN SW”. The POS button should read “Chkn Sw”. The aggregator listing should read whatever converts best. Forcing one name across all four produces either an unreadable kitchen ticket or an unprofessional receipt.

FR-MNU-006 [M] — Variants SHALL support independent pricing, independent recipes, independent barcodes, and independent availability.

FR-MNU-007 [S] — The System SHALL support item-level and category-level sort order, with drag-and-drop configuration and a live POS preview.

10.3 Modifier Configuration

FR-MNU-010 [M] — Modifier groups SHALL be defined once and attached to multiple items, with per-item overrides for price and default selection.

FR-MNU-011 [M] — Modifier groups SHALL support: minimum selections, maximum selections, required flag, allow-repeat flag, default selection, and free-quantity threshold (first two free, third charged).

FR-MNU-012 [M] — Individual modifiers SHALL support linkage to a stock item and a consumption quantity, so their inventory impact is computed.

FR-MNU-013 [S] — Modifiers SHALL support a “recipe delta” specification, expressing changes to the parent recipe rather than an independent consumption:

```
modifier: "Extra Cheese"
price_delta: 500          # 5.00
recipe_delta:
  - component: cheddar_slice
    operation: add
    quantity: 20
    unit: gram

modifier: "No Onion"
price_delta: 0
recipe_delta:
  - component: onion_diced
```

```
operation: remove_all

modifier: "Chicken instead of Beef"
price_delta: -300
recipe_delta:
  - component: beef_patty
    operation: remove_all
  - component: chicken_fillet
    operation: add
    quantity: 130
    unit: gram
```

10.4 Pricing Configuration

FR-MNU-020 [M] — The System SHALL support named price lists, each with a scope (tenant, brand, branch, or branch group), a validity window, an optional recurrence schedule, and a priority.

FR-MNU-021 [M] — The System SHALL support order-type-specific pricing, so delivery prices may differ from dine-in prices for the same item.

FR-MNU-022 [M] — The System SHALL support time-based pricing with recurring schedules (e.g. weekdays 15:00–18:00), evaluated in the branch’s timezone.

FR-MNU-023 [M] — Price changes SHALL be schedulable with a future effective date and SHALL propagate to offline terminals in advance of that date, activating locally at the correct local time.

Rationale: A price change that requires connectivity at the moment it takes effect will not take effect on a branch that is offline. Prices must be shipped ahead and activated by the device clock, or a Ramadan promotion will fail at exactly the moment it matters.

FR-MNU-024 [M] — The System SHALL maintain full price change history: who changed what, from what to what, when, and effective when.

FR-MNU-025 [S] — The System SHALL support bulk price operations: percentage increase across a category, rounding to a configured price point (e.g. always end in .95), and CSV import with a preview-and-confirm step.

FR-MNU-026 [S] — The System SHALL warn when a price change would move an item’s contribution margin below a configurable threshold, or below its cost.

10.5 Availability Management

FR-MNU-030 [M] — The System SHALL support manual availability toggling (“86-ing”) per item, per branch, with an optional automatic re-enable time.

FR-MNU-031 [M] — The System SHALL automatically mark an item unavailable when any non-optional ingredient’s stock reaches zero, if automatic availability is enabled for that branch.

FR-MNU-032 [M] — Automatic unavailability SHALL be overridable by an authorised user, with the override recorded.

FR-MNU-033 [S] — The System SHALL compute and display “remaining sellable quantity” per item, being the minimum across ingredients of (available stock ÷ required quantity per portion).

Rationale: “We can sell 14 more of these” is far more actionable to a manager than “cheese is at 280 g”. It is also the number that tells the kitchen when to start the next batch of prep.

FR-MNU-034 [S] — Availability state SHALL propagate to connected aggregator platforms within 60 seconds where the aggregator API supports it.

FR-MNU-035 [C] — The System SHALL support daily quantity limits per item (“only 20 specials today”), decrementing on sale and auto-disabling at zero.

10.6 Recipe Management

FR-MNU-040 [M] — The System SHALL support recipes for menu item variants, for sub-recipes (prep items), and for central-kitchen production items.

FR-MNU-041 [M] — Recipes SHALL specify components as either stock items or other recipes (sub-recipes), with quantity and unit.

FR-MNU-042 [M] — The System SHALL detect and reject circular sub-recipe references, displaying the full cycle path.

FR-MNU-043 [M] — Recipes SHALL support a yield specification: the quantity produced, and a yield percentage representing preparation loss.

FR-MNU-044 [M] — Recipe lines SHALL support a per-component wastage percentage representing trim loss (e.g. peeling potatoes loses 18%).

Rationale: Yield and trim loss are where naive costing systems become useless. A kilogram of purchased potatoes does not produce a kilogram of chips. If the system costs by purchase weight, food cost is understated by 15–25% on every fresh-produce item, and the resulting “excellent” food cost percentage makes the entire report worthless.

FR-MNU-045 [M] — Recipes SHALL be versioned. Publishing a new version SHALL supersede but not delete the prior version, and completed orders SHALL retain their reference to the version in force at sale time.

FR-MNU-046 [M] — Recipe cost SHALL be automatically recomputed when any component’s valuation changes, and the recomputation SHALL cascade through dependent sub-recipes and parent recipes.

FR-MNU-047 [S] — The System SHALL support recipe scoping, allowing a branch to hold a variant recipe differing from the brand standard, with the deviation visible in a compliance report.

FR-MNU-048 [S] — The System SHALL support recipe scaling: displaying a recipe scaled to an arbitrary output quantity for batch production.

FR-MNU-049 [S] — The System SHALL store localised preparation instructions and reference images per recipe, viewable from the KDS.

FR-MNU-050 [C] — The System SHALL compute nutritional information per portion by aggregating component nutritional data where supplied. This is informational only and is explicitly not certified.

BR-MNU-012 — An item MAY be sold with an incomplete or absent recipe. The System SHALL permit the sale, SHALL record zero or partial cost, and SHALL list the item in a “recipes requiring completion” report.

Rationale: Requiring complete recipes before selling is the design decision that kills adoption. A restaurant onboarding on Tuesday cannot document 180 recipes to gram precision before opening on Wednesday. The correct approach is to let them sell immediately and make recipe completeness a visible, nagging, gradually-improving metric.

10.7 Menu Engineering

FR-MNU-055 [S] — The System SHALL classify menu items on the Boston-matrix menu engineering model using popularity (units sold relative to menu average) and profitability (contribution margin relative to menu average):

Classification	Popularity	Margin	Recommended Action
Star	High	High	Protect: maintain quality, feature prominently
Plough-horse	High	Low	Re-engineer: reduce cost or raise price carefully
Puzzle	Low	High	Promote: reposition, rename, train upsell
Dog	Low	Low	Remove or reinvent

FR-MNU-056 [S] — The classification SHALL be computable per branch, per day-part, and per date range.

FR-MNU-057 [C] — The System SHALL analyse modifier attachment rates and suggest price adjustments for modifiers whose attachment rate exceeds a threshold (indicating under-pricing) or falls below one (indicating over-pricing).

Chapter 11 — Inventory Management | إدارة المخزون

11.1 Module Purpose

Inventory is where restaurant margin is won or lost. This module maintains an accurate, auditable record of physical goods across all locations, and — critically — makes the difference between what should be there and what is there both computable and explicable.

11.2 Stock Item Master

FR-INV-001 [M] — The System SHALL maintain a stock item master with the following attributes:

Attribute	Notes
Code / SKU	Unique per tenant
Names (localised)	
Category	Hierarchical: Food — Protein — Poultry
Base unit of measure	Immutable after first movement
Purchase unit(s)	With conversion factor to base unit
Recipe unit	Usually base unit
Costing method	FIFO, weighted average, or standard
Batch tracked	Boolean
Expiry tracked	Boolean
Storage requirement	Ambient, chilled, frozen — drives storage location validation
Shelf life	Days from receipt or from production
Default supplier	
Reorder point	Per location
Reorder quantity	Per location
Minimum / maximum stock	Per location
Is sellable directly	Bottled water sold as-is
Is produced	Made by central kitchen
Barcode(s)	Multiple, including supplier barcodes
Allergens	Propagates to menu items via recipes
Account code	Accounting export mapping

FR-INV-002 [M] — The base unit of measure SHALL be immutable once any stock movement exists for the item.

Rationale: Changing the base unit after movements exist retroactively reinterprets every historical quantity. A change from kilograms to grams would multiply history

by 1,000 and produce an inventory value in the millions. This must be structurally impossible, not merely discouraged.

FR-INV-003 [M] — The System SHALL support multiple purchase units per item with independent conversion factors, so an item may be bought in cases of 12 from one supplier and cases of 24 from another.

FR-INV-004 [M] — The System SHALL support mass→volume conversion for an item where a density factor is configured, and SHALL reject such conversion where it is not.

FR-INV-005 [S] — The System SHALL support supplier-specific item codes and barcodes, so that a supplier's invoice or delivery note can be matched automatically.

11.3 Stock Levels and Valuation

FR-INV-010 [M] — The System SHALL maintain current stock levels per (item, location) pair as a projection over the stock movement ledger.

FR-INV-011 [M] — Stock levels SHALL be reconcilable to the movement ledger at any point in time, and a scheduled job SHALL verify the reconciliation daily and alert on divergence.

FR-INV-012 [M] — The System SHALL support three costing methods, configurable per item:

Method	Computation	Best For
Weighted Average	(existing_value + received_value) / (existing_qty + received_qty) recomputed on each receipt	Most food items; simple and stable
FIFO	Consumption valued at the cost of the oldest remaining batch	Items with volatile pricing; more accurate COGS
Standard Cost	Fixed cost per unit; purchase price variance posted separately	Central kitchen outputs; budgeting

FR-INV-013 [M] — Where an item is batch-tracked, FIFO valuation SHALL follow batch receipt order, and consumption SHALL record which batch was consumed.

FR-INV-014 [M] — The System SHALL permit negative stock levels, recording them and raising an alert, rather than blocking the transaction that causes them.

FR-INV-015 [M] — The System SHALL report total inventory value per location, per category, and per item, at any historical date, computed from the movement ledger.

11.4 Batch and Expiry Management

FR-INV-020 [M] — For batch-tracked items, the System SHALL create a batch record on receipt or production, carrying: batch number, production date, expiry date, quantity, unit cost, supplier, and originating document.

FR-INV-021 [M] — The System SHALL default the expiry date from the item's configured shelf life, allowing override at receipt.

FR-INV-022 [M] — The System SHALL support two consumption strategies, configurable per item:

- **FIFO** — consume the oldest received batch first.
- **FEFO** — consume the batch with the nearest expiry first.

FR-INV-023 [M] — FEFO SHALL be the default for items with expiry tracking enabled.

Rationale: FIFO and FEFO diverge whenever a later delivery has an earlier expiry — which happens constantly in fresh produce and dairy, because suppliers rotate their own stock. Using FIFO on perishables systematically produces expired stock behind fresh stock. FEFO is the correct default for anything that spoils.

FR-INV-024 [M] — The System SHALL generate expiry alerts at configurable horizons per item category (default: 7, 3, and 1 days before expiry).

FR-INV-025 [M] — The System SHALL provide an expiry action worklist showing items approaching expiry, with quantity, value, and one-tap actions: mark wasted, transfer to another location, or create a markdown promotion.

FR-INV-026 [S] — The System SHALL support automatic write-off of expired batches at day close, configurable per item category, creating waste records with reason expired.

FR-INV-027 [S] — The System SHALL support batch traceability reporting: given a batch, list every order that consumed it (forward trace); given an order, list every batch consumed (backward trace).

Rationale: This is a food-safety and legal requirement in an incident. If a supplier issues a recall on a specific lot of chicken, the operator must be able to answer, within minutes, which branches received it, how much remains, and which dates it was sold on. Manual traceability at this granularity is impossible; systemic traceability is a one-query answer.

11.5 Stock Movements and Transfers

FR-INV-030 [M] — The System SHALL record every stock change as an immutable movement per the schema in 7.4.3.

FR-INV-031 [M] — Inter-location transfers SHALL be modelled as a two-step process — dispatch and receipt — creating paired movements, with quantity in transit visible between them.

FR-INV-032 [M] — The receiving location SHALL be able to record a received quantity differing from the dispatched quantity, creating a transfer discrepancy record requiring investigation and approval.

FR-INV-033 [S] — Transfers SHALL support a printed or digital transfer note with barcode/QR for scanning at receipt.

FR-INV-034 [M] — Transfers between locations with different costing bases SHALL transfer at the sending location's cost, with the difference posted to a transfer variance account where configured.

FR-INV-035 [M] — Manual stock adjustments SHALL require a reason code, SHALL be permission-gated, and SHALL require approval above a configurable value threshold.

11.6 Stock Counting

FR-INV-040 [M] — The System SHALL support stock count sessions covering: a full location, a category, a storage area, or an ad-hoc item list.

FR-INV-041 [M] — The System SHALL support two count modes:

- **Blind count** — expected quantity is hidden during counting.
- **Open count** — expected quantity is displayed.

FR-INV-042 [M] — Blind count SHALL be the default and SHALL be enforceable as mandatory by policy configuration.

FR-INV-043 [M] — The System SHALL support counting on a mobile device with barcode scanning, offline-capable, syncing on completion.

FR-INV-044 [M] — The System SHALL freeze the expected quantity at the moment a count session opens, and SHALL account for movements occurring during the count window so that the variance calculation is not corrupted by concurrent activity.

Rationale: A count that takes 90 minutes while the restaurant is trading will show a variance equal to everything sold during the count. The count must be reconciled against a snapshot at session open, with subsequent movements applied afterwards, or the resulting variance is noise.

FR-INV-045 [M] — On posting, the System SHALL compute per-item variance in quantity and value, and SHALL create `count_adjustment` movements bringing recorded stock to counted stock.

FR-INV-046 [M] — Variances exceeding a configurable threshold (by percentage or value) SHALL require a recount or a written explanation before posting.

FR-INV-047 [M] — Count posting SHALL be permission-gated and SHALL be an approval-requiring action for high-value adjustments.

FR-INV-048 [S] — The System SHALL support cycle counting: a rolling schedule that counts high-value or high-variance items frequently and low-risk items rarely, so that a full count is never required.

Class	Criterion	Count Frequency
A	Top 20% by value or high historical variance	Weekly
B	Next 30%	Fortnightly
C	Remaining 50%	Monthly

FR-INV-049 [S] — The System SHALL provide a count sheet ordered by physical storage location to minimise walking, configurable by the tenant.

FR-INV-050 [M] — Count sessions SHALL retain full history, including counted values, counter identity, timestamps, recounts, and approvals.

FR-INV-051 [M] — A scheduled reconciliation job SHALL verify that the sum of movements equals the stock level projection for every (item, location) pair, and SHALL raise a platform alert on any divergence.

11.7 Waste Management

FR-INV-055 [M] — The System SHALL support waste recording at any point: from the POS (void disposition), from the kitchen (KDS), from a stock screen, and from a mobile device.

FR-INV-056 [M] — Every waste record SHALL capture: item, quantity, unit, reason code, location, employee, timestamp, and computed value. Photographs SHALL be optionally attachable.

FR-INV-057 [M] — The System SHALL provide a configurable reason code taxonomy, defaulting to:

Reason	Category	Typical Root Cause
Expired	Storage	Over-ordering, poor rotation
Spoiled	Storage	Temperature failure, poor handling
Damaged in delivery	Supplier	Supplier handling
Preparation error	Kitchen	Training
Overproduction	Kitchen	Forecasting
Incorrect portion	Kitchen	Training, tooling
Burnt / overcooked	Kitchen	Training, equipment
Customer return	Service	Quality, order accuracy
Order error	Service	Order-taking accuracy
Staff meal	Policy	Not true waste; separate

Reason	Category	Typical Root Cause
		category
Sampling / tastings	Policy	Not true waste
Equipment failure	Facility	Maintenance
Theft / unexplained	Control	Investigation required

FR-INV-058 [M] — Waste above a configurable value threshold SHALL require manager approval before posting.

FR-INV-059 [S] — The System SHALL distinguish “true waste” from “controlled consumption” (staff meals, tastings, marketing samples), reporting them separately so that the waste percentage is not inflated by legitimate consumption.

FR-INV-060 [S] — The System SHALL provide waste analysis by item, reason, employee, station, shift, day-part, and branch, with trend comparison.

FR-INV-061 [C] — The System SHALL detect anomalous waste patterns using statistical baselines — for example, waste of a high-value item consistently recorded by one employee during a single shift pattern — and raise them for review.

Rationale: Theft is very often recorded as waste, because waste is the sanctioned way to make inventory disappear. The signature is statistical, not individual: no single waste record looks wrong, but the distribution does. Detection must therefore operate on the distribution.

11.8 Reordering

FR-INV-065 [M] — The System SHALL maintain per-location reorder points and reorder quantities per item.

FR-INV-066 [M] — The System SHALL generate low-stock alerts when available quantity falls below the reorder point.

FR-INV-067 [S] — The System SHALL compute a suggested order quantity using:

```

forecast_demand    = average_daily_usage(last N days, day-of-week
weighted)
                    × (supplier_lead_time_days + review_period_days)
safety_stock       = z × σ(daily_usage) × √(lead_time)
target_level       = forecast_demand + safety_stock
suggested_order    = target_level - (on_hand + on_order - allocated)
rounded_order      = round_up_to(supplier_pack_size, min_order_qty)

```

with z derived from a configurable target service level (default 95%, $z \approx 1.65$).

FR-INV-068 [S] — The suggestion SHALL be capped by remaining shelf life: the System SHALL NOT suggest ordering more of a perishable item than can be consumed before expiry at forecast usage.

FR-INV-069 [S] — The System SHALL incorporate known future demand — scheduled events, catering orders, promotions — into the forecast where recorded.

FR-INV-070 [C] — The System SHALL adjust forecasts for known seasonality, including Ramadan, Eid, and local holidays defined in the country pack.

Rationale: In MENA markets, Ramadan alters demand patterns by 200–400% and inverts the day-part distribution entirely. A forecasting model that treats it as an anomaly to be smoothed will under-order catastrophically on the first day and over-order on the last. Ramadan is not noise; it is a calendar feature and must be modelled as one.

11.9 Inventory Non-Functional Requirements

NFR-PERF-005 [M] — Stock level queries for a single location with up to 3,000 items SHALL return within 500 ms at p95.

NFR-PERF-006 [M] — Recipe expansion and inventory depletion for a completed order of up to 30 lines SHALL complete within 200 ms at p95 and SHALL execute within the order’s transaction.

NFR-DATA-001 [M] — Stock movement records SHALL be retained for the statutory period of the branch’s jurisdiction or 7 years, whichever is greater.

11.10 Use Case: UC-INV-01 — Perform a Weekly Stock Count

Field	Content
Actor	Storekeeper, Branch Manager
Preconditions	Items configured with storage locations; user holds <code>inventory.count</code> permission

Main flow:

1. Manager creates a count session, selecting scope (e.g. “Walk-in chiller”) and mode (blind).
2. System snapshots expected quantities and timestamps the session open.
3. System generates a count sheet ordered by storage position and pushes it to the mobile app.
4. Storekeeper counts, scanning each item’s barcode and entering quantity in any configured unit; the System converts to base units.
5. Storekeeper submits. Session state becomes `submitted`.
6. System applies any movements that occurred between snapshot and submission, computes adjusted expected quantities, and calculates variance per item in quantity and value.
7. Items exceeding the variance threshold are flagged and presented to the manager.
8. Manager either triggers a recount for flagged items or records an explanation.
9. Manager posts the count. System creates `count_adjustment` movements, publishes `stock.counted`, and writes audit entries.

10. Costing module recomputes variance analysis and updates the theoretical-versus-actual report.

Alternate flows:

- **4a.** Device is offline: count is stored locally and synced on reconnection; the session snapshot remains authoritative.
- **4b.** An item not on the sheet is found: storekeeper adds it ad hoc; System includes it in the session.
- **7a.** Variance exceeds the block threshold: posting is prevented until recount or approval by a user with the `inventory.approve_high_variance` permission.
- **9a.** Two counts for the same scope are submitted concurrently: the System rejects the second with a conflict error; sessions for overlapping scope are mutually exclusive.

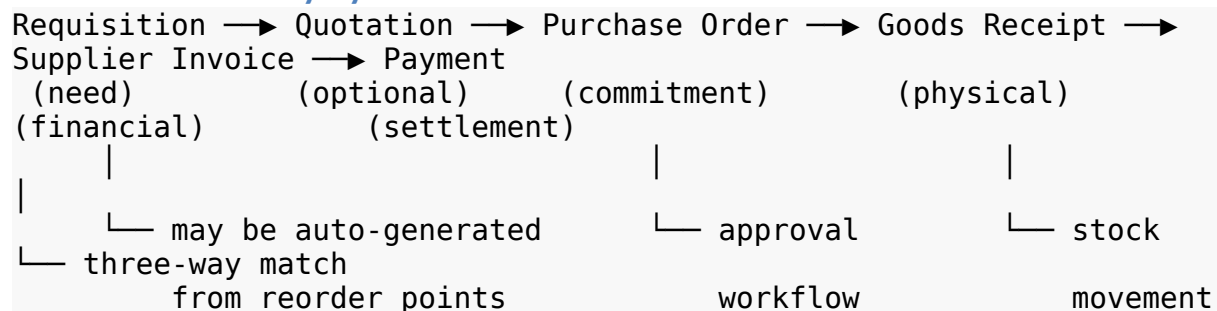
Chapter 12 — Purchasing & Suppliers | المشتريات والموردون

12.1 Module Purpose

Purchasing is the point where money leaves the business and inventory enters it. It is also, in most restaurants, the least controlled process: goods are ordered by phone, received by whoever is at the door, and invoiced by a supplier whose prices are checked by nobody.

This module imposes a controlled procure-to-pay cycle without imposing corporate bureaucracy on a business that operates at speed.

12.2 Procure-to-Pay Cycle



FR-PRC-001 [M] — The System SHALL implement the procure-to-pay cycle above, with configurable steps that may be skipped per tenant policy.

FR-PRC-002 [M] — A tenant SHALL be able to operate in “simple mode” where a purchase order and goods receipt are combined into a single receiving action, for small operations that do not raise formal POs.

Rationale: A single-branch café that buys vegetables from the market each morning will never raise a purchase order. Requiring one guarantees the module goes unused and inventory receipts never get recorded, which destroys the entire costing chain. The formal cycle must be available, not mandatory.

12.3 Suppliers

FR-PRC-005 [M] — The System SHALL maintain a supplier master with: code, legal name, trading name, tax registration number, addresses, contacts, payment terms, currency, delivery lead time, minimum order value, delivery days, and status.

FR-PRC-006 [M] — The System SHALL maintain supplier price lists per item, with unit, pack size, price, validity window, and optional volume break tiers.

FR-PRC-007 [M] — The System SHALL support multiple suppliers per item with a preference ranking, and SHALL display comparative pricing when ordering.

FR-PRC-008 [S] — The System SHALL track price history per supplier per item and SHALL alert when a received price differs from the agreed price by more than a configurable tolerance.

Rationale: Supplier price creep is one of the most common and least detected margin leaks. A supplier raises the price of cooking oil by 4% every second month. Nobody checks. Over a year the cost rises 25% and the food cost report shows a slow deterioration that gets attributed to portion control. Automated price-variance detection at receipt makes this visible on the day it happens.

FR-PRC-009 [S] — The System SHALL maintain a supplier scorecard computing:

Metric	Computation
On-time delivery rate	$\text{Deliveries within promised window} \div \text{total deliveries}$
Fill rate	$\text{Quantity received} \div \text{quantity ordered}$
Price stability	$\text{Standard deviation of price changes over period}$
Quality rejection rate	$\text{Rejected quantity} \div \text{received quantity}$
Invoice accuracy	$\text{Invoices matching without dispute} \div \text{total invoices}$
Average lead time	$\text{Actual delivery date} - \text{order date}$

FR-PRC-010 [S] — The System SHALL support approved-supplier lists per item category, and SHALL warn or block when ordering a category from a non-approved supplier.

FR-PRC-011 [S] — The System SHALL store supplier compliance documents (food safety certificates, licences, insurance) with expiry dates and expiry alerting.

12.4 Requisitions and Purchase Orders

FR-PRC-015 [S] — Branches SHALL be able to raise purchase requisitions for central review, supporting the common group structure where branches request and head office buys.

FR-PRC-016 [S] — The System SHALL support consolidating requisitions from multiple branches into a single supplier purchase order, retaining branch attribution for delivery and cost allocation.

FR-PRC-017 [M] — Purchase orders SHALL contain: supplier, delivery location, expected delivery date, lines with item, quantity, unit, unit price, tax, and total.

FR-PRC-018 [M] — The System SHALL support a configurable approval workflow for purchase orders based on total value:

Value Band	Approver
Below threshold 1	Auto-approved
Threshold 1 – 2	Branch Manager
Threshold 2 – 3	Operations Director
Above threshold 3	Tenant Owner

FR-PRC-019 [M] — The approval workflow SHALL enforce segregation of duties: the requester SHALL NOT be an approver of their own requisition or order.

FR-PRC-020 [M] — Approval SHALL be actionable from the mobile app and from an email link with a signed, single-use, time-limited token.

FR-PRC-021 [S] — Approved purchase orders SHALL be transmittable to suppliers by email (PDF plus structured attachment), WhatsApp, or supplier portal where integrated.

FR-PRC-022 [S] — The System SHALL generate suggested purchase orders from reorder-point analysis (FR- INV- 067), grouped by preferred supplier, presented for review and adjustment before approval.

FR-PRC-023 [M] — Purchase orders SHALL be amendable before receipt, with amendment history retained and re-approval required if the value increases beyond the approved band.

12.5 Goods Receipt

FR-PRC-030 [M] — The System SHALL support receiving against a purchase order, receiving without a purchase order (direct receipt), and receiving a partial quantity.

FR-PRC-031 [M] — At receipt the System SHALL capture per line: quantity received, unit, batch number, production date, expiry date, unit price if differing from PO, and rejection quantity with reason.

FR-PRC-032 [M] — Receipt SHALL create stock movements of type purchase_receipt and, for batch-tracked items, SHALL create batch records.

FR-PRC-033 [M] — The System SHALL enforce configurable over-receipt tolerance, requiring approval to receive more than the ordered quantity beyond the tolerance.

FR-PRC-034 [S] — The System SHALL support receiving on a mobile device with barcode scanning and camera capture of delivery notes.

FR-PRC-035 [S] — The System SHALL support temperature recording at receipt for chilled and frozen items, with configurable acceptable ranges and automatic rejection prompts outside them.

FR-PRC-036 [M] — Rejected goods SHALL create a rejection record with reason and SHALL NOT enter stock, and SHALL generate a supplier credit expectation.

FR-PRC-037 [S] — The System SHALL support supplier returns after receipt, creating a negative stock movement and a credit note expectation.

12.6 Supplier Invoices and Matching

FR-PRC-040 [M] — The System SHALL support recording supplier invoices against goods receipts.

FR-PRC-041 [M] — The System SHALL perform a three-way match across purchase order, goods receipt, and supplier invoice, evaluating:

Check	Tolerance
Quantity: invoice vs receipt	Configurable, default 0%
Unit price: invoice vs PO	Configurable, default 2%
Total: computed vs stated	1 minor unit (rounding)
Tax: computed vs stated	1 minor unit

FR-PRC-042 [M] — Invoices matching within tolerance SHALL be eligible for payment approval. Invoices outside tolerance SHALL enter a dispute state requiring resolution.

FR-PRC-043 [S] — The System SHALL support supplier credit notes, applied against outstanding invoices.

FR-PRC-044 [S] — The System SHALL maintain a supplier account statement showing invoices, credits, payments, and outstanding balance, with an ageing breakdown (current, 30, 60, 90+ days).

FR-PRC-045 [S] — The System SHALL generate a payment proposal listing invoices due within a horizon, respecting payment terms and available early-settlement discounts.

FR-PRC-046 [C] — The System SHALL support invoice capture by photograph with OCR extraction of supplier, date, invoice number, line items, and totals, presented for human verification before posting.

Rationale: OCR is presented as an accelerator, never as an authority. An OCR pipeline that posts directly to the ledger will eventually mis-read a quantity and create a five-figure inventory error that nobody notices for a month. Human confirmation of extracted values is a hard requirement.

12.7 Purchasing Use Case: UC-PRC-01 — Weekly Order to Delivery

Field	Content
Actor	Branch Manager, Purchasing Officer, Storekeeper

Main flow:

1. System runs the scheduled reorder analysis overnight and generates suggested orders grouped by supplier.
2. Branch Manager reviews suggestions, adjusts quantities, and submits as a requisition.
3. Purchasing Officer consolidates requisitions across branches for the same supplier.
4. System creates a purchase order and routes it through the approval chain per value band.
5. Approver approves from mobile. System transitions the PO to approved and enqueues transmission via the outbox.
6. Supplier receives the PO by email with PDF and structured attachment.
7. Goods arrive. Storekeeper opens the PO on a mobile device, scans items, enters received quantities, batch numbers, and expiry dates, and records chiller temperature.
8. Two items arrive short and one is rejected for damage. Storekeeper records both.
9. Storekeeper posts the receipt. System creates stock movements and batches, publishes stock . received, and updates weighted-average costs.
10. Costing module recomputes affected recipe costs, cascading through sub-recipes.
11. Supplier invoice arrives and is recorded. System performs the three-way match.
12. One line's price is 6% above the PO price, exceeding tolerance. The invoice enters disputed.
13. Purchasing Officer contacts the supplier, obtains a credit note, records it, and the invoice matches.
14. Invoice enters the payment proposal for its due date.

Alternate flows:

- **7a.** Delivery arrives with no PO in the System (emergency purchase): storekeeper performs a direct receipt, which is flagged in the unauthorised-purchase report.
- **7b.** Chiller temperature is out of range: System prompts for rejection or acceptance with justification, and records the reading regardless.
- **11a.** Invoice references a receipt already fully invoiced: System rejects as a probable duplicate and surfaces the original.

Chapter 13 — Costing, Waste & Profitability | التكلفة والهدر والربحية

13.1 Module Purpose

This module computes the numbers that determine whether the business is viable. It is the analytical heart of the product and the primary reason a restaurant would pay for ROS rather than a cheaper POS.

13.2 Cost of Goods Sold

FR-CST-001 [M] — On order completion, the System SHALL compute COGS by expanding each order line's recipe to base ingredients, applying modifier recipe deltas, and valuing consumption at the item's current cost per the configured costing method.

FR-CST-002 [M] — COGS SHALL be recorded on the order line as `unit_cost_snapshot` and SHALL NOT be recomputed retroactively when ingredient costs change.

FR-CST-003 [M] — The System SHALL report food cost percentage as:

$$\text{Food Cost \%} = \text{COGS} \div \text{Net Sales} \times 100$$

where $\text{Net Sales} = \text{Gross Sales} - \text{Discounts} - \text{Refunds} - \text{Tax}$

FR-CST-004 [M] — Food cost percentage SHALL be reportable by item, category, branch, brand, day-part, order type, and date range.

FR-CST-005 [M] — The System SHALL compute contribution margin per item:

Contribution Margin	= Selling Price (ex-tax) – Direct Cost
Contribution Margin %	= Contribution Margin ÷ Selling Price (ex-tax) × 100
Total Contribution	= Contribution Margin × Units Sold

FR-CST-006 [S] — The System SHALL support inclusion of configurable non-recipe direct costs in the margin calculation: packaging (per order type), delivery commission (per channel), and disposables.

Rationale: A burger with a 68% margin dine-in may have a 31% margin on a delivery aggregator once a 25% commission and packaging cost are applied. Restaurants routinely discover, too late, that their delivery channel is loss-making at volume. Channel-aware margin is the number that prevents this, and computing it requires the channel's commission structure to be modelled explicitly.

FR-CST-007 [S] — The System SHALL compute channel profitability, showing revenue, COGS, commission, packaging, and net contribution per sales channel.

13.3 Theoretical versus Actual Usage

This is the core analytical capability of the System.

FR-CST-010 [M] — For a defined period and location, the System SHALL compute **theoretical usage** per stock item as the sum, across all sold items, of recipe-expanded ingredient quantities.

FR-CST-011 [M] — For the same period and location, the System SHALL compute **actual usage** as:

$$\begin{aligned} \text{Actual Usage} = & \text{Opening Count} \\ & + \text{Purchases Received} \\ & + \text{Transfers In} \\ & + \text{Production Output} \end{aligned}$$

- Transfers Out
- Production Input
- Closing Count

FR-CST-012 [M] — The System SHALL compute **variance**:

Variance Quantity = Actual Usage – Theoretical Usage
 Variance Value = Variance Quantity × Current Unit Cost
 Variance Percentage = Variance Quantity ÷ Theoretical Usage × 100

FR-CST-013 [M] — Recorded waste SHALL be presented as a separate column so that the report distinguishes explained from unexplained variance:

Unexplained Variance = Variance Quantity – Recorded Waste Quantity

FR-CST-014 [M] — The variance report SHALL be sortable by variance value descending, so that the largest financial losses appear first regardless of quantity.

Rationale: Sorting by quantity variance surfaces flour and salt. Sorting by value surfaces the beef, the salmon, and the imported cheese. Only one of those orderings is useful. This sounds like a trivial UI decision; it determines whether the report changes behaviour or gets closed after ten seconds.

FR-CST-015 [S] — The System SHALL classify variance by likely cause using the following heuristics, presented as hypotheses rather than conclusions:

Signature	Likely Cause
Consistent positive variance on one item across all shifts	Recipe understates actual portion
Variance concentrated in one shift pattern	Portion control or theft in that shift
High variance on high-value items only	Theft
Variance correlating with a specific employee's shifts	Requires investigation
Variance appearing after a supplier change	Yield difference in the new product
Sudden variance with no pattern change	Missing receipt or unrecorded transfer
Negative variance (used less than theoretical)	Under-portioning, or unrecorded receipt

FR-CST-016 [S] — The System SHALL compute variance at a per-shift granularity where opening and closing counts are available at that granularity, and SHALL fall back to the count period otherwise.

FR-CST-017 [S] — The System SHALL provide a variance trend view showing an item's variance percentage over successive count periods.

13.4 Waste Analysis

FR-CST-020 [M] — The System SHALL report waste by value and quantity, grouped by item, reason, category, employee, station, day-part, shift, and branch.

FR-CST-021 [M] — The System SHALL compute waste as a percentage of both COGS and net sales.

FR-CST-022 [S] — The System SHALL compute a rolling baseline of expected waste per item and location, and SHALL flag deviations beyond a configurable number of standard deviations.

FR-CST-023 [S] — The System SHALL produce a “cost of waste” summary translating waste into the revenue required to offset it:

Revenue Required to Offset = Waste Value ÷ (Average Contribution Margin %)

Rationale: “You wasted 4,200 EGP this month” is abstract. “You wasted 4,200 EGP, which requires 6,800 EGP of extra sales to recover” is a sentence that changes behaviour. Framing matters, and the framing that works is the one that converts loss into the effort required to recover it.

FR-CST-024 [C] — The System SHALL correlate waste by reason with operational factors (staffing level, order volume, new employee presence, equipment fault reports) to support root-cause analysis.

13.5 Labour Cost Analysis

FR-CST-030 [S] — The System SHALL compute labour cost per shift from attendance records and configured hourly rates.

FR-CST-031 [S] — The System SHALL compute:

Metric	Formula
Labour Cost %	Labour Cost ÷ Net Sales × 100
Sales per Labour Hour	Net Sales ÷ Hours Worked
Covers per Labour Hour	Orders (or Guests) ÷ Hours Worked
Prime Cost	COGS + Labour Cost
Prime Cost %	Prime Cost ÷ Net Sales × 100

FR-CST-032 [S] — The System SHALL display sales and labour on a common hourly axis so over- and under-staffing by hour is visible.

Rationale: Prime cost is the number experienced operators watch, because it captures the two variable costs that management actually controls. A prime cost above roughly 65% of net sales is a warning; above 70% the business generally cannot cover rent and overheads. Presenting it prominently is more useful than presenting a dozen sub-metrics.

13.6 Profitability Reporting

FR-CST-035 [S] — The System SHALL produce a branch profitability summary:

```
Gross Sales
- Discounts
- Refunds
= Net Sales (excl. tax)
- COGS
= Gross Profit
- Labour Cost
= Contribution After Labour
- Recorded Operating Expenses
= Branch Operating Profit
```

FR-CST-036 [S] — The System SHALL support recording branch operating expenses (rent, utilities, maintenance) with category, recurrence, and allocation.

FR-CST-037 [S] — The System SHALL support branch ranking by any metric, with configurable normalisation (absolute, per seat, per square metre, per labour hour) so that branches of different sizes are comparable.

FR-CST-038 [C] — The System SHALL compute a break-even sales level per branch from fixed costs and average contribution margin, and SHALL show progress against it during the month.

13.7 Fraud and Anomaly Detection

FR-CST-040 [S] — The System SHALL monitor and flag the following patterns for review. Every flag is a prompt for human investigation, never an accusation.

Pattern	Signal
Excessive voids	Void count or value per employee exceeding peer baseline by $>2\sigma$
Post-payment voids	Any void occurring after payment initiation
Refund concentration	Refunds by one employee exceeding baseline
Refunds to different tender	Cash refund of a card sale
Discount concentration	Discounts by employee, hour, or reason exceeding baseline
No-sale drawer opens	Drawer opened without a transaction, by count per shift
Cash variance pattern	Consistent shortage by one cashier, or unusually perfect zero-variance
Order deletion before payment	Orders opened and cancelled with items fired
Waste concentration	High-value waste concentrated by employee or shift
Count adjustment pattern	Repeated positive adjustments by one user

Pattern	Signal
Price override frequency	Manual price overrides beyond baseline
Transactions outside trading hours	Sales recorded when the branch is closed
Round-number cash sales	Statistically improbable frequency of round totals

FR-CST-041 [M] — Anomaly flags SHALL be visible only to users holding an explicit `governance.view_anomalies` permission.

FR-CST-042 [M] — The System SHALL present anomaly flags with the underlying evidence — the specific transactions, timestamps, and comparison baseline — so that a manager can evaluate them rather than accept them.

Rationale: An unexplained accusation against an employee is worse than no detection at all. It produces false accusations, damages trust, and generates legal exposure. The System’s role is to direct attention, and it must always show its working.

FR-CST-043 [S] — Baselines SHALL be computed per branch and per role, not globally, because normal void rates differ enormously between a fast-food counter and a fine-dining room.

13.8 Use Case: UC-CST-01 — Investigate a Food Cost Increase

Field	Content
Actor	Operations Director
Trigger	Weekly scorecard shows Branch 3 food cost at 36.2% against a 31% target

Main flow:

1. Director opens the branch scorecard and drills into food cost.
2. System displays food cost by category, showing Proteins at 44% against a 38% target while other categories are within tolerance.
3. Director drills into Proteins and opens the variance report for the period.
4. System lists items by variance value: chicken breast shows 18.4 kg unexplained variance valued at 2,760 EGP.
5. Director opens the chicken breast detail. System shows theoretical usage 142 kg, actual usage 164.2 kg, recorded waste 3.8 kg, unexplained 18.4 kg.
6. Director views the variance-by-shift breakdown. System shows variance concentrated in the evening shift on four of seven days.
7. Director views the corresponding sales mix and finds no recipe or menu change in the period.
8. System’s cause hypothesis panel indicates “variance concentrated in one shift pattern — portion control or unrecorded consumption.”
9. Director opens the audit log filtered to that shift pattern and reviews voids, comps, and waste records.

10. Director schedules a portion-control audit and a targeted count of chicken at shift handover.
11. Two weeks later, variance falls to 2.1 kg. The trend view records the improvement.

Alternate outcomes:

- Investigation reveals the supplier changed to a product with higher trim loss. Resolution: update the recipe's wastage percentage; variance disappears and food cost target is revised.
- Investigation reveals a goods receipt was never entered. Resolution: enter the receipt; the variance was an artefact of missing data, not of loss.

Chapter 14 — Workforce Management | إدارة الموظفين

14.1 Module Purpose

Labour is the second-largest controllable cost in a restaurant and the largest source of operational variance. This module manages who works, when, at what cost, with what authority, and to what effect.

A structural distinction enforced throughout: an **Employee** is a person in a job. A **User** is a login credential. They are separate entities with an optional link. A kitchen porter is an Employee with no User. A group accountant is a User who is not an Employee of any branch. Conflating them produces a permission model that cannot express either.

14.2 Employee Records

FR-HRM-001 [M] — The System SHALL maintain employee records containing: employee code, names (localised), national/residency identifier, contact details, emergency contact, date of birth, hire date, termination date, position, department, home branch, permitted branches, employment type, and status.

FR-HRM-002 [M] — The System SHALL support employment types: full-time, part-time, casual, contractor, and trainee, with different rules for scheduling and overtime.

FR-HRM-003 [M] — The System SHALL store compensation basis (hourly rate, monthly salary, or per-shift rate) with effective dating, and SHALL restrict visibility of compensation to holders of an explicit permission.

FR-HRM-004 [S] — The System SHALL store document references with expiry dates (residency permit, work permit, health certificate, food-handling licence) and SHALL alert before expiry.

Rationale: In GCC markets, expired residency or health certificates create immediate regulatory exposure for the employer. A restaurant inspected with a lapsed food-handling certificate faces closure. Expiry alerting is a compliance feature with direct commercial value, and it is a common driver of the purchase decision.

FR-HRM-005 [M] — Employees SHALL be assignable to multiple branches with a designated home branch, supporting the common practice of staff covering shifts across a group.

FR-HRM-006 [M] — Employee records SHALL be deactivatable but SHALL NOT be deletable while historical transactions reference them.

14.3 Scheduling

FR-HRM-010 [S] — The System SHALL support creating shift schedules by branch, week, position, and employee.

FR-HRM-011 [S] — The System SHALL support shift templates and week-pattern copying to avoid rebuilding a schedule each week.

FR-HRM-012 [S] — The System SHALL validate schedules against configurable rules and warn or block on violation:

Rule	Default
Maximum consecutive working days	6
Minimum rest between shifts	11 hours
Maximum shift length	12 hours
Maximum weekly hours before overtime	48
Minimum staffing per position per hour	Per branch configuration
Employee availability and leave	Must not conflict
Required certification for position	Must be valid on shift date

FR-HRM-013 [S] — The System SHALL project scheduled labour cost for the week and compare it against a configurable target percentage of forecast sales.

FR-HRM-014 [S] — The System SHALL forecast required staffing per hour from historical sales patterns for the same weekday and season, and SHALL overlay this against the drafted schedule.

FR-HRM-015 [S] — The System SHALL publish schedules to employees by mobile notification and SHALL record acknowledgement.

FR-HRM-016 [C] — The System SHALL support shift swap requests between employees, subject to rule validation and manager approval.

FR-HRM-017 [S] — The System SHALL support leave requests, approval workflow, and a leave balance per configured leave type.

14.4 Attendance

FR-HRM-020 [M] — The System SHALL support clock-in and clock-out via: POS terminal with PIN, mobile app with geofence validation, and biometric device integration where available.

FR-HRM-021 [M] — The System SHALL record clock events with timestamp, method, terminal or device, and — where mobile — GPS coordinates.

FR-HRM-022 [M] — The System SHALL detect and flag: late arrival beyond a grace period, early departure, missing clock-out, clock-in outside the geofence, and clock-in with no scheduled shift.

FR-HRM-023 [M] — The System SHALL prevent a clock-in more than a configurable interval before the scheduled shift start, to prevent inflated paid hours.

FR-HRM-024 [S] — Missing clock-outs SHALL be auto-closed at a configurable maximum shift length, flagged for manager correction, and SHALL NOT silently accrue hours.

FR-HRM-025 [M] — Manual attendance corrections SHALL require a reason, SHALL be permission-gated, and SHALL create an audit entry retaining the original value.

FR-HRM-026 [S] — The System SHALL support break tracking, distinguishing paid from unpaid breaks per configuration.

FR-HRM-027 [C] — The System SHALL support photo capture at clock-in as a deterrent to buddy-punching, subject to explicit configuration and employee notice.

Rationale: Buddy-punching (one employee clocking in for another) is the most common form of labour fraud. Geofencing addresses it for mobile clock-in; photo capture addresses it for terminal clock-in. Both are privacy-sensitive and must be opt-in per tenant with clear notice, because in several jurisdictions covert biometric or photographic capture of employees is unlawful.

14.5 Performance and Payroll Export

FR-HRM-030 [S] — The System SHALL compute per-employee metrics: total sales, order count, average order value, items per order, upsell attachment rate, average service time, void count and value, discount count and value, and cash variance.

FR-HRM-031 [S] — The System SHALL compute kitchen employee metrics from KDS data: items prepared, average preparation time by item, and remake count.

FR-HRM-032 [S] — The System SHALL support ranking employees on any metric within a branch, position, and period.

FR-HRM-033 [M] — The System SHALL compute worked hours, split into regular and overtime per the branch's country pack overtime rules.

FR-HRM-034 [M] — Overtime beyond a configurable threshold SHALL require pre-approval, and unapproved overtime SHALL be flagged in a separate report rather than silently included.

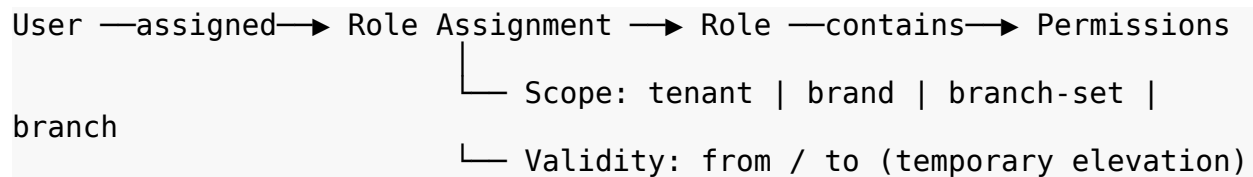
FR-HRM-035 [M] — The System SHALL export payroll data in configurable formats (CSV with mappable columns, plus provider-specific templates) containing employee identifier, period, regular hours, overtime hours, absence, tips allocated, and deductions recorded.

FR-HRM-036 [S] — The System SHALL NOT compute net pay, tax withholding, or statutory contributions. It exports inputs to payroll; it is not a payroll engine.

Chapter 15 — Roles, Permissions & Access Control | الأدوار والصلاحيات

15.1 Access Control Model

ROS uses **Role-Based Access Control with attribute-based scoping** — RBAC for *what*, ABAC for *where and when*.



A permission answers “may this action be performed?” A scope answers “on which data?” Both must be satisfied.

FR-SEC-001 [M] — The System SHALL implement role-based access control where permissions are granted to roles and roles are assigned to users.

FR-SEC-002 [M] — Role assignments SHALL carry a scope, restricting the assignment to a tenant, a brand, a set of branches, or a single branch.

FR-SEC-003 [M] — A user MAY hold multiple role assignments with different scopes — for example, Branch Manager at Branch 1 and Cashier at Branch 2.

FR-SEC-004 [M] — Where multiple assignments apply, effective permissions SHALL be the union of granted permissions within each assignment’s own scope. Permissions SHALL NOT leak across scopes.

FR-SEC-005 [S] — Role assignments SHALL support validity dates, enabling temporary elevation (covering a manager’s leave) that expires automatically.

Rationale: Temporary elevation without expiry is the mechanism by which every organisation’s permission model degrades. Someone covers a shift, gains manager rights, and keeps them for three years. Automatic expiry is the only control that survives contact with operational reality.

15.2 Permission Catalogue

Permissions are granular, named, and grouped. The catalogue below is representative rather than exhaustive; the full catalogue is maintained in Appendix C.

Sales

Permission	Description
pos.order.create	Create and modify orders
pos.order.void_line_prefire	Void a line before firing
pos.order.void_line_postfire	Void a line after firing
pos.order.cancel	Cancel an entire order

Permission	Description
pos.order.cancel_after_production	Cancel after kitchen production started
pos.discount.apply	Apply discounts within limits
pos.discount.approve	Approve discounts above limits
pos.discount.unlimited	Apply discounts without limit
pos.comp.apply	Give complimentary items
pos.price.override	Manually override a price
pos.refund.issue	Issue a refund
pos.refund.different_tender	Refund to a tender other than the original
pos.reprint.receipt	Reprint a receipt
pos.order.transfer	Transfer an order between tables or servers
pos.order.reopen	Reopen a closed order (highly restricted)

Cash

Permission	Description
cash.session.open	Open a shift
cash.session.close	Close own shift
cash.session.close_other	Close another user's shift
cash.drawer.open_no_sale	Open the drawer without a sale
cash.payin / cash.payout	Record cash in / out
cash.safedrop	Perform a safe drop
cash.variance.approve	Approve a variance beyond tolerance
cash.day.close	Close the business day

Inventory

Permission	Description
inventory.view	View stock levels
inventory.count.perform	Perform a stock count
inventory.count.post	Post a count and create adjustments
inventory.approve_high_variance	Post counts exceeding variance thresholds
inventory.adjust	Make manual adjustments
inventory.transfer.create / .receive	Initiate / receive transfers
inventory.waste.record	Record waste
inventory.waste.approve	Approve waste above threshold
inventory.cost.view	View item costs and valuation

Catalogue & Recipes

Permission	Description
menu.item.manage	Create and edit menu items
menu.price.change	Change prices
menu.availability.toggle	86 items
recipe.view	View recipes
recipe.edit	Edit recipes
recipe.publish	Publish a recipe version

Procurement

Permission	Description
purchase.requisition.create	Raise requisitions
purchase.order.create	Create purchase orders
purchase.order.approve_tier_1/2/3	Approve within a value band
purchase.receipt.post	Receive goods
purchase.invoice.record	Record supplier invoices
purchase.invoice.approve_payment	Approve payment
supplier.manage	Manage supplier master data

Workforce

Permission	Description
hr.employee.view / .manage	Employee records
hr.compensation.view	View pay rates
hr.schedule.manage	Build schedules
hr.attendance.correct	Correct clock records
hr.overtime.approve	Approve overtime
hr.payroll.export	Export payroll data

Governance & Platform

Permission	Description
report.view.<category>	View a report category
report.export	Export report data
audit.view	View the audit log
governance.view_anomalies	View fraud/anomaly flags
security.user.manage	Create and manage users
security.role.manage	Create and modify roles
settings.branch.manage	Branch configuration

Permission	Description
<code>settings.tenant.manage</code>	Tenant configuration
<code>integration.manage</code>	Configure integrations
<code>api.key.manage</code>	Manage API credentials

15.3 Standard Roles

FR-SEC-010 [M] — The System SHALL ship predefined roles suitable for immediate use, which tenants may clone and modify but not delete.

Role	Scope	Character
Owner	Tenant	All permissions
Operations Director	Brand(s)	All operational, no user or billing management
Brand Manager	Brand	Menu, pricing, reports; no financial approval
Branch Manager	Branch	Full branch operations, approvals within band
Shift Supervisor	Branch	Approvals within a lower band, no configuration
Cashier	Branch	Sales, own shift only
Waiter	Branch	Sales, no cash handling
Kitchen Staff	Branch	KDS only
Head Chef	Branch(es)	KDS, recipes, waste, inventory view
Storekeeper	Branch/Warehouse	Inventory, receiving, counting
Purchasing Officer	Tenant	Procurement, supplier management
Accountant	Tenant	Financial read and export, no operational write
Auditor	Tenant	Read-only everything including audit log
HR Officer	Tenant	Employee records, schedules, attendance

FR-SEC-011 [M] — Tenants SHALL be able to create custom roles by selecting permissions from the catalogue.

FR-SEC-012 [M] — The role editor SHALL display, for each permission, a plain-language description of what it allows and a warning marker on sensitive permissions.

15.4 Segregation of Duties

FR-SEC-015 [M] — The System SHALL define incompatible permission pairs and SHALL warn when a role or user combines them.

Permission A	Permission B	Risk
inventory.count.perform	inventory.count.post	Counter approves own count
purchase.order.create	purchase.order.approve_*	Self-approved purchasing
purchase.receipt.post	purchase.invoice.approve_payment	Fictitious receipt paid
pos.discount.apply	pos.discount.approve	Self-approved discount
cash.session.close (own)	cash.variance.approve	Self-approved shortage
hr.attendance.correct	hr.payroll.export	Inflated hours exported
security.user.manage	audit.view + delete	Cover tracks (delete is never granted)

FR-SEC-016 [M] — The System SHALL block, not merely warn, on the following combinations regardless of role configuration: approving one's own requisition, approving one's own discount, approving one's own cash variance, and posting a count one performed where the tenant has enabled strict SoD.

FR-SEC-017 [S] — The System SHALL provide an SoD conflict report listing every user whose effective permissions contain an incompatible pair.

15.5 Authentication

FR-SEC-020 [M] — The System SHALL support the following authentication methods:

Method	Context
Email + password	Web dashboard
PIN (4–8 digits)	POS terminal, fast switching
Employee card (magnetic or NFC)	POS terminal
Biometric (device-provided)	Mobile app
SSO via SAML 2.0 / OIDC	Enterprise tier

FR-SEC-021 [M] — PIN authentication SHALL be valid only on registered terminals within the employee's permitted branches, and SHALL NOT grant access to the web dashboard.

FR-SEC-022 [M] — PINs SHALL be stored as salted hashes, SHALL be unique within a branch, and SHALL be subject to logout after a configurable number of failures.

Rationale: A 4-digit PIN has 10,000 combinations and is not a security control against a determined attacker. It is a control against casual misattribution and a

mechanism for accountability. It must therefore be scoped narrowly — terminal-bound, branch-bound, and never sufficient for back-office access. Systems that allow PIN login to the management dashboard have effectively no access control.

FR-SEC-023 [M] — Multi-factor authentication SHALL be supported for dashboard access and SHALL be enforceable as mandatory per role.

FR-SEC-024 [M] — MFA SHALL be mandatory for any role holding `security.user.manage`, `settings.tenant.manage`, or `api.key.manage`.

FR-SEC-025 [M] — Password policy SHALL be configurable per tenant with a minimum of 10 characters, and SHALL check candidate passwords against a known-breached-password list.

FR-SEC-026 [M] — Sessions SHALL expire after a configurable idle period: default 15 minutes on POS, 60 minutes on dashboard, 8 hours on KDS.

FR-SEC-027 [M] — The System SHALL support forced logout of a user's sessions across all devices by an administrator.

FR-SEC-028 [M] — Terminals SHALL be individually registered, and the System SHALL support revoking a terminal's registration, immediately invalidating its credentials and wiping its local data on next contact.

15.6 Approval Workflow Engine

FR-SEC-030 [M] — The System SHALL provide a general approval mechanism used by discounts, refunds, purchase orders, waste, count adjustments, expenses, and price changes.

FR-SEC-031 [M] — Approval requests SHALL specify: the requesting user, the action, the affected entity, the value, the required approver permission, and an expiry.

FR-SEC-032 [M] — Approvals SHALL be obtainable synchronously (manager PIN on the terminal) or asynchronously (push notification to the manager's mobile device), with the terminal remaining usable while awaiting an asynchronous decision.

FR-SEC-033 [M] — Approval decisions SHALL record approver, timestamp, decision, and any comment, and SHALL be immutable.

FR-SEC-034 [S] — The System SHALL support escalation: if no decision is made within a configured period, the request escalates to the next approval level.

FR-SEC-035 [M] — Where an operation must proceed offline and no approver is present, the System SHALL support a configurable policy: block the operation, or permit it with mandatory retrospective approval flagged in an exception report.

Rationale: An offline terminal cannot reach a remote approver. Blocking all approvals offline means a manager standing next to the terminal cannot approve a discount because the internet is down, which is absurd. Manager PIN entry works offline. Remote approval does not, and the fallback policy must be an explicit tenant decision rather than an accident of implementation.

Chapter 16 — Cash & Financial Management | إدارة النقدية والمالية

16.1 Module Purpose

This module tracks physical and electronic money from the moment a customer pays until the moment the business day is closed and reconciled. Its purpose is accountability: every unit of currency must be attributable to a person, a shift, and a drawer.

16.2 Cash Session Model

```
Drawer (physical container, belongs to a branch)
├── Cash Session (one employee, one shift, one drawer)
│   ├── Opening Float (declared)
│   ├── Cash Sales (system-computed)
│   ├── Cash Refunds (system-computed)
│   ├── Pay-ins (recorded)
│   ├── Pay-outs (recorded)
│   ├── Safe Drops (recorded)
│   ├── Expected Cash (computed)
│   ├── Counted Cash (declared, by denomination)
│   └── Variance (computed)
```

FR-FIN-001 [M] — The System SHALL model cash drawers as branch-level entities, with one open cash session per drawer at any time.

FR-FIN-002 [M] — A cash session SHALL be bound to exactly one employee. Two employees SHALL NOT share an open session on the same drawer.

FR-FIN-003 [S] — The System SHALL support shared-drawer mode where a branch policy permits multiple cashiers on one drawer, in which case accountability attaches to the shift rather than the individual, and this SHALL be visibly indicated as a reduced control environment.

FR-FIN-004 [M] — Expected cash SHALL be computed as:

```
Expected Cash = Opening Float
                + Cash Sales
                + Cash Tips (if placed in drawer)
                + Pay-ins
                - Cash Refunds
                - Pay-outs
                - Safe Drops
                ± Cash Rounding Adjustments
```

FR-FIN-005 [M] — Cash variance SHALL be computed as Counted Cash - Expected Cash and SHALL be recorded on the session.

FR-FIN-006 [M] — Variance beyond a configurable tolerance SHALL require a reason and approval by a user with `cash.variance.approve`, who SHALL NOT be the session owner.

FR-FIN-007 [M] — Cash sessions SHALL be immutable once closed. Corrections SHALL be recorded as adjusting entries referencing the session.

16.3 Non-Cash Tender Reconciliation

FR-FIN-010 [M] — The System SHALL record, per session and per day, totals by tender type: cash, each card scheme, each wallet, gift card, voucher, on-account, and aggregator-settled.

FR-FIN-011 [S] — The System SHALL support reconciliation of card totals against the payment terminal's batch settlement report, flagging differences.

FR-FIN-012 [S] — The System SHALL support reconciliation of aggregator-settled orders against aggregator payout statements, computing expected payout net of commission and flagging discrepancies.

Rationale: Aggregator payout reconciliation is a real and under-served pain. Aggregators deduct commissions, promotional contributions, cancellation adjustments, and fees in ways that are difficult to verify. Restaurants frequently accept payouts without checking. A system that says "expected 41,320, received 39,880, difference 1,440 attributable to 9 orders" pays for itself in one month.

16.4 Expenses

FR-FIN-015 [S] — The System SHALL support recording branch operating expenses with category, amount, payment method, supplier, date, attachment, and approval status.

FR-FIN-016 [S] — Petty cash paid from the drawer SHALL be recorded as a pay-out linked to the expense record, so that the drawer reconciles.

FR-FIN-017 [S] — Expenses above a configurable threshold SHALL require approval before posting.

FR-FIN-018 [S] — The System SHALL support recurring expense templates for rent, utilities, and subscriptions.

16.5 Day Close

FR-FIN-020 [M] — The System SHALL support a business-day close operation per branch.

FR-FIN-021 [M] — Day close SHALL be blocked while any cash session remains open, and SHALL list the blocking sessions.

FR-FIN-022 [M] — Day close SHALL produce a Z report containing: gross sales, discounts, refunds, net sales, tax by rate, sales by category, sales by tender, sales by order type, transaction count, average order value, void and comp summary, cash reconciliation, and variance summary.

FR-FIN-023 [M] — Z reports SHALL be sequentially numbered per branch, immutable, and retrievable for any historical date.

FR-FIN-024 [M] — The System SHALL support a configurable business-day boundary per branch (e.g. 04:00), so that late-night trading is attributed to the correct operating day.

FR-FIN-025 [S] — Day close SHALL be performable automatically at the configured boundary where the branch enables it, with any open sessions force-closed and flagged.

FR-FIN-026 [M] — Day close SHALL trigger: fiscal document finalisation, inventory day-end snapshot, report pre-aggregation, and accounting export generation where configured.

16.6 Tax Computation

FR-FIN-030 [M] — Tax SHALL be computed by the country pack's tax engine, never by hard-coded logic.

FR-FIN-031 [M] — The System SHALL support tax-inclusive and tax-exclusive pricing, configurable per branch and per price list.

FR-FIN-032 [M] — The System SHALL support multiple simultaneous tax components (e.g. VAT plus municipality fee plus service tax), each with its own rate, base, and rounding.

FR-FIN-033 [M] — The System SHALL support tax classes per item (standard, reduced, zero-rated, exempt) and SHALL support order-type-dependent rates where the jurisdiction differentiates dine-in from takeaway.

FR-FIN-034 [M] — Tax SHALL be computed at line level and summed, not computed on the order total.

Rationale: Computing tax on the order total rather than per line produces amounts that differ by one or two minor units from line-level computation, and tax authorities that validate submitted invoices will reject the discrepancy. Per-line computation with explicit summation is the only approach that reconciles with fiscal validation, and this is not negotiable.

FR-FIN-035 [M] — The rounding mode and rounding point SHALL be specified by the country pack and SHALL be applied consistently across POS, server, receipt, and fiscal submission.

16.7 Rounding Policy

This section is normative and applies system-wide.

BR-FIN-001 — Percentage computations (tax, discount, service charge) SHALL be performed at full decimal precision and rounded exactly once, at the point specified by the country pack.

BR-FIN-002 — The default rounding mode is HALF_UP (round half away from zero), overridable per country pack.

BR-FIN-003 — Where a discount is applied to an order and must be distributed across lines for tax purposes, distribution SHALL use `Money.allocate` so that the sum of distributed amounts exactly equals the original discount.

BR-FIN-004 — Cash rounding (to the nearest 5 or 25 minor units where the jurisdiction has withdrawn small denominations) SHALL be applied only to the cash portion of a payment and SHALL be recorded as a distinct rounding_adjustment, never absorbed into revenue or tax.

BR-FIN-005 — The client and server SHALL produce byte-identical monetary results for the same inputs. This is verified by the shared conformance suite in 21.9 and any divergence is a release-blocking defect.

Chapter 17 — Branches & Central Kitchen | الفروع والمطبخ المركزي

17.1 Module Purpose

This module enables an operator to run many locations as one business. Its features exist to answer three questions: *are my branches consistent?*, *which branch needs attention?*, and *how do I produce centrally and distribute efficiently?*

17.2 Multi-Branch Operation

FR-BRN-001 [M] — The System SHALL support an unlimited number of branches per brand and an unlimited number of brands per tenant, subject to plan limits.

FR-BRN-002 [M] — Each branch SHALL hold its own: inventory, cash drawers, staff roster, operating hours, timezone, currency, tax configuration, and country pack assignment.

FR-BRN-003 [M] — Branches in different countries SHALL operate under different country packs within one tenant, and consolidated reporting SHALL handle multi-currency correctly.

FR-BRN-004 [M] — Consolidated multi-currency reports SHALL display both the original currency amounts and a converted total, with the conversion rate, rate source, and rate date shown explicitly.

Rationale: A consolidated total that silently converts currencies at an unstated rate is a number nobody can reconcile or defend. Showing the rate and its date is what makes the figure usable in a board pack rather than merely indicative.

FR-BRN-005 [M] — The System SHALL support branch groups (regions, clusters, franchise territories) as a reporting and permission-scoping dimension.

FR-BRN-006 [S] — The System SHALL support central menu and price management with per-branch override capability, and the System SHALL report which branches deviate from the brand standard.

FR-BRN-007 [S] — The System SHALL support central recipe management with the same override-and-report pattern.

FR-BRN-008 [S] — The System SHALL support a branch template: a new branch may be created by copying configuration (menu, stations, roles, settings, printers) from an existing branch.

Rationale: A growing chain opens branches on a schedule. If opening branch 12 requires a week of manual configuration, the System becomes the bottleneck in the customer's expansion — which is the worst possible position for a vendor whose value proposition is enabling growth.

17.3 Branch Comparison and Performance

FR-BRN-010 [S] — The System SHALL provide a branch scorecard comparing branches on: net sales, transaction count, average order value, food cost %, labour cost %, prime cost %, waste %, variance value, void/discount rates, and average service time.

FR-BRN-011 [S] — Comparison SHALL support normalisation by seat count, floor area, labour hours, and trading hours, so branches of different sizes are meaningfully comparable.

FR-BRN-012 [S] — The System SHALL rank branches on any metric and SHALL indicate movement against the prior period.

FR-BRN-013 [S] — The System SHALL highlight outliers, defined as branches deviating beyond a configurable number of standard deviations from the group mean on any tracked metric.

FR-BRN-014 [C] — The System SHALL support like-for-like comparison, excluding branches open for less than a configurable maturity period from group averages.

17.4 Inter-Branch Transfers

FR-BRN-015 [M] — The System SHALL support stock transfers between any two locations within a tenant, per FR - INV - 031.

FR-BRN-016 [S] — The System SHALL support transfer requests: a branch requests stock, the source location approves and dispatches.

FR-BRN-017 [S] — The System SHALL suggest transfers where one location holds excess stock approaching expiry and another location shows a shortage of the same item.

17.5 Central Kitchen

FR-BRN-020 [S] — The System SHALL model central kitchens as locations that consume input stock and produce output stock.

FR-BRN-021 [S] — Production SHALL be driven by a production order specifying output item, target quantity, and target date.

FR-BRN-022 [S] — A production order SHALL expand its recipe to compute required inputs, SHALL check availability, and SHALL flag shortages before production begins.

FR-BRN-023 [S] — On completion, a production order SHALL record actual output quantity, consume input stock, produce output stock, and create an output batch with production date and expiry.

FR-BRN-024 [S] — The System SHALL compute production yield variance:

Yield Variance = Actual Output – (Theoretical Output from Actual Inputs)

and SHALL report it by item, batch, date, and operator.

FR-BRN-025 [S] — Production output cost SHALL be computed as the sum of consumed input costs plus configurable production overhead (labour, energy) per unit or per batch.

FR-BRN-026 [S] — The System SHALL generate a production plan from branch demand forecasts and current branch stock, indicating what the central kitchen should produce for the next production cycle.

FR-BRN-027 [S] — The System SHALL support distribution orders allocating produced quantities to branches, generating transfer movements and, where relevant, delivery manifests.

FR-BRN-028 [S] — Where produced quantity is insufficient for all branch requests, the System SHALL support allocation rules: proportional, by priority ranking, or manual.

FR-BRN-029 [S] — The System SHALL maintain full batch traceability from raw material receipt through production to the branch and to the customer order.

FR-BRN-030 [C] — The System SHALL support internal transfer pricing between central kitchen and branch, enabling the central kitchen to be evaluated as a cost centre or a profit centre per tenant policy.

17.6 Franchise Support

FR-BRN-035 [C] — The System SHALL support franchise branches with restricted configuration authority: franchisees may operate but may not modify brand-controlled menu, recipes, or pricing.

FR-BRN-036 [C] — The System SHALL compute franchise royalties as a configurable percentage of net sales, with reporting per franchisee per period.

FR-BRN-037 [C] — The System SHALL provide franchisor visibility into franchisee operational compliance: menu deviation, price deviation, recipe deviation, and mandated-supplier usage.

Chapter 18 — Customer & Loyalty | العملاء والولاء

18.1 Module Purpose

Restaurants historically know nothing about their customers. This module captures the relationship where the customer is willing to give it, and turns purchase history into targeted action.

18.2 Customer Records

FR-CRM-001 [M] — The System SHALL maintain customer records with: name, phone (primary identifier), email, addresses, date of birth, preferred language, tags, and consent flags.

FR-CRM-002 [M] — Phone number SHALL be the primary identifier, unique within a tenant, stored in E.164 format.

FR-CRM-003 [M] — Customer creation from the POS SHALL require no more than phone number and name, completable in under 15 seconds.

Rationale: Every additional required field reduces capture rate. A form demanding email, birthday, and address at the counter will be filled in with fake data or skipped entirely, and the resulting database is worse than none because it is trusted while being wrong.

FR-CRM-004 [M] — The System SHALL maintain per-customer order history with full detail, retrievable from the POS.

FR-CRM-005 [S] — The System SHALL compute per-customer: total spend, order count, average order value, first and last order dates, favourite items, preferred branch, preferred order type, and preferred day-part.

FR-CRM-006 [S] — The System SHALL support multiple delivery addresses per customer with labels and delivery notes.

FR-CRM-007 [S] — The System SHALL support customer blocking, preventing on-account sales or delivery to a flagged customer, with reason and audit record.

FR-CRM-008 [M] — The System SHALL record explicit consent flags per communication channel and per purpose, with timestamp and source, and SHALL NOT send marketing communication absent recorded consent.

FR-CRM-009 [M] — The System SHALL support customer data export and erasure requests, with erasure anonymising the customer record while retaining transactional and fiscal records as legally required.

Rationale: Erasure cannot delete a sales transaction, because tax law requires its retention. The correct implementation replaces personally identifying fields with an anonymised token while preserving the financial record and its referential integrity. Deleting the order in response to an erasure request creates a tax compliance breach.

18.3 Loyalty

FR-CRM-015 [S] — The System SHALL support a points-based loyalty programme with configurable earn rate, redemption rate, expiry, and eligibility rules.

FR-CRM-016 [S] — Points SHALL be earnable on net spend excluding tax and excluding already-discounted amounts, configurable per tenant.

FR-CRM-017 [S] — Points SHALL be redeemable as a tender type at the POS, converting to a monetary discount at the configured rate.

FR-CRM-018 [S] — The System SHALL support tiered loyalty with configurable thresholds, tier benefits, and tier review periods.

FR-CRM-019 [S] — The System SHALL support a stamp-card model (“buy 9 coffees, get the 10th free”) as an alternative to points, configurable per item or item group.

FR-CRM-020 [M] — Loyalty balances SHALL be maintained as an append-only transaction ledger, not as a mutable balance field.

Rationale: A mutable balance combined with offline operation is a duplication vulnerability: a customer redeems the same points at two branches while both are offline. A ledger with server-side reconciliation on sync detects the double-spend, and the tenant’s configured policy determines whether to honour or reverse it. A mutable field simply produces a wrong number that nobody can explain.

FR-CRM-021 [M] — Offline loyalty accrual SHALL be permitted. Offline redemption SHALL be permitted up to a configurable per-transaction limit, with the risk of overdraw accepted and reported on sync.

FR-CRM-022 [S] — The System SHALL provide the customer a loyalty balance view via a QR code on the receipt leading to a lightweight web page requiring no app installation.

18.4 Promotions

FR-CRM-025 [S] — The System SHALL support a promotion engine with conditions and effects.

Conditions: date and time range, day of week, branch set, order type, channel, minimum order value, specific items or categories present, minimum quantity, customer tag or tier, first order, Nth order, and coupon code presence.

Effects: percentage discount on order, percentage discount on qualifying items, fixed amount off, free item, buy-X-get-Y, cheapest-free, bundle price, free delivery, and points multiplier.

FR-CRM-026 [S] — Promotions SHALL support usage limits: total redemptions, per-customer redemptions, and per-day redemptions.

FR-CRM-027 [M] — Promotion evaluation SHALL be deterministic and identical offline and online, and is covered by the shared conformance suite.

FR-CRM-028 [S] — The System SHALL support single-use and multi-use coupon codes, with bulk generation and per-code tracking.

FR-CRM-029 [S] — The System SHALL report promotion performance: redemptions, incremental revenue, discount cost, margin impact, and redeeming customer profile.

FR-CRM-030 [S] — Promotion stacking SHALL be governed by explicit configuration; the default SHALL be non-stackable with best-value selection.

18.5 Customer Analytics

FR-CRM-035 [C] — The System SHALL compute RFM segmentation (Recency, Frequency, Monetary) and SHALL classify customers into actionable segments: champions, loyal, at-risk, hibernating, and new.

FR-CRM-036 [C] — The System SHALL compute a churn indicator based on a customer's own historical inter-purchase interval rather than a global threshold.

FR-CRM-037 [C] — The System SHALL support exporting a segment for a marketing campaign, subject to consent flags.

Chapter 19 — Reporting & Analytics | التقارير والتحليلات

19.1 Design Philosophy

Most restaurant systems produce many reports and change no behaviour. The failure is structural: reports present data rather than answers, require the user to know what to look for, and arrive too late to act on.

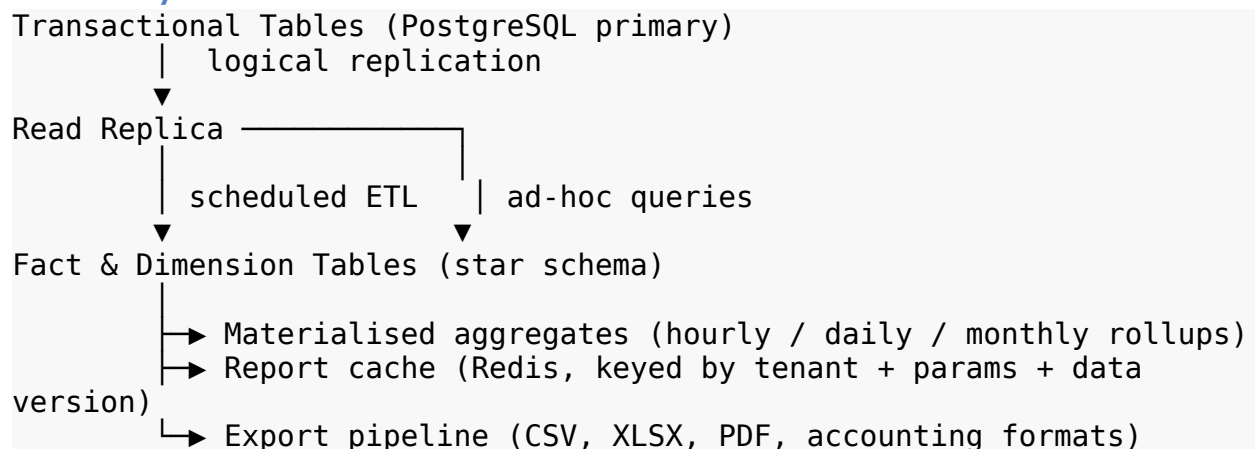
Three principles govern this module:

Principle 1 — Exception over enumeration. The default view surfaces what is abnormal, not what happened. A manager should not have to scan a table of forty items to notice that one is wrong.

Principle 2 — Every number is drillable. Any figure a user can see can be opened to the transactions that produced it, within four interactions. A number that cannot be traced to its source will not be trusted, and an untrusted number is worse than an absent one.

Principle 3 — Reports arrive; they are not fetched. The most valuable report is the one delivered at 08:00 to the manager's phone. Reports that must be sought out are read by the people who already look.

19.2 Analytics Architecture



FR-RPT-001 [M] — Analytical queries SHALL execute against a read replica, never the transactional primary.

FR-RPT-002 [M] — The System SHALL maintain pre-aggregated rollups at hourly, daily, weekly, and monthly grain for the core fact tables.

FR-RPT-003 [M] — Rollups SHALL be incrementally updated as new transactions arrive, and SHALL be fully rebuildable from source on demand.

FR-RPT-004 [M] — Every report SHALL display the timestamp of the data it reflects, and SHALL indicate when data is not yet complete for the period shown.

Rationale: A manager looking at today's sales at 14:00 must understand that the figure is partial. Systems that display an incomplete figure without indication generate support tickets ("the report is wrong") that are not defects.

19.2.1 Star Schema

Fact tables: fact_sales_line, fact_payment, fact_stock_movement, fact_waste, fact_labour_hour, fact_kitchen_timing, fact_purchase_line.

Dimension tables: dim_date, dim_time_of_day, dim_branch, dim_brand, dim_employee, dim_menu_item, dim_stock_item, dim_supplier, dim_customer, dim_order_type, dim_channel, dim_tender, dim_reason_code.

FR-RPT-005 [M] — Dimension tables SHALL be slowly-changing (Type 2) for attributes that affect historical interpretation — item category, employee position, branch region — so that reclassifying an item today does not silently restate last year.

19.3 Report Catalogue

Sales Reports

Report	Key Content
Sales Summary	Gross, discounts, refunds, net, tax, by period
Sales by Branch	Comparative with variance to prior period and target
Sales by Category / Item	Units, revenue, margin, mix percentage
Sales by Employee	Revenue, order count, AOV, upsell rate
Sales by Tender	Reconciliation basis
Sales by Order Type / Channel	Including channel-specific margin
Sales by Hour / Day-part	Demand curve for staffing
Discount & Comp Analysis	By reason, employee, approver
Void & Refund Analysis	By reason, employee, timing relative to payment
Average Order Value Trend	With basket-size decomposition
Menu Mix	Item share of units and of revenue

Inventory Reports

Report	Key Content
Current Stock Valuation	By location, category, item
Stock Movement Ledger	Full auditable movement history

Report	Key Content
Low Stock & Reorder	With suggested quantities
Expiry Watch	Items by days to expiry with value at risk
Stock Count Variance	By session, item, value
Theoretical vs Actual Usage	The variance report
Waste Analysis	By item, reason, employee, shift
Inventory Turnover	Days of stock on hand by item
Dead Stock	Items with no movement in N days

Kitchen Reports

Report	Key Content
Preparation Time by Item	Average, p50, p90, against target
Station Performance	Throughput, queue depth, bottleneck hours
Delayed Orders	Count and duration by hour, station, item
Remake Analysis	Frequency and cost

Financial Reports

Report	Key Content
Z Report	Statutory day close
Cash Reconciliation	By session, cashier, drawer, variance
Tax Summary	By rate, class, jurisdiction, period
Expense Summary	By category, branch, period
Branch P&L	Down to branch operating profit
Prime Cost	COGS + labour against net sales

Workforce Reports

Report	Key Content
Attendance	Scheduled vs actual, lateness, absence
Overtime	Approved vs unapproved
Labour Cost	By branch, position, hour
Sales per Labour Hour	Productivity
Employee Performance	Composite scorecard

Governance Reports

Report	Key Content
Audit Log	Filterable full activity history
Approval History	Requests, approvers, timings

Report	Key Content
Anomaly Flags	With evidence
SoD Conflicts	Users with incompatible permissions
Configuration Change Log	Price, recipe, permission, setting changes

19.4 Dashboards

FR-RPT-030 [M] — The System SHALL provide role-appropriate default dashboards.

FR-RPT-031 [S] — The Executive dashboard SHALL present: net sales against target, prime cost %, branch ranking, top and bottom performing items, exception count, and trend sparklines.

FR-RPT-032 [S] — The Branch Manager dashboard SHALL present: today's sales against forecast, hourly sales curve with labour overlay, food cost trend, items requiring reorder, expiry watch, open exceptions, and staff on shift.

FR-RPT-033 [S] — The System SHALL provide a live operations view: open orders, table states, kitchen queue depth, average current wait, and active terminals.

FR-RPT-034 [C] — Dashboards SHALL be customisable by the user through widget selection and arrangement, with a tenant-level default layout per role.

19.5 Delivery, Export and Alerts

FR-RPT-040 [S] — The System SHALL support scheduled report delivery by email and by mobile push, with configurable recipients, schedule, and format.

FR-RPT-041 [S] — The System SHALL support a daily digest ("morning brief") delivered at a configurable time, summarising the prior business day in a form readable on a phone in under 30 seconds.

FR-RPT-042 [M] — Every aggregate figure SHALL support drill-down to the contributing transactions in no more than four interactions.

FR-RPT-043 [M] — The System SHALL support export in CSV, XLSX, and PDF, with exports of more than 50,000 rows processed asynchronously and delivered by notification.

FR-RPT-044 [M] — All exports SHALL be logged in the audit trail with the requesting user, filters applied, and row count.

FR-RPT-045 [S] — The System SHALL support configurable alerts delivered in real time:

Alert	Default Trigger
Cash variance exceeds tolerance	On shift close
Discount exceeds threshold	On application
Void after payment	Immediately
Stock reaches zero	On depletion

Alert	Default Trigger
Item approaching expiry	Daily at configured hour
Order delayed beyond target	On threshold breach
Sales significantly below forecast	Hourly comparison
Terminal offline beyond threshold	After 15 minutes
Sync backlog exceeds threshold	On breach
Failed fiscal submission	On final retry failure

FR-RPT-046 [M] — Alerts SHALL be rate-limited and de-duplicated per recipient per type per period.

Rationale: Alert fatigue is the standard failure mode of operational alerting. A manager receiving forty notifications a day will disable them within a week, at which point the one alert that mattered is never seen. Aggressive rate limiting and digest grouping are not conveniences; they are what makes the alerting system functional at all.

FR-RPT-047 [C] — The System SHALL support a natural-language query interface over the reporting layer, translating questions such as “which branch had the worst food cost last week” into parameterised queries, with the generated query shown to the user for verification.

19.6 Reporting Non-Functional Requirements

NFR-PERF-010 [M] — Standard reports over a 31-day period for a single branch SHALL render within 2 seconds at p95.

NFR-PERF-011 [M] — Consolidated reports across up to 100 branches over a 31-day period SHALL render within 5 seconds at p95.

NFR-PERF-012 [M] — Dashboard initial load SHALL complete within 3 seconds at p95.

NFR-DATA-002 [M] — Aggregated reporting data SHALL be no more than 15 minutes behind the transactional source during trading hours.

Chapter 20 — Audit, Security & Compliance | التدقيق والأمان

20.1 Audit Log

FR-AUD-001 [M] — The System SHALL record an immutable audit entry for every state-changing operation.

FR-AUD-002 [M] — Each audit entry SHALL contain:

Field	Content
id	ULID
tenant_id, branch_id	Scope

Field	Content
occurred_at, recorded_at	Business and system time
actor_id, actor_type	User, system, device, or integration
impersonated_by	Set when a support user is acting as a tenant user
action	Canonical verb, e.g. <code>order.discount.applied</code>
entity_type, entity_id	Target
before, after	JSONB state snapshots (changed fields only)
reason_code, reason_text	Where required
approver_id, approval_id	Where an approval was involved
ip_address, user_agent, terminal_id	Origin
correlation_id, causation_id	Causal chain
hash, previous_hash	Tamper-evidence chain

FR-AUD-003 [M] — Audit entries SHALL be append-only. The application database role SHALL hold INSERT and SELECT grants on the audit table and SHALL NOT hold UPDATE or DELETE.

FR-AUD-004 [M] — Audit entries SHALL be hash-chained: each entry’s hash covers its own content and the previous entry’s hash, per tenant.

```
hash(n) = SHA-256( canonical_json(entry_n) || hash(n-1) )
```

FR-AUD-005 [M] — A scheduled job SHALL verify chain integrity and SHALL raise a platform-level security alert on any break.

Rationale: Hash chaining does not prevent tampering by an actor with database access. It makes tampering *detectable*, which is what an auditor requires and what makes the log admissible as evidence in a dispute with an employee or a franchisee. A log that could have been silently edited proves nothing.

FR-AUD-006 [M] — The following actions SHALL always generate audit entries: authentication success and failure, permission changes, role changes, price changes, recipe changes, discounts, comps, voids, refunds, cash variances, stock adjustments, count postings, waste records, purchase approvals, configuration changes, data exports, integration credential changes, and impersonation sessions.

FR-AUD-007 [M] — Audit log access SHALL itself be audited.

FR-AUD-008 [M] — The audit log SHALL be searchable and filterable by actor, entity, action, date range, branch, and correlation ID, and SHALL be exportable by users with `audit.view` plus `report.export`.

FR-AUD-009 [M] — Audit entries SHALL be retained for the greater of 7 years or the statutory period of the branch’s jurisdiction.

FR-AUD-010 [M] — Support staff access to tenant data SHALL require an explicit impersonation session with a recorded reason, a time limit, tenant-visible notification, and full audit capture of every action performed.

Rationale: Vendor support access is a legitimate necessity and a genuine risk. Tenants must be able to see, after the fact, exactly what the vendor's staff did in their account. Making this visible by default converts a trust liability into a trust asset, and it is frequently asked about in enterprise security reviews.

20.2 Threat Model (STRIDE)

Threat	Vector	Mitigation
Spoofing	Stolen PIN used by another employee	Terminal binding, PIN lockout, optional photo capture, anomaly detection on behaviour change
Spoofing	Forged terminal registering to a tenant	Device attestation, registration token with limited validity, admin approval of new terminals
Tampering	Direct database modification to hide theft	Hash-chained audit log, append-only ledgers, restricted DB roles, integrity verification job
Tampering	Modified POS client bypassing controls	Signed application builds, server-side revalidation of all financially significant operations on sync
Tampering	Replayed sync payload	Idempotency keys, HLC ordering, server-side duplicate detection
Repudiation	"I never applied that discount"	Immutable audit with actor, approver, and terminal; hash chain
Information Disclosure	Cross-tenant data leakage	RLS as second layer, generated isolation test suite, fail-closed tenant context
Information Disclosure	Data at rest on a lost POS device	Encrypted local database, remote wipe on registration revocation, no card data stored
Information Disclosure	Excessive data in logs	Structured logging with a field allowlist; PII redaction in the logging layer
Denial of Service	One tenant's report	Per-tenant rate limits,

Threat	Vector	Mitigation
	exhausting the pool	separate analytical pool, statement timeouts
Denial of Service	Sync flood after a long outage	Batch size limits, backpressure, prioritised queueing
Elevation of Privilege	Self-approval of sensitive actions	Enforced SoD (FR-SEC-016)
Elevation of Privilege	Permission check only in the UI	Server-side authorisation on every endpoint; UI checks are cosmetic only
Elevation of Privilege	IDOR via guessable identifiers	ULIDs plus mandatory scope validation on every resource access

20.3 Security Requirements

FR-SEC-040 [M] — All network traffic SHALL use TLS 1.3 with TLS 1.2 as the minimum fallback.

FR-SEC-041 [M] — All data at rest SHALL be encrypted: database volumes, object storage, backups, and the POS local database.

FR-SEC-042 [M] — Sensitive fields (national identifiers, bank details, API secrets) SHALL be encrypted at the application layer with envelope encryption and per-tenant data keys, in addition to volume encryption.

FR-SEC-043 [M] — Encryption keys SHALL be managed by a managed KMS, SHALL be rotated at least annually, and SHALL never be committed to source control or logged.

FR-SEC-044 [M] — The System SHALL NOT store card PANs, CVV, magnetic stripe data, or PIN blocks anywhere, including logs, backups, and crash reports.

FR-SEC-045 [M] — Every API endpoint SHALL enforce authorisation server-side. Client-side permission checks are presentation only and SHALL NOT be relied upon.

FR-SEC-046 [M] — The System SHALL apply rate limiting and progressive logout on authentication endpoints.

FR-SEC-047 [M] — All user input SHALL be validated against an explicit schema at the API boundary, rejecting unknown fields rather than ignoring them.

FR-SEC-048 [M] — Database access SHALL use parameterised queries exclusively; string-concatenated SQL is prohibited and SHALL fail static analysis.

FR-SEC-049 [M] — Dependencies SHALL be scanned for known vulnerabilities on every build, and builds SHALL fail on critical findings.

FR-SEC-050 [M] — Secrets SHALL be injected at runtime from a secret manager, never present in images or repositories.

FR-SEC-051 [S] — The System SHALL undergo an external penetration test before general availability and annually thereafter.

FR-SEC-052 [M] — The System SHALL support IP allow-listing for API access and for dashboard access at the Enterprise tier.

FR-SEC-053 [M] — Security-relevant events SHALL be forwarded to a SIEM-compatible sink, and Enterprise tenants SHALL be able to receive their own security event stream.

20.4 Data Protection and Privacy

FR-SEC-060 [M] — The System SHALL classify data by sensitivity and apply corresponding controls:

Class	Examples	Controls
Public	Menu items, prices	Standard
Internal	Sales figures, stock levels	Tenant isolation, RBAC
Confidential	Costs, margins, supplier terms	RBAC with explicit permission
Restricted	Employee compensation, national IDs, customer contact	Explicit permission, field encryption, access audited
Prohibited	Card PAN, CVV	Never stored

FR-SEC-061 [M] — Data retention periods SHALL be configurable within statutory bounds, and a scheduled job SHALL purge or anonymise data past retention.

FR-SEC-062 [M] — The System SHALL support data subject requests: access (export), rectification, and erasure (anonymisation preserving financial records).

FR-SEC-063 [M] — Tenants SHALL be able to select a data residency region at signup, and their data SHALL remain within that region including backups.

FR-SEC-064 [M] — Backups SHALL be encrypted, tested by restoration at least quarterly, and retained per a documented schedule.

20.5 Compliance Mapping

Requirement Source	Applicable Obligation	ROS Implementation
PCI-DSS v4.0	Card data protection	Out of scope by design (A - 04, FR - SEC - 044); SAQ-C or SAQ-P2PE where applicable
OWASP ASVS 4.0 L2	Application security	Verified in the security test suite
Egypt ETA	E-invoice and e-receipt	Country pack EG (Chapter

Requirement Source	Applicable Obligation	ROS Implementation
	submission	22)
KSA ZATCA Phase 2	Cryptographic stamp, QR, clearance	Country pack SA (Chapter 22)
UAE FTA	VAT invoice content and retention	Country pack AE
GDPR (EU tenants)	Lawful basis, DSR, DPA	Consent records, DSR tooling, EU region
Local labour law	Working time records	Attendance and overtime records

Chapter 21 — Offline Operation & Synchronisation | التشغيل دون إنترنت والمزامنة

21.1 The Problem Statement

The POS must complete sales when the server is unreachable, and the resulting data must converge to a single correct state when connectivity returns — without losing sales, duplicating sales, corrupting inventory, or producing sequence gaps that break fiscal compliance.

This is a distributed systems problem with a hard constraint: **money and stock must not be double-counted, and no sale may be lost.**

The design is derived from the CAP theorem's implications for this domain. During a partition, ROS chooses **availability** for order capture (the sale must complete) and **consistency** for financial posting (the server is the arbiter of the final ledger). These are different decisions for different operations, and mixing them up is the source of most offline POS defects.

21.2 Operating Modes

Mode	Condition	Behaviour
Online	Server reachable, latency < 500 ms	Local write, immediate sync, near-real-time server state
Degraded	Server reachable, high latency or intermittent	Local write, batched sync, UI unaffected
Offline	Server unreachable, LAN available	Local write, LAN peer coordination for KDS and shared state, queued sync
Isolated	No network at all	Local write only, all shared state frozen at last-known values

FR-OFF-001 [M] — The POS SHALL detect its operating mode continuously and SHALL indicate it unobtrusively but unambiguously in the interface.

FR-OFF-002 [M] — Mode transitions SHALL NOT interrupt an in-progress order.

FR-OFF-003 [M] — The POS SHALL operate in Isolated mode for a minimum of 72 hours without functional degradation of sales capture.

21.3 Local Data Model

The POS holds a local SQLite database containing:

Data	Sync Direction	Refresh
Menu, items, variants, modifiers	Down	On change, plus periodic full reconcile
Prices and price lists (incl. future-dated)	Down	On change
Recipes (for offline availability and cost)	Down	On change
Tax configuration and country pack	Down	On change, version-pinned
Employees, PINs (hashed), permissions	Down	On change
Customers (recent + frequent subset)	Down	Nightly + on demand
Loyalty balances (last known)	Down	On sync
Stock levels (last known, for availability)	Down	Periodic
Orders, lines, payments	Up	Continuous
Shifts, cash sessions, drawer events	Up	Continuous
Attendance events	Up	Continuous
Waste records	Up	Continuous
Audit events	Up	Continuous
Outbox	Up	Continuous

FR-OFF-010 [M] — The local database SHALL be encrypted at rest with a key derived from device credentials and a server-issued key, unusable if the device registration is revoked.

FR-OFF-011 [M] — Downward reference data SHALL be versioned, and the client SHALL request only deltas since its last known version.

FR-OFF-012 [M] — The client SHALL periodically perform a full reference-data reconciliation (checksum comparison per entity type) to detect and repair silent drift.

FR-OFF-013 [M] — Local storage SHALL be bounded: completed and fully-synced orders older than a configurable period (default 30 days) SHALL be pruned locally while remaining available from the server when online.

21.4 Identifier and Sequence Strategy

FR-OFF-015 [M] — All entities created on a device SHALL receive a client-generated ULID as their permanent primary key. The server SHALL NOT reassign identifiers.

FR-OFF-016 [M] — Human-readable order numbers SHALL be allocated from server-issued blocks per terminal per business day, as specified in FR-POS-002.

FR-OFF-017 [M] — Where a jurisdiction requires a strictly gapless fiscal document sequence, the country pack SHALL specify the sequence strategy:

Strategy	Mechanism	Used Where
Server-assigned on sync	Local document held as provisional; fiscal number assigned at sync	Jurisdictions permitting delayed issuance
Pre-allocated block	Server issues a signed block of fiscal numbers to the terminal in advance	Jurisdictions requiring immediate issuance
Device-scoped series	Each terminal holds its own registered series	Jurisdictions permitting per-device series

FR-OFF-018 [M] — Unused numbers in an expired pre-allocated block SHALL be reported as void to the fiscal authority where required, and SHALL never be silently discarded.

Rationale: Fiscal sequence gaps are the most common compliance failure in offline-capable POS systems. A tax authority that sees invoice 1001 followed by 1004 assumes three sales were concealed. The block allocation strategy must include explicit void reporting, and the strategy must be selectable per jurisdiction because jurisdictions differ irreconcilably on this point.

21.5 Synchronisation Protocol

21.5.1 Upward Sync (Device → Server)

Every local mutation is appended to a local outbox with an HLC timestamp, an idempotency key, and a causal parent reference.

```
POST /v1/sync/batch
{
  "deviceId": "...",
  "batchId": "01J8...",           // idempotency key for the batch
  "lastServerCursor": "01J8...",
  "operations": [
    {
      "opId": "01J8...",           // idempotency key for the
```

```

operation
  "hlc": "1722765753.0042.dev07",
  "type": "order.create",
  "entityId": "01J8...",
  "causedBy": null,
  "payload": { ... }
},
...
]
}

```

FR-OFF-020 [M] — Sync SHALL be batched, with batch size bounded by plan limits and by payload size.

FR-OFF-021 [M] — Every operation SHALL carry an idempotency key. The server SHALL persist processed keys for at least 30 days and SHALL return the original result for a repeated key rather than reprocessing.

FR-OFF-022 [M] — Operations within a batch SHALL be applied in causal order. An operation whose causal parent has not been applied SHALL be deferred, not rejected.

FR-OFF-023 [M] — The server SHALL respond per-operation with accepted, duplicate, conflict, or rejected, including a reason for the latter two. A single failing operation SHALL NOT fail the batch.

FR-OFF-024 [M] — The client SHALL remove an operation from the outbox only upon receiving a definitive server response for it.

FR-OFF-025 [M] — Sync SHALL be resumable: a connection lost mid-batch SHALL resume without duplication, guaranteed by batch and operation idempotency.

FR-OFF-026 [M] — The client SHALL apply exponential backoff with jitter on failure, and SHALL prioritise financially significant operations (orders, payments, cash sessions) ahead of others in the queue.

21.5.2 Downward Sync (Server → Device)

FR-OFF-030 [M] — The client SHALL pull reference data changes since its cursor on a configurable interval and immediately on regaining connectivity.

FR-OFF-031 [S] — Where a persistent connection is available, the server SHALL push change notifications by WebSocket so that price changes, 86'd items, and permission revocations propagate within seconds.

FR-OFF-032 [M] — Permission and terminal revocations SHALL be applied by the client immediately on receipt and SHALL be re-verified on every reconnection.

21.6 LAN Peer Coordination

FR-OFF-035 [M] — POS and KDS devices on the same local network SHALL discover each other by mDNS and SHALL exchange order and bump events directly, so that the kitchen functions with no internet connection.

FR-OFF-036 [M] — One device per branch SHALL be electable as the LAN coordinator, holding the authoritative branch-local state during an outage. Election SHALL be automatic with deterministic tie-breaking on device identifier.

FR-OFF-037 [M] — LAN peer communication SHALL be authenticated using a branch-scoped key distributed during registration, and SHALL be encrypted.

FR-OFF-038 [S] — Where the LAN coordinator fails, remaining devices SHALL elect a new coordinator within 30 seconds and SHALL reconcile divergent state on election.

21.7 Conflict Detection and Resolution

Conflicts arise when two devices, or a device and the server, modify the same entity while partitioned.

FR-OFF-040 [M] — The System SHALL classify every entity type by conflict strategy:

Entity	Strategy	Justification
Order (created on one device)	No conflict possible	Single-writer by construction
Order lines on a shared table	CRDT: add-wins set with per-line last-writer-wins on mutable fields	Two waiters may add to one table
Payment	Server-authoritative, idempotent by key	Money must not duplicate
Stock level	Not synced as a value — derived from movements	Movements are commutative additions
Stock movement	Append-only, commutative	Order of independent movements is irrelevant
Loyalty balance	Ledger append; server recomputes; overdraw flagged	Prevents double redemption
Cash session	Single-writer (one drawer, one device)	Physically single-writer
Reference data (menu, price, recipe)	Server-authoritative; client changes rejected	Configuration originates centrally
Stock count session	Exclusive lock by scope; second submission rejected	Two counts of one shelf are meaningless
Employee clock event	Append-only; duplicates detected by (employee, type, time window)	

Entity	Strategy	Justification
Table state	Last-writer-wins with HLC	Low stakes, high churn

FR-OFF-041 [M] — Ordering across devices SHALL use Hybrid Logical Clocks, which preserve causality without depending on synchronised wall clocks.

On local event:

```
l' = max(l, physical_time)
c' = (l' == l) ? c + 1 : 0
```

On receiving message with (l_msg, c_msg):

```
l' = max(l, l_msg, physical_time)
c' = if l' == l == l_msg then max(c, c_msg) + 1
     else if l' == l      then c + 1
     else if l' == l_msg  then c_msg + 1
     else 0
```

Rationale: Device clocks in restaurants are frequently wrong — sometimes by hours, occasionally by years after a factory reset. Pure wall-clock ordering would allow a device with a fast clock to overwrite every other device’s changes indefinitely. HLC bounds clock skew’s effect while preserving happens-before relationships, and it degrades gracefully rather than catastrophically.

FR-OFF-042 [M] — The System SHALL detect device clock skew exceeding a configurable threshold (default 5 minutes) on every sync, SHALL record it, SHALL alert the branch manager, and SHALL preserve the device’s original timestamp alongside the server-corrected one.

FR-OFF-043 [M] — Conflicts that cannot be resolved automatically SHALL be recorded in a conflict register, SHALL raise an alert, and SHALL be presented to a manager for manual resolution with both versions displayed.

FR-OFF-044 [M] — All automatic conflict resolutions SHALL be recorded in the audit log with both input states and the applied rule.

21.8 Server-Side Revalidation

FR-OFF-045 [M] — On receiving an operation, the server SHALL revalidate all financially significant computations — prices, discounts, taxes, totals, loyalty accrual — rather than trusting client-computed values.

FR-OFF-046 [M] — Where server revalidation produces a different result, the server SHALL accept the transaction (the sale physically occurred), record both values, and raise a reconciliation exception for review.

Rationale: Rejecting a synced sale because the server disagrees about a price is not an option: the customer already paid and left. The correct behaviour is to accept the record of what happened, flag the discrepancy, and let a human decide. Systems that reject on mismatch lose revenue records, which is far worse than recording a two-piastre difference.

FR-OFF-047 [M] — Systematic revalidation mismatches from one terminal SHALL be treated as a signal of stale reference data or client tampering, and SHALL escalate to a platform alert after a configurable count.

21.9 Shared Conformance Test Suite

FR-OFF-050 [M] — Business logic that must produce identical results on client and server SHALL be specified as a language-neutral test corpus, executed by both the Dart client test suite and the TypeScript server test suite in CI.

Scope of the corpus: price resolution, modifier price computation, discount application and distribution, promotion evaluation, tax computation and rounding, service charge, cash rounding, loyalty accrual, and recipe expansion to base ingredients.

Corpus format:

```
{
  "id": "tax-eg-inclusive-multi-rate-001",
  "description": "EG: tax-inclusive pricing, two rates, order discount distributed",
  "context": {
    "countryPack": "EG-2026.1",
    "currency": "EGP",
    "pricingMode": "tax_inclusive",
    "roundingMode": "HALF_UP"
  },
  "input": {
    "lines": [
      { "itemId": "burger", "qty": 2, "unitPrice": 12000, "taxClass": "standard" },
      { "itemId": "water", "qty": 1, "unitPrice": 1500, "taxClass": "zero" }
    ],
    "orderDiscount": { "type": "percentage", "value": 10 }
  },
  "expected": {
    "subtotal": 25500,
    "discountTotal": 2550,
    "taxTotal": 2661,
    "grandTotal": 22950,
    "lineTax": [2661, 0]
  }
}
```

FR-OFF-051 [M] — Any divergence between client and server on the corpus SHALL block release.

FR-OFF-052 [M] — Every production discrepancy detected under FR-OFF-046 SHALL be triaged, and where it reveals a logic divergence, a new corpus case SHALL be added before the fix is merged.

21.10 Offline Non-Functional Requirements

NFR-REL-010 [M] — No committed sale SHALL be lost under any single-device failure, including abrupt power loss, application crash, or storage exhaustion.

NFR-REL-011 [M] — The System SHALL guarantee at-most-once financial effect for any operation, enforced by idempotency keys.

NFR-PERF-020 [M] — A backlog of 5,000 queued operations SHALL synchronise within 5 minutes on a 2 Mbps connection.

NFR-PERF-021 [M] — Local order persistence SHALL complete within 50 ms at p95 on the reference device.

NFR-CAP-001 [M] — The local store SHALL accommodate at least 20,000 orders and their lines without performance degradation beyond stated budgets.

21.11 Use Case: UC-OFF-01 — Six-Hour Internet Outage During Peak Trading

Preconditions: Branch with 3 POS terminals, 2 KDS screens, one shared LAN. Internet fails at 12:00.

Flow:

1. **12:00:00** — Terminals detect loss of server reachability within 10 seconds and transition to Offline mode. A subtle indicator appears. No dialog interrupts the cashier.
2. **12:00:10** — LAN coordinator election completes; Terminal 1 becomes coordinator.
3. **12:00–18:00** — 412 orders are processed. All order capture, modifier selection, pricing, tax, discounts within manager-PIN authority, cash and card payment recording, receipt printing, and kitchen routing function normally.
4. Kitchen routing operates over the LAN. KDS screens receive tickets within the normal latency budget. Bumps propagate back to the POS.
5. Two remote approval requests cannot reach absent managers. Per the branch's configured policy (FR-SEC-035), the on-site supervisor approves by PIN, and both are flagged for retrospective review.
6. A cashier attempts to redeem 4,000 loyalty points. The offline redemption limit is 2,000; the excess is declined with a clear message and the customer is offered the permitted amount.
7. Fiscal receipts are issued from the terminal's pre-allocated fiscal number block. 412 numbers are consumed from a block of 500.
8. Shift changes occur at 15:00. Cash sessions close and open locally with blind counts.
9. **18:00** — Connectivity returns. Terminals detect it and begin prioritised sync.
10. Orders and payments sync first, in causal order, in batches of 500. 412 orders, 439 payments, 6 cash sessions, and 1,204 audit events transmit over approximately 90 seconds.
11. The server revalidates every order. Four orders show a two-piastre tax difference caused by a price change published at 13:00 that the terminals never received. All four are accepted; four reconciliation exceptions are raised.

12. Inventory depletion is applied server-side in causal order. Three items go negative, indicating an unrecorded goods receipt; negative-stock alerts are raised.
13. Fiscal documents are queued for submission through the outbox. Submission proceeds at the authority's rate limit over 12 minutes.
14. The loyalty ledger is reconciled. No double-redemption is detected.
15. The manager's dashboard shows an outage summary: duration, orders captured, exceptions raised, and items requiring attention.

Postconditions: No sale lost. No sale duplicated. Four exceptions logged for review. Fiscal sequence intact with 88 unused numbers reported void. Inventory reflects all consumption.

Chapter 22 — Localisation & Country Packs | اللغات وحزم الدول

22.1 Bilingual and Multilingual Design

FR-LOC-001 [M] — The System SHALL support Arabic and English as first-class languages across every surface: POS, KDS, dashboard, mobile, receipts, tickets, reports, exports, emails, and error messages.

FR-LOC-002 [M] — The Arabic interface SHALL use clear Modern Standard Arabic, not a regional dialect.

FR-LOC-003 [M] — Layout direction SHALL follow the selected language: full RTL mirroring for Arabic, LTR for English, applied to layout, iconography with directional meaning, progress indicators, and navigation.

FR-LOC-004 [M] — The System SHALL correctly render bidirectional text: an Arabic dish name containing a Latin brand name, or an English item name containing an Arabic word, SHALL render with correct ordering per the Unicode Bidirectional Algorithm.

Rationale: Bidirectional rendering failures are the clearest signal that Arabic was retrofitted. “Coca Cola ٢ لتر” rendering as “٢ Coca Cola لتر” in the wrong visual order looks broken to every Arabic reader and is immediately noticed in a demo.

FR-LOC-005 [M] — Numeral rendering SHALL be configurable between Western (0123456789) and Arabic-Indic (٠١٢٣٤٥٦٧٨٩), independently for the interface and for printed documents.

FR-LOC-006 [M] — All user-authored content — item names, descriptions, categories, modifiers, reason codes, station names, expense categories — SHALL be storable in every enabled language, with a designated fallback.

FR-LOC-007 [M] — Where a translation is missing, the System SHALL display the fallback language content rather than an empty string or a key.

FR-LOC-008 [M] — Language SHALL be selectable per user, per terminal, and per printed document type independently. A branch may run an English-language dashboard, an Arabic POS, an Urdu kitchen ticket, and a bilingual customer receipt.

FR-LOC-009 [S] — The System SHALL support additional languages by translation pack: French, Turkish, Urdu, Bengali, Hindi, Tagalog.

Rationale: Kitchen staff in Gulf restaurants frequently read Urdu, Bengali, or Tagalog more fluently than Arabic or English. Kitchen ticket language is a genuine operational requirement, not a nicety — a cook who cannot read the ticket produces the wrong dish.

FR-LOC-010 [M] — Dates, times, numbers, and currency SHALL be formatted per locale, and the System SHALL support both Gregorian and Hijri calendar display where the country pack enables it.

FR-LOC-011 [M] — Arabic typography SHALL use fonts selected and tested for legibility at small sizes on thermal printers and at 2-metre viewing distance on kitchen displays.

FR-LOC-012 [M] — Thermal printing of Arabic SHALL be tested across the supported printer matrix, with the System supporting both native Arabic font rendering and image-based rendering as a fallback for printers with poor Arabic support.

Rationale: Many thermal printers in the market have incomplete or broken Arabic support, particularly for letter joining and diacritics. Rendering the receipt as a bitmap is slower but always correct. Both paths are required, selectable per printer model.

22.2 Country Pack Architecture

A Country Pack is a signed, versioned configuration bundle that defines everything jurisdiction-specific.

countryPack:

```
code: EG
version: "2026.1"
effectiveFrom: "2026-01-01"
signature: "..."
```

currency:

```
code: EGP
exponent: 2
symbol: "ج.م"
symbolPosition: suffix
cashRounding: { enabled: false }
```

tax:

```
engine: "vat_standard"
registrationLabel: "الرقم الضريبي"
registrationPattern: "^[0-9]{9}$"
pricingMode: tax_inclusive
computationLevel: line
roundingMode: HALF_UP
roundingPrecision: 2
```

```

    classes:
      - { code: standard, rate: 14.0, label: { ar: "قياسي", en:
"Standard" } }
      - { code: reduced, rate: 5.0 }
      - { code: zero, rate: 0.0 }
      - { code: exempt, rate: null }
    serviceChargeTaxable: true
    orderTypeOverrides: []

    invoice:
      template: "eg_standard_v3"
      requiredFields: [seller_trn, buyer_trn_if_b2b, uuid,
submission_datetime, qr]
      sequenceStrategy: pre_allocated_block
      blockSize: 500
      voidUnusedOnExpiry: true
      qr:
        standard: "eta_qr_v1"

    fiscal:
      provider: "eta_egypt"
      mode: e_receipt
      endpointRef: "secret://eta/endpoint"
      submissionWindowHours: 24
      retryPolicy: { attempts: 8, backoff: exponential, maxDelayMinutes:
120 }
      offlineBehaviour: queue_and_submit

    labour:
      standardWeeklyHours: 48
      overtimeMultiplier: 1.35
      nightShiftDefinition: { from: "22:00", to: "06:00" }

    calendar:
      weekStart: saturday
      weekend: [friday]
      publicHolidays: [...]
      observances: [ramadan, eid_al_fitr, eid_al_adha]

    legal:
      receiptFooter:
        ar: "... "
        en: "... "
      dataRetentionYears: 5
      requiresBuyerDetailsAbove: 5000000

```

FR-LOC-020 [M] — All jurisdiction-specific behaviour SHALL be driven by the country pack. No country-specific logic SHALL be compiled into core application code.

FR-LOC-021 [M] — Country packs SHALL be versioned with effective dates, and historical transactions SHALL be interpreted under the pack version in force at their transaction time.

FR-LOC-022 [M] — Country packs SHALL be cryptographically signed, and the System SHALL reject an unsigned or invalidly-signed pack.

FR-LOC-023 [M] — Country packs SHALL be validated against a conformance test suite before activation, covering tax computation, rounding, invoice field completeness, and QR generation.

FR-LOC-024 [M] — Country packs SHALL be distributed to offline terminals in advance of their effective date and SHALL activate locally by date.

FR-LOC-025 [M] — The tax engine referenced by a pack SHALL be one of a registered set of strategy implementations. Adding a genuinely novel tax model requires a new strategy implementation; changing rates, classes, or rounding does not.

22.3 Implemented Country Packs

22.3.1 Egypt (EG)

IR-LOC-EG-001 [M] — The System SHALL integrate with the Egyptian Tax Authority e-Receipt system for B2C transactions and the e-Invoice system for B2B transactions.

IR-LOC-EG-002 [M] — The System SHALL submit receipt documents within the mandated window, SHALL store the returned UUID and submission status, and SHALL render the required QR code on the printed receipt.

IR-LOC-EG-003 [M] — The System SHALL handle authority downtime by queueing submissions and retrying per the pack's retry policy, without blocking sales.

IR-LOC-EG-004 [M] — The System SHALL support the required document types including receipt, invoice, credit note, and debit note, with correct referencing of the original document on corrections.

22.3.2 Saudi Arabia (SA)

IR-LOC-SA-001 [M] — The System SHALL comply with ZATCA Fatoora Phase 2 requirements for the applicable taxpayer wave.

IR-LOC-SA-002 [M] — The System SHALL generate simplified tax invoices for B2C with a TLV-encoded, Base64-represented QR code containing seller name, VAT number, timestamp, invoice total, VAT total, and the cryptographic stamp and hash where required.

IR-LOC-SA-003 [M] — The System SHALL maintain the invoice hash chain (PIH) required by the specification, with correct handling across offline periods.

IR-LOC-SA-004 [M] — The System SHALL support the reporting model for simplified invoices (submission within 24 hours) and the clearance model for standard invoices (pre-clearance before issuance).

Risk: The clearance model requires a synchronous authority round-trip before an invoice may be issued, which conflicts directly with offline operation. The pack therefore restricts B2B standard invoices to online operation while permitting B2C simplified invoices offline. This constraint is a legal reality and must be surfaced clearly in the interface rather than worked around.

IR-LOC-SA-005 [M] — The System SHALL manage cryptographic stamp identifiers and certificate lifecycle, alerting well in advance of certificate expiry.

22.3.3 United Arab Emirates (AE)

IR-LOC-AE-001 [M] — The System SHALL apply the applicable VAT rate with correct treatment of zero-rated and exempt supplies.

IR-LOC-AE-002 [M] — The System SHALL produce tax invoices containing all fields mandated by FTA regulations, in Arabic where required.

IR-LOC-AE-003 [M] — The System SHALL support municipality fees and tourism dirham charges as additional tax components where the emirate applies them.

IR-LOC-AE-004 [C] — The System SHALL be architecturally prepared for the UAE e-invoicing mandate, with the fiscal provider interface already abstracted.

22.3.4 Additional Packs

Jordan, Kuwait, Qatar, Bahrain, Oman, Morocco, and Tunisia are specified as Phase 3 packs. Each requires: currency and exponent, tax model, invoice content rules, fiscal integration where mandated, labour rules, and calendar.

FR-LOC-030 [S] — The System SHALL provide a country pack authoring and validation tool so that a new pack can be produced, tested against the conformance suite, and certified without core code changes.

FR-LOC-031 [M] — A country pack SHALL NOT be activatable in production until it passes the full conformance suite and is signed by an authorised release key.

Chapter 23 — External Integrations | التكامل الخارجي

23.1 Integration Principles

FR-INT-001 [M] — Every external system SHALL be accessed through an Anti-Corruption Layer that translates between the external model and the ROS domain model. External data structures SHALL NOT propagate into the domain.

FR-INT-002 [M] — All outbound integration calls SHALL be dispatched through the transactional outbox and SHALL be idempotent.

FR-INT-003 [M] — All inbound integration calls SHALL be authenticated, rate-limited, validated against a schema, and idempotent by an external reference key.

FR-INT-004 [M] — Integration failures SHALL be visible in an integration health dashboard showing per-connector status, last success, error rate, and queue depth.

FR-INT-005 [M] — Integration credentials SHALL be stored encrypted, SHALL be rotatable without downtime, and SHALL never appear in logs or error messages.

FR-INT-006 [M] — Each connector SHALL implement a circuit breaker: after a configured failure threshold, calls are suspended and queued, with periodic probing for recovery.

23.2 Payment Terminals

IR-INT-010 [M] — The System SHALL integrate with certified payment terminals over the terminal vendor's local protocol (typically TCP/IP on the local network or USB).

IR-INT-011 [M] — The integration SHALL support: sale, refund, void, pre-authorisation where available, tip adjustment where available, batch settlement query, and last-transaction query.

IR-INT-012 [M] — The last-transaction query SHALL be invoked before initiating a new authorisation following any communication failure, to determine whether the prior transaction succeeded.

Rationale: This single requirement eliminates the most damaging payment defect: the terminal approved the charge, the response was lost, the cashier retried, and the customer was charged twice. Querying the terminal's last transaction before retrying resolves the ambiguity deterministically.

IR-INT-013 [M] — The System SHALL record: authorisation code, terminal reference, card scheme, last four digits, entry mode, and approval status. It SHALL NOT record PAN, CVV, or track data.

IR-INT-014 [S] — Target integrations: Network International, Geidea, PayTabs, Fawry, Amwal, and Checkout.com terminal SDKs, prioritised by market.

23.3 Delivery Aggregators

IR-INT-020 [M] — The System SHALL accept inbound orders from aggregator platforms, mapping them to ROS orders with `channel = aggregator` and the aggregator's reference retained.

IR-INT-021 [M] — Aggregator menu items SHALL be mapped to ROS menu items by an explicit mapping table, and unmapped items SHALL be rejected with a clear operational alert rather than silently created.

IR-INT-022 [M] — The System SHALL push order state changes (accepted, preparing, ready, collected, rejected) back to the aggregator where the API supports it.

IR-INT-023 [S] — The System SHALL push item availability changes to connected aggregators within 60 seconds.

IR-INT-024 [S] — The System SHALL push menu and price changes to aggregators where the API supports it, with a review-and-confirm step before publishing.

IR-INT-025 [S] — The System SHALL record per-aggregator commission structures and SHALL compute net revenue per order accordingly.

IR-INT-026 [S] — The System SHALL support aggregator payout reconciliation per FR - FIN - 012.

IR-INT-027 [M] — Aggregator orders SHALL deplete inventory identically to POS orders. No inventory path shall be channel-specific.

IR-INT-028 [S] — Target integrations: Talabat, HungerStation, Jahez, Careem, Deliveroo, Elmenus, Noon Food, and a generic webhook connector for unlisted platforms.

23.4 Accounting Systems

IR-INT-030 [S] — The System SHALL generate a daily journal export per branch containing: revenue by category, tax by rate, discounts, COGS, inventory movement value, cash by tender, and expenses.

IR-INT-031 [S] — The System SHALL support a configurable chart-of-accounts mapping, so that each ROS category maps to the tenant's account codes.

IR-INT-032 [S] — Exports SHALL be available as CSV and in provider-specific formats for QuickBooks, Xero, Zoho Books, and Odoo.

IR-INT-033 [C] — The System SHALL support direct API posting to QuickBooks Online and Xero, with a reconciliation view showing posted versus pending.

IR-INT-034 [M] — Exported journals SHALL balance. The System SHALL validate that debits equal credits before export and SHALL block an unbalanced export.

23.5 Notifications

IR-INT-040 [M] — The System SHALL support SMS, WhatsApp Business, email, and mobile push as notification channels.

IR-INT-041 [M] — Notification templates SHALL be localised and tenant-customisable within defined variable slots.

IR-INT-042 [M] — The System SHALL respect consent flags and SHALL suppress marketing notifications absent consent, while permitting transactional notifications (receipt, order status) per applicable law.

IR-INT-043 [S] — The System SHALL record delivery status per notification and SHALL surface persistent failures.

23.6 Public API and Webhooks

IR-INT-050 [S] — The System SHALL expose a REST API covering: menu, orders, inventory, customers, and reports.

IR-INT-051 [M] — API access SHALL use scoped API keys or OAuth 2.0 client credentials, with permissions expressed in the same catalogue as user permissions.

IR-INT-052 [M] — API keys SHALL be scoped to a tenant and optionally to specific branches, SHALL be revocable, and SHALL be rotatable without downtime.

IR-INT-053 [S] — The System SHALL support webhooks for the domain event catalogue, with per-endpoint secrets, HMAC signature headers, retry with exponential backoff, and a dead-letter view.

IR-INT-054 [M] — Webhook payloads SHALL include the full event envelope, and consumers SHALL be documented as required to handle at-least-once delivery.

23.7 Hardware

IR-INT-060 [M] — The System SHALL support ESC/POS thermal printers over network, USB, and Bluetooth.

IR-INT-061 [M] — The System SHALL support cash drawers opened via the printer's kick-out port.

IR-INT-062 [M] — The System SHALL support 1D and 2D barcode scanners in keyboard-wedge and SDK modes.

IR-INT-063 [S] — The System SHALL support serial and Bluetooth weighing scales for weighed items.

IR-INT-064 [S] — The System SHALL support customer-facing displays showing order contents and total.

IR-INT-065 [S] — The System SHALL support label printers for production labelling with item name, batch, production date, expiry, and barcode.

IR-INT-066 [M] — Printer configuration SHALL support routing rules: which document types print to which printer, with fallback printers on failure.

Chapter 24 — Design Patterns & Engineering Practices | أنماط التصميم

24.1 How Patterns Are Used in This System

A pattern is a named solution to a recurring problem, applied where that problem actually exists. Patterns applied where the problem does not exist are pure cost: more indirection, more files, more concepts to learn, no benefit.

This chapter lists the patterns ROS uses, the specific problem each solves in this system, and where it is applied. It also lists patterns deliberately *not* used, because an architecture is defined as much by what it excludes.

24.2 Architectural Patterns

24.2.1 Layered / Clean Architecture

Problem: Business rules become entangled with frameworks, databases, and delivery mechanisms, making them untestable and making infrastructure changes require domain rewrites.

Application: Every module follows the domain → application → infrastructure/presentation structure of 5.4, with dependencies pointing inward.

Concrete benefit: Recipe expansion, variance computation, and tax calculation are pure functions over domain objects, testable in milliseconds without a database. These are the three most correctness-critical algorithms in the system.

24.2.2 Modular Monolith

Problem: Microservices impose distributed-system costs disproportionate to a nine-engineer team; an undisciplined monolith becomes unmaintainable.

Application: Enforced module boundaries per 5.2.3.

24.2.3 CQRS (Command Query Responsibility Segregation) — Applied Selectively

Problem: The write model (normalised, aggregate-oriented, consistency-focused) and the read model (denormalised, query-oriented, speed-focused) have opposing optimal shapes.

Application: ROS applies *lightweight* CQRS: commands go through aggregates and repositories; queries bypass the domain model entirely and read from purpose-shaped projections and views.

```
// Command side — through the aggregate, enforcing invariants
class ApplyDiscountHandler {
  constructor(
    private readonly orders: OrderRepository,
    private readonly policy: DiscountPolicy,
    private readonly approvals: ApprovalService,
  ) {}

  async execute(cmd: ApplyDiscountCommand): Promise<void> {
    const order = await this.orders.findById(cmd.orderId);

    const decision = this.policy.evaluate({
      order, discount: cmd.discount, actor: cmd.actor,
    });

    if (decision.requiresApproval && !cmd.approvalId) {
      throw new ApprovalRequiredError(decision.reason);
    }

    order.applyDiscount(cmd.discount, cmd.actor, cmd.approvalId); //
```

```

invariants live here
    await this.orders.save(order); //
events dispatched in UoW
  }
}

// Query side – no aggregate, no domain objects, shaped for the screen
class OrderListQuery {
  async execute(p: OrderListParams): Promise<OrderListRow[]> {
    return this.readDb.query(`
      SELECT o.id, o.order_number, o.state, o.grand_total,
             o.opened_at, t.label AS table_label, e.display_name AS
server_name
      FROM   orders o
      LEFT   JOIN tables t   ON t.id = o.table_id
      LEFT   JOIN employees e ON e.id = o.served_by
      WHERE  o.branch_id = $1 AND o.business_day = $2
      ORDER  BY o.opened_at DESC
      LIMIT  $3 OFFSET $4
    `, [p.branchId, p.businessDay, p.limit, p.offset]);
  }
}

```

Explicitly not applied: full CQRS with separate write and read databases and eventual consistency between them. The operational cost is not justified until reporting volume forces it, at which point the query side is already isolated and can be repointed.

24.2.4 Event-Driven Architecture (In-Process)

Problem: A completed order must affect six modules. Direct calls from Sales into Inventory, Costing, Treasury, Fiscal, and Customer would make Sales depend on everything, and every new consumer would require modifying Sales.

Application: Aggregates record events; the unit of work dispatches them within the transaction.

```

abstract class AggregateRoot {
  private _events: DomainEvent[] = [];
  protected record(e: DomainEvent) { this._events.push(e); }
  pullEvents(): DomainEvent[] { const e = this._events; this._events = []; return e; }
}

class Order extends AggregateRoot {
  complete(payments: Payment[], at: Instant): void {
    if (this.state !== OrderState.Open && this.state !==
OrderState.PartiallyPaid)
      throw new InvalidOrderStateError(this.state, 'complete');

    const paid = Money.sum(payments.map(p => p.amount));
  }
}

```

```

        if (paid.lessThan(this.grandTotal.minus(this.compTotal)))
            throw new InsufficientPaymentError(this.grandTotal, paid);

        this.state = OrderState.Completed;
        this.completedAt = at;

        this.record(new OrderCompleted({
            orderId: this.id, branchId: this.branchId, businessDay:
this.businessDay,
            lines: this.lines.map(l => l.toConsumptionSpec()),
            totals: this.totals(), payments: payments.map(p =>
p.toSummary()),
            completedAt: at, customerId: this.customerId,
        }));
    }
}

```

24.2.5 Transactional Outbox

Problem: Dual-write inconsistency between the database and external systems.

Application: All external effects (fiscal submission, aggregator push, notification, webhook) per 5.5.3.

24.2.6 Anti-Corruption Layer

Problem: External APIs have models that are wrong for our domain — inconsistent, aggregator-specific, and subject to change without notice. Letting them into the domain couples the core to the vagaries of third parties.

Application: Every integration.

```

// External shape – dictated by the aggregator, not by us
interface TalabatOrderPayload {
    order_id: string;
    branch_code: string;
    items: Array<{ sku: string; qty: number; price: number; addons:
any[] }>;
    payment: { method: string; paid: boolean; amount: number };
    customer?: { name?: string; phone?: string };
}

```

```

// ACL – the only place this shape is known
class TalabatOrderTranslator implements
AggregatorTranslator<TalabatOrderPayload> {
    constructor(
        private readonly mapping: ItemMappingRepository,
        private readonly branches: BranchRepository,
    ) {}

    async toDomain(p: TalabatOrderPayload, tenantId: TenantId):

```

```

Promise<CreateOrderCommand> {
    const branch = await this.branches.findByExternalCode(tenantId,
'talabat', p.branch_code);
    if (!branch) throw new UnmappedBranchError('talabat',
p.branch_code);

    const lines = await Promise.all(p.items.map(async i => {
        const m = await this.mapping.resolve(tenantId, 'talabat',
i.sku);
        if (!m) throw new UnmappedItemError('talabat', i.sku);    // fail
loudly – never invent an item
        return {
            menuItemId: m.menuItemId,
            variantId: m.variantId,
            quantity: Decimal.of(i.qty),
            unitPrice: Money.of(BigInt(Math.round(i.price * 100)),
branch.currency),
            modifiers: await this.translateAddons(tenantId, i.addons),
        };
    }));

    return new CreateOrderCommand({
        tenantId, branchId: branch.id,
        orderType: OrderType.Delivery, channel: Channel.Aggregator,
        aggregatorRef: p.order_id,
        idempotencyKey: `talabat:${p.order_id}`,
        lines,
        customer: p.customer?.phone
            ? { phone: PhoneNumber.parse(p.customer.phone,
branch.countryCode), name: p.customer.name }
            : undefined,
    });
}
}

```

24.3 Domain Patterns

24.3.1 Aggregate

Applied per 7.1. The consistency boundaries chosen are Order, Recipe version, CountSession, CashSession, PurchaseOrder, and ProductionOrder. Stock level is deliberately *not* an aggregate: it is a projection over an append-only movement log, which is what makes concurrent depletion from multiple terminals correct without locking.

24.3.2 Value Object

Applied to Money, Quantity, UnitOfMeasure, TaxRate, Percentage, DateRange, PhoneNumber, Address, and all identifier types.

Concrete benefit: the type system prevents entire defect classes. `Money.plus(Quantity)` does not compile. Adding EGP to SAR throws at runtime rather than producing a plausible wrong number.

24.3.3 Repository

Problem: Domain code must persist and retrieve aggregates without knowing about SQL.

```
// Port – in application/ports, expressed in domain terms
interface OrderRepository {
    findById(id: OrderId): Promise<Order>;
    findOpenByBranch(branchId: BranchId): Promise<Order[]>;
    nextOrderNumber(branchId: BranchId, day: BusinessDay):
    Promise<OrderNumber>;
    save(order: Order): Promise<void>;
}

// Adapter – in infrastructure/persistence
class PostgresOrderRepository implements OrderRepository {
    constructor(private readonly uow: UnitOfWork, private readonly bus:
    EventBus) {}

    async save(order: Order): Promise<void> {
        await this.uow.run(async tx => {
            await tx.upsert('orders', OrderMapper.toRow(order));
            await tx.replaceChildren('order_lines', 'order_id', order.id,
            order.lines.map(OrderLineMapper.toRow));
            for (const e of order.pullEvents()) await this.bus.dispatch(e,
            tx); // same transaction
        });
    }
}
```

24.3.4 Specification

Problem: Business rules for selecting or validating entities get duplicated across queries, validation, and UI.

```
interface Specification<T> {
    isSatisfiedBy(candidate: T): boolean;
    toQuery(): QueryFragment;
}

class ItemRequiresReorder implements Specification<StockLevelView> {
    constructor(private readonly horizonDays: number) {}

    isSatisfiedBy(s: StockLevelView): boolean {
        const projected = s.onHand + s.onOrder -
        s.forecastUsage(this.horizonDays);
        return projected < s.reorderPoint;
    }
}
```

```

    }

    toQuery(): QueryFragment {
        return sql`(on_hand + on_order - forecast_usage($
{this.horizonDays})) < reorder_point`;
    }
}

// Composable
const urgent = new ItemRequiresReorder(3).and(new
IsHighValue(50_00n)).and(new HasApprovedSupplier());

```

Concrete benefit: the same rule is used to run the nightly reorder job, to render the low-stock badge, and to filter the purchasing screen. It cannot drift out of sync between them.

24.3.5 Policy (Strategy for Business Rules)

Problem: Rules such as “when does a discount need approval” vary by tenant, branch, and role, and are the rules most likely to change.

```

interface DiscountPolicy {
    evaluate(ctx: DiscountContext): DiscountDecision;
}

class ThresholdDiscountPolicy implements DiscountPolicy {
    constructor(private readonly config: DiscountThresholds) {}

    evaluate(ctx: DiscountContext): DiscountDecision {
        const limits = this.config.forRole(ctx.actor.roleId,
ctx.order.branchId);

        if (ctx.discount.percentage > limits.maxPercentage)
            return DiscountDecision.requiresApproval(`exceeds $
{limits.maxPercentage}%`);

        if (ctx.discount.amount.greaterThan(limits.maxAmount))
            return DiscountDecision.requiresApproval(`exceeds $
{limits.maxAmount.format()}`);

        if (ctx.shiftDiscountCount >= limits.maxPerShift)
            return DiscountDecision.requiresApproval('shift discount limit
reached');

        if (ctx.order.hasPaymentStarted && !limits.allowAfterPayment)
            return DiscountDecision.denied('discount not permitted after
payment started');

        return DiscountDecision.allowed();
    }
}

```

24.3.6 Domain Event

Applied per 5.5.4.

24.3.7 Unit of Work

Problem: One use case touches several repositories and must succeed or fail as one.

Application: A `UnitOfWork` wraps a database transaction, sets the RLS tenant context, collects aggregate events, and dispatches them before commit. Every command handler executes inside exactly one unit of work.

24.4 Creational and Structural Patterns

24.4.1 Factory

Problem: Constructing an `Order` correctly requires resolving prices, tax classes, country pack, business day, and order number — knowledge that does not belong in the `Order` constructor.

```
class OrderFactory {
  constructor(
    private readonly catalogue: CataloguePort,
    private readonly pricing: PricingService,
    private readonly tax: TaxEngineResolver,
    private readonly numbers: OrderNumberAllocator,
    private readonly clock: Clock,
  ) {}

  async create(cmd: CreateOrderCommand, branch: Branch):
  Promise<Order> {
    const day = BusinessDay.resolve(this.clock.now(),
branch.dayBoundary, branch.timezone);
    const number = await this.numbers.allocate(branch.id, day);
    const engine = this.tax.forCountryPack(branch.countryPackVersion);

    const order = Order.open({
      id: cmd.orderId ?? OrderId.generate(),
      branchId: branch.id, terminalId: cmd.terminalId,
      orderNumber: number, businessDay: day,
      orderType: cmd.orderType, channel: cmd.channel,
      currency: branch.currency, openedBy: cmd.actor.employeeId,
      openedAt: this.clock.now(),
    });

    for (const l of cmd.lines) {
      const priced = await this.pricing.resolve(l, branch,
cmd.orderType, this.clock.now());
      order.addLine(priced, engine);
    }
    return order;
  }
}
```

```
}  
}
```

24.4.2 Builder

Applied to report definitions, complex query construction, and receipt/document composition, where an object has many optional parts assembled conditionally.

24.4.3 Adapter

Applied to every external system: payment terminals, printers, fiscal providers, aggregators, accounting exports, notification gateways.

24.4.4 Facade

Applied to each module's contract: a single, small, stable interface hiding the module's internal structure from its consumers.

24.4.5 Composite

Problem: A recipe contains stock items and other recipes; both must be traversable uniformly for cost computation and ingredient expansion.

```
interface RecipeComponent {  
    costPerUnit(ctx: CostingContext): Money;  
    expandToStockItems(qty: Quantity, ctx: CostingContext):  
    StockConsumption[];  
}  
  
class StockItemComponent implements RecipeComponent {           // leaf  
    expandToStockItems(qty: Quantity): StockConsumption[] {  
        return [{ stockItemId: this.id, quantity:  
qty.convertTo(this.baseUnit) }];  
    }  
}  
  
class SubRecipeComponent implements RecipeComponent {           //  
    composite  
    expandToStockItems(qty: Quantity, ctx: CostingContext):  
    StockConsumption[] {  
        if (ctx.depth > 10) throw new RecipeDepthExceededError(this.id);  
        const factor = qty.dividedBy(this.version.yieldQuantity);  
        return this.version.lines.flatMap(l =>  
            l.component.expandToStockItems(l.quantity.times(factor),  
ctx.deeper()),  
        );  
    }  
}
```


24.4.6 Decorator

Applied to cross-cutting concerns on handlers: logging, metrics, authorisation, caching, retry, and idempotency, each layered without modifying the handler.

```
const handler = new IdempotencyDecorator(  
  new AuthorizationDecorator(  
    new MetricsDecorator(  
      new LoggingDecorator(  
        new CompleteOrderHandler(orders, events, clock)))))
```

24.5 Behavioural Patterns

24.5.1 State

Problem: Order and Ticket lifecycles have many states with different permitted operations. Encoding this in conditionals produces a method with fourteen branches that nobody dares modify.

```
abstract class OrderState {  
  abstract canAddLine(): boolean;  
  abstract canVoidLine(): boolean;  
  abstract canApplyDiscount(): boolean;  
  abstract canAcceptPayment(): boolean;  
  abstract canCancel(): boolean;  
  abstract transitionsTo(): OrderStateName[];  
}  
  
class CompletedState extends OrderState {  
  canAddLine()      { return false; }  
  canVoidLine()     { return false; }  
  canApplyDiscount() { return false; }  
  canAcceptPayment() { return false; }  
  canCancel()       { return false; }  
  transitionsTo()   { return ['PARTIALLY_REFUNDED', 'REFUNDED']; }  
}
```

Concrete benefit: adding a state is adding a class, and the compiler enumerates every question the new state must answer. Nothing is silently forgotten.

24.5.2 Strategy

Applied to: inventory costing (FIFO / weighted average / standard), batch consumption (FIFO / FEFO), tax computation per country pack, reorder forecasting, combo revenue allocation, and conflict resolution per entity type.

```
interface CostingStrategy {  
  valueConsumption(item: StockItem, qty: Quantity, loc: LocationId):  
    Promise<CostedConsumption>;  
}
```

```

class FifoCostingStrategy implements CostingStrategy {
    async valueConsumption(item, qty, loc): Promise<CostedConsumption> {
        const batches = await this.batches.availableOrdered(item.id, loc,
'received_at ASC');
        return this.drawDown(batches, qty);
    }
}

class FefoCostingStrategy implements CostingStrategy {
    async valueConsumption(item, qty, loc): Promise<CostedConsumption> {
        const batches = await this.batches.availableOrdered(item.id, loc,
'expiry_date ASC NULLS LAST');
        return this.drawDown(batches, qty);
    }
}

```

24.5.3 Chain of Responsibility

Applied to the approval workflow: a request passes along a chain of approval levels until one accepts it or the chain is exhausted.

24.5.4 Observer

Applied to the KDS: displays subscribe to ticket changes for their station and re-render on notification, rather than polling.

24.5.5 Template Method

Applied to report generation, where the skeleton (resolve parameters — check permissions — fetch — aggregate — format — deliver) is fixed and each report overrides fetching and aggregation.

24.5.6 Command

Every write operation is a command object with a handler, giving uniform validation, authorisation, idempotency, audit, and retry handling.

24.5.7 Saga / Process Manager

Applied to multi-step processes with compensations: fiscal submission with retry and fallback, central kitchen production and distribution, and aggregator order acceptance.

24.6 Data Patterns

24.6.1 Event Sourcing — Applied Selectively

Full event sourcing is **not** used. Two specific data sets, however, are event-sourced in effect:

- **Stock movements** — current stock is a fold over an append-only movement log.
- **Loyalty transactions** — balance is a fold over an append-only ledger.

Rationale for selective application: these two are exactly the data where (a) history is the answer to the most important questions, (b) concurrent writers must not conflict, and (c) offline devices must be able to append without coordination. Event sourcing solves all three. Applying it to menu configuration, by contrast, would add complexity for no benefit.

24.6.2 Materialised View / Projection

Applied to `stock_levels`, reporting rollups, and dashboard aggregates. Every projection has a documented rebuild procedure and a reconciliation job.

24.6.3 Snapshot

Applied to captured values on transactions — item names, prices, tax classes, costs, recipe versions — per BR-POS-004.

24.6.4 Optimistic Concurrency

Aggregates carry a version. Updates assert the expected version and fail on mismatch, forcing the caller to reload. Pessimistic locking is used only for order-number allocation and count-session exclusivity.

24.6.5 Soft Delete with Referential Preservation

Master data (items, employees, suppliers) is deactivated rather than deleted while transactions reference it. Hard deletion is available only for records with no references and only via an explicitly audited administrative operation.

24.7 Resilience Patterns

Pattern	Application
Retry with exponential backoff and jitter	Sync, fiscal submission, aggregator push, notification
Circuit breaker	Every external connector
Bulkhead	Separate connection pools and worker pools for transactional, analytical, sync, and integration work
Timeout	Every external call and every database statement, with explicit budgets
Dead letter queue	Failed outbox entries after exhausting retries, with an operator review interface
Idempotency key	Every command that can be retried
Graceful degradation	Loss of reporting, notifications, or aggregators never blocks order capture
Backpressure	Sync batch size limits and queue depth thresholds
Health check and readiness probe	Every service, with dependency-aware readiness

24.8 Anti-Patterns Explicitly Rejected

Anti-Pattern	Why It Is Rejected Here
Anemic domain model	Business rules scattered into services means no single place enforces an invariant; the Order aggregate would not be able to guarantee its own totals
Shared database between modules	Destroys boundaries and makes extraction impossible; enforced against by per-module DB grants
Distributed transactions (2PC)	Fragile, poorly supported, and unnecessary given the modular monolith
Floating-point money	Guarantees eventual reconciliation failures
Client-trusted totals	Server revalidates everything financially significant
UI-only permission checks	Every endpoint authorises server-side
Generic “utils” module	Becomes a dumping ground and a hidden dependency hub; shared code must have a named, bounded purpose
Country logic in <code>if (country === 'EG')</code>	Makes expansion a development project; violates CR-03
Synchronous fan-out to external systems inside a request	Couples user-facing latency to third-party availability
Premature microservices	See ADR-001
Nullable everything in the database	Makes invariants unexpressible at the schema level; columns are NOT NULL unless nullability carries meaning

Chapter 25 — Data Architecture | معمارية البيانات

25.1 Schema Organisation

The database uses PostgreSQL schemas to group tables by bounded context, giving a clear namespace and enabling per-module role grants.

Schema	Contents
identity	tenants, users, sessions, terminals, api_keys
org	brands, branches, warehouses, central_kitchens, stations, tables, settings
catalogue	menus, categories, menu_items, variants, modifier_groups, modifiers, combos, price_lists

Schema	Contents
production	recipes, recipe_versions, recipe_lines, yield_profiles
inventory	stock_items, uom, stock_batches, stock_movements, stock_levels, count_sessions, waste_records
procurement	suppliers, supplier_prices, requisitions, purchase_orders, goods_receipts, supplier_invoices
sales	orders, order_lines, order_line_modifiers, order_discounts, order_payments, refunds
kitchen	tickets, ticket_lines, station_routing_rules
workforce	employees, schedules, scheduled_shifts, attendance_records, clock_events
treasury	drawers, cash_sessions, cash_movements, expenses, day_closes
crm	customers, addresses, loyalty_accounts, loyalty_transactions, promotions
fiscal	country_packs, tax_documents, fiscal_submissions
governance	audit_entries, approval_requests, approval_decisions, anomaly_flags
sync	sync_batches, sync_operations, idempotency_keys, conflict_records, device_state
platform	outbox, jobs, notifications, feature_flags, migrations
analytics	fact_, dim_, rollup_*

25.2 Representative DDL

```
-- =====
-- sales.orders
-- =====
CREATE TABLE sales.orders (
    id                UUID                PRIMARY KEY,           -- ULID
    stored as UUID
    tenant_id         UUID                NOT NULL REFERENCES
identity.tenants(id),
    branch_id         UUID                NOT NULL REFERENCES
org.branches(id),
    terminal_id       UUID                NOT NULL REFERENCES
identity.terminals(id),

    order_number      VARCHAR(24)        NOT NULL,
```

business_day	DATE	NOT NULL,
order_type	sales.order_type_enum	NOT NULL,
channel	sales.channel_enum	NOT NULL,
state	sales.order_state_enum	NOT NULL,
table_id	UUID	REFERENCES org.tables(id),
guest_count	SMALLINT	CHECK (guest_count IS NULL OR
guest_count > 0),		
customer_id	UUID	REFERENCES crm.customers(id),
opened_by	UUID	NOT NULL REFERENCES
workforce.employees(id),		
served_by	UUID	REFERENCES
workforce.employees(id),		
closed_by	UUID	REFERENCES
workforce.employees(id),		
cash_session_id	UUID	REFERENCES
treasury.cash_sessions(id),		
currency	CHAR(3)	NOT NULL,
subtotal	BIGINT	NOT NULL DEFAULT 0,
discount_total	BIGINT	NOT NULL DEFAULT 0 CHECK
(discount_total >= 0),		
service_charge_total	BIGINT	NOT NULL DEFAULT 0 CHECK
(service_charge_total >= 0),		
tax_total	BIGINT	NOT NULL DEFAULT 0 CHECK
(tax_total >= 0),		
rounding_adjustment	BIGINT	NOT NULL DEFAULT 0,
grand_total	BIGINT	NOT NULL DEFAULT 0,
paid_total	BIGINT	NOT NULL DEFAULT 0 CHECK
(paid_total >= 0),		
tip_total	BIGINT	NOT NULL DEFAULT 0 CHECK
(tip_total >= 0),		
cogs_total	BIGINT,	
opened_at	TIMESTAMPTZ	NOT NULL,
first_fired_at	TIMESTAMPTZ,	
completed_at	TIMESTAMPTZ,	
origin_device_time	TIMESTAMPTZ	NOT NULL,
hlc	VARCHAR(48)	NOT NULL,
sync_state	sync.state_enum	NOT NULL DEFAULT 'local',
idempotency_key	VARCHAR(80)	NOT NULL,
aggregator_ref	VARCHAR(80),	
country_pack_version	VARCHAR(24)	NOT NULL,
notes	TEXT,	
metadata	JSONB	NOT NULL DEFAULT '{}'::jsonb,
version	INTEGER	NOT NULL DEFAULT 1,

```

        created_at          TIMESTAMPTZ    NOT NULL DEFAULT now(),
        updated_at          TIMESTAMPTZ    NOT NULL DEFAULT now(),

        CONSTRAINT uq_order_number UNIQUE (branch_id, business_day,
order_number),
        CONSTRAINT uq_idempotency UNIQUE (tenant_id, idempotency_key),
        CONSTRAINT ck_dine_in_table CHECK (order_type <> 'dine_in' OR
table_id IS NOT NULL),
        CONSTRAINT ck_completed CHECK (state <> 'completed' OR
completed_at IS NOT NULL)
) PARTITION BY RANGE (business_day);

CREATE INDEX idx_orders_branch_day ON sales.orders (branch_id,
business_day DESC);
CREATE INDEX idx_orders_open ON sales.orders (branch_id,
state)
WHERE state IN
('draft', 'open', 'held', 'parked', 'partially_paid');
CREATE INDEX idx_orders_customer ON sales.orders (customer_id,
opened_at DESC)
WHERE customer_id IS NOT NULL;
CREATE INDEX idx_orders_sync_pending ON sales.orders (sync_state,
opened_at)
WHERE sync_state <> 'synced';
CREATE INDEX idx_orders_aggregator ON sales.orders (tenant_id,
aggregator_ref)
WHERE aggregator_ref IS NOT
NULL;

ALTER TABLE sales.orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE sales.orders FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON sales.orders
USING (tenant_id = current_setting('app.tenant_id',
true)::uuid)
WITH CHECK (tenant_id = current_setting('app.tenant_id',
true)::uuid);

-- =====
-- inventory.stock_movements (append-only ledger)
-- =====
CREATE TABLE inventory.stock_movements (
    id                UUID                PRIMARY KEY,
    tenant_id         UUID                NOT NULL,
    location_id       UUID                NOT NULL,
    stock_item_id     UUID                NOT NULL REFERENCES
inventory.stock_items(id),
    batch_id         UUID                REFERENCES
inventory.stock_batches(id),

```

```

        movement_type          inventory.movement_type_enum NOT NULL,
        quantity                NUMERIC(18,6) NOT NULL CHECK (quantity <>
0),
        unit_id                 UUID                NOT NULL REFERENCES
inventory.uom(id),
        unit_cost               BIGINT              NOT NULL,
        total_cost              BIGINT              NOT NULL,
        balance_after           NUMERIC(18,6) NOT NULL,

        reference_type          VARCHAR(32)         NOT NULL,
        reference_id            UUID                NOT NULL,
        counterpart_movement_id UUID                REFERENCES
inventory.stock_movements(id),

        occurred_at             TIMESTAMPTZ         NOT NULL,
        recorded_at             TIMESTAMPTZ         NOT NULL DEFAULT now(),
        performed_by            UUID                NOT NULL,
        reason_code_id          UUID,
        notes                   TEXT,

        CONSTRAINT ck_batch_required CHECK (
            movement_type NOT IN ('purchase_receipt', 'production_output')
OR batch_id IS NOT NULL
        ),
        CONSTRAINT ck_reason_required CHECK (
            movement_type NOT IN ('waste', 'manual_adjustment') OR
reason_code_id IS NOT NULL
        )
) PARTITION BY RANGE (occurred_at);

CREATE INDEX idx_mv_item_loc_time ON inventory.stock_movements
(stock_item_id, location_id, occurred_at DESC);
CREATE INDEX idx_mv_reference ON inventory.stock_movements
(reference_type, reference_id);
CREATE INDEX idx_mv_batch ON inventory.stock_movements
(batch_id) WHERE batch_id IS NOT NULL;

-- Enforce append-only at the database level, not merely by convention
REVOKE UPDATE, DELETE ON inventory.stock_movements FROM app_role;
CREATE RULE no_update AS ON UPDATE TO inventory.stock_movements DO
INSTEAD NOTHING;
CREATE RULE no_delete AS ON DELETE TO inventory.stock_movements DO
INSTEAD NOTHING;

-- =====
-- governance.audit_entries (hash-chained, append-only)
-- =====
CREATE TABLE governance.audit_entries (
    id                UUID                PRIMARY KEY,

```


tenant_id	UUID	NOT NULL,
branch_id	UUID,	
sequence_no	BIGINT	NOT NULL,
occurred_at	TIMESTAMPTZ	NOT NULL,
recorded_at	TIMESTAMPTZ	NOT NULL DEFAULT now(),
actor_id	UUID,	
actor_type	VARCHAR(16)	NOT NULL,
impersonated_by	UUID,	
action	VARCHAR(80)	NOT NULL,
entity_type	VARCHAR(48)	NOT NULL,
entity_id	UUID,	
before_state	JSONB,	
after_state	JSONB,	
reason_code	VARCHAR(48),	
reason_text	TEXT,	
approver_id	UUID,	
approval_id	UUID,	
ip_address	INET,	
user_agent	TEXT,	
terminal_id	UUID,	
correlation_id	UUID	NOT NULL,
causation_id	UUID,	
entry_hash	BYTEA	NOT NULL,
previous_hash	BYTEA,	

```

    CONSTRAINT uq_audit_sequence UNIQUE (tenant_id, sequence_no)
) PARTITION BY RANGE (occurred_at);

```

```

CREATE INDEX idx_audit_actor      ON governance.audit_entries
(tenant_id, actor_id, occurred_at DESC);
CREATE INDEX idx_audit_entity    ON governance.audit_entries
(entity_type, entity_id, occurred_at DESC);
CREATE INDEX idx_audit_action    ON governance.audit_entries
(tenant_id, action, occurred_at DESC);
CREATE INDEX idx_audit_correlation ON governance.audit_entries
(correlation_id);

```

```

REVOKE UPDATE, DELETE ON governance.audit_entries FROM app_role;

```

```

-- =====
-- platform.outbox
-- =====
CREATE TABLE platform.outbox (
    id          UUID          PRIMARY KEY,
    tenant_id   UUID          NOT NULL,
    topic       VARCHAR(80)   NOT NULL,
    payload     JSONB         NOT NULL,
    headers     JSONB         NOT NULL DEFAULT '{}'::jsonb,
    status      VARCHAR(16)   NOT NULL DEFAULT 'pending',

```

```

    attempts          SMALLINT    NOT NULL DEFAULT 0,
    next_attempt_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_error         TEXT,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    processed_at       TIMESTAMPTZ
);

CREATE INDEX idx_outbox_ready ON platform.outbox (next_attempt_at)
WHERE status = 'pending';

```

25.3 Partitioning Strategy

FR-DR-001 [M] — High-volume, time-ordered tables SHALL be range-partitioned by time:

Table	Partition Key	Interval	Retention Online
sales.orders	business_day	Monthly	24 months, then archived
sales.order_lines	business_day	Monthly	24 months
inventory.stock_movements	occurred_at	Monthly	24 months
governance.audit_entries	occurred_at	Monthly	12 months, then cold storage
analytics.fact_sales_line	date_key	Monthly	36 months
sync.sync_operations	created_at	Weekly	90 days

FR-DR-002 [M] — Partitions SHALL be created automatically at least 3 months in advance by a scheduled job, with alerting if creation fails.

FR-DR-003 [M] — Archived partitions SHALL be exported to object storage in a queryable columnar format and SHALL remain retrievable within the statutory retention window.

25.4 Indexing Principles

Principle	Rationale
Index for the actual query, not the entity	Composite indexes ordered by selectivity and query shape
Partial indexes for hot subsets	Open orders are a tiny fraction of all orders; a partial index is orders of magnitude smaller
Covering indexes via INCLUDE	Avoids heap fetches on high-frequency dashboard queries
tenant_id leading in multi-tenant indexes	Aligns with RLS filtering and improves locality
No index without a query that uses it	Every index is write cost; unused indexes are pure loss

Principle	Rationale
Review index usage monthly	pg_stat_user_indexes drives removal of unused indexes
GIN indexes on JSONB only where queried	JSONB columns used only for storage need no index

25.5 Migration Strategy

FR-DR-010 [M] — All schema changes SHALL be expressed as versioned, forward-only migrations checked into source control.

FR-DR-011 [M] — Migrations SHALL be backward compatible with the previously deployed application version, enabling zero-downtime deployment and rollback.

FR-DR-012 [M] — Breaking schema changes SHALL follow the expand–migrate–contract sequence:

1. Expand : add the new column/table, nullable, with no constraint. Deploy.
2. Backfill : populate in batches with throttling to avoid lock escalation and replication lag.
3. Dual write: application writes both old and new. Deploy.
4. Migrate : switch reads to the new. Deploy. Observe.
5. Contract : stop writing old; drop the old column after a soak period. Deploy.

FR-DR-013 [M] — Migrations SHALL be tested against a production-sized dataset in staging with timing measured, and any migration exceeding a 5-second lock SHALL be rejected.

FR-DR-014 [M] — Adding a NOT NULL column to a large table SHALL use a nullable column plus a backfill plus a validated constraint, never a single blocking statement.

25.6 Data Volume Projections

Assumptions: mature tenant with 30 branches, 400 orders/branch/day, 3.2 lines/order.

Table	Rows/Day	Rows/Year	Est. Size/Year
orders	12,000	4.4 M	3.2 GB
order_lines	38,400	14.0 M	6.1 GB
order_line_modifiers	26,000	9.5 M	2.4 GB
order_payments	13,200	4.8 M	1.4 GB
stock_movements	155,000	56.6 M	18.5 GB
audit_entries	92,000	33.6 M	24.0 GB

Table	Rows/Day	Rows/Year	Est. Size/Year
fact_sales_line	38,400	14.0 M	4.0 GB
Total (one large tenant)			~60 GB/year

Note: stock_movements dominates because every order line generates a movement per expanded ingredient. A 3-line order with recipes averaging 6 ingredients produces 18 movements. This is the correct behaviour — it is what makes traceability and variance possible — but it makes movement volume the primary capacity driver and the primary partitioning target.

25.7 Backup and Recovery

Objective	Target
RPO (Recovery Point Objective)	≤ 5 minutes
RTO (Recovery Time Objective)	≤ 60 minutes
Backup frequency	Continuous WAL archiving + daily base backup
Retention	35 days point-in-time, 12 monthly, 7 annual
Restore testing	Quarterly full restore drill with documented timing
Cross-region copy	Encrypted, within the same data-residency jurisdiction

FR-DR-020 [M] — Backups SHALL be encrypted, and restore procedures SHALL be documented and rehearsed quarterly with recorded results.

FR-DR-021 [M] — The System SHALL support restoring a single tenant's data without restoring the entire database, using logical export filtered by tenant_id.

Chapter 26 — API Design | تصميم الواجهات البرمجية

26.1 Conventions

Aspect	Convention
Style	REST over HTTPS, JSON
Versioning	URL path: /v1/ . . . ; breaking changes increment the major version
Naming	Plural, kebab-case resources: /v1/purchase-orders
Field naming	camelCase in JSON
Timestamps	RFC 3339 with explicit offset
Money	{ "amount": 12500, "currency": "EGP", "exponent": 2 } — never a

Aspect	Convention
	float
Quantities	Decimal string to preserve precision: "1.250000"
Identifiers	ULID as string
Pagination	Cursor-based: ?cursor=...&limit=...; offset supported only for small result sets
Sorting	?sort=-createdAt,name
Filtering	? filter[branchId]=...&filter[businessDay][gte]=2026-08-01
Sparse fieldsets	?fields=id,orderNumber,grandTotal
Idempotency	Idempotency-Key header required on all POST/PATCH
Concurrency	If-Match with ETag on updates
Correlation	X-Correlation-Id accepted and echoed; generated if absent
Rate limits	X-RateLimit-Limit, -Remaining, -Reset headers

26.2 Error Model

All errors return RFC 7807 Problem Details:

```
{
  "type": "https://api.ros.app/errors/discount-approval-required",
  "title": "Discount requires approval",
  "status": 422,
  "detail": "A discount of 25% exceeds the 15% limit for role 'cashier'.",
  "instance": "/v1/orders/01J8XZ.../discounts",
  "code": "DISCOUNT_APPROVAL_REQUIRED",
  "correlationId": "01J8XZ...",
  "errors": [
    { "field": "discount.percentage", "code": "EXCEEDS_LIMIT",
      "message": "Maximum permitted is 15%.", "limit": 15 }
  ],
  "meta": { "requiredPermission": "pos.discount.approve" }
}
```

FR-API-001 [M] — Error responses SHALL use a stable machine-readable code in addition to the human-readable title and detail.

FR-API-002 [M] — Error messages SHALL be localised per the request's Accept-Language header.

FR-API-003 [M] — Error responses SHALL NOT leak internal details: stack traces, SQL, internal hostnames, or other tenants' data.

Status	Use
200	Success with body
201	Created; Location header set
202	Accepted for asynchronous processing; status URL returned
204	Success, no body
400	Malformed request
401	Not authenticated
403	Authenticated but not authorised
404	Not found, or not visible in this tenant scope
409	Conflict — version mismatch or state conflict
422	Semantically invalid — business rule violation
429	Rate limited; Retry-After set
500	Unexpected server error
503	Dependency unavailable; Retry-After set

Rationale for 404 on cross-tenant access: returning 403 confirms that a resource exists in another tenant. Returning 404 does not. In a multi-tenant system this distinction leaks information about competitors and must not be made.

26.3 Representative Endpoints

Orders

POST	/v1/orders	Create an order
GET	/v1/orders/{id}	Retrieve
GET	/v1/orders	List with filters
POST	/v1/orders/{id}/lines	Add a line
DELETE	/v1/orders/{id}/lines/{lineId}	Void a line (reason required)
POST	/v1/orders/{id}/fire	Fire to kitchen
POST	/v1/orders/{id}/discounts	Apply a discount
POST	/v1/orders/{id}/payments	Record a payment
POST	/v1/orders/{id}/complete	Complete
POST	/v1/orders/{id}/refunds	Refund
GET	/v1/orders/{id}/receipt	Receipt document
	(PDF/JSON)	

Inventory

GET	/v1/inventory/levels	Current levels
-----	----------------------	----------------

GET	/v1/inventory/movements	Movement ledger
POST	/v1/inventory/transfers	Create a transfer
POST	/v1/inventory/transfers/{id}/receive	Receive a transfer
POST	/v1/inventory/counts	Open a count session
POST	/v1/inventory/counts/{id}/lines	Submit counted lines
POST	/v1/inventory/counts/{id}/post	Post the count
POST	/v1/inventory/waste	Record waste
# Catalogue		
GET	/v1/menu	Effective menu for a context
GET	/v1/menu-items	List
POST	/v1/menu-items	Create
PATCH	/v1/menu-items/{id}	Update
POST	/v1/menu-items/{id}/availability	Toggle availability
GET	/v1/recipes/{id}/versions	Version history
POST	/v1/recipes/{id}/versions	Create a draft version
POST	/v1/recipes/{id}/versions/{v}/publish	Publish
# Procurement		
POST	/v1/purchase-orders	Create
POST	/v1/purchase-orders/{id}/approve	Approve
POST	/v1/goods-receipts	Receive goods
POST	/v1/supplier-invoices	Record invoice
GET	/v1/supplier-invoices/{id}/match	Three-way match result
# Sync		
POST	/v1/sync/batch	Upload an operation batch
GET	/v1/sync/changes	Download reference-data changes
GET	/v1/sync/status	Device sync state
# Reporting		
POST	/v1/reports/{reportId}/run	Run (sync for small, async for large)
GET	/v1/reports/runs/{runId}/status	Retrieve result or status
GET	/v1/dashboards/{dashboardId}	Dashboard payload

26.4 Authentication and Authorisation

FR-API-010 [M] — Interactive clients SHALL authenticate using short-lived access tokens (15 minutes) with refresh tokens (30 days, rotating on use).

FR-API-011 [M] — Machine clients SHALL authenticate using OAuth 2.0 client credentials or scoped API keys.

FR-API-012 [M] — Tokens SHALL carry: subject, tenant, scope set, and permitted branch set. Every request SHALL be authorised against both the permission and the scope.

FR-API-013 [M] — Refresh token rotation SHALL detect reuse of a consumed refresh token and SHALL revoke the entire token family on detection.

FR-API-014 [M] — API keys SHALL be displayed exactly once at creation, stored as a hash, and identifiable thereafter only by a non-secret prefix.

26.5 Idempotency

FR-API-020 [M] — Every POST and PATCH SHALL accept an Idempotency-Key header, and it SHALL be mandatory on all financially significant endpoints.

FR-API-021 [M] — The System SHALL store the key, the request fingerprint, and the response for at least 30 days.

FR-API-022 [M] — A repeated key with an identical request fingerprint SHALL return the stored response with `Idempotent-Replay: true`.

FR-API-023 [M] — A repeated key with a *different* request fingerprint SHALL return 409 Conflict, because this indicates a client defect rather than a retry.

26.6 API Non-Functional Requirements

NFR-PERF-030 [M] — Read endpoints SHALL respond within 200 ms at p95 and 500 ms at p99, excluding reporting endpoints.

NFR-PERF-031 [M] — Write endpoints SHALL respond within 400 ms at p95.

NFR-PERF-032 [M] — The sync batch endpoint SHALL process 500 operations within 3 seconds at p95.

NFR-API-001 [M] — The API SHALL be documented as an OpenAPI 3.1 specification generated from the implementation, so that documentation cannot drift from behaviour.

NFR-API-002 [M] — A deprecated endpoint or field SHALL be announced at least 180 days before removal, marked in the specification, and SHALL emit a `Deprecation` header.

Chapter 27 — Non-Functional Requirements | المتطلبات غير الوظيفية

27.1 Performance

Every performance requirement states a percentile and a reference condition. A requirement without both is untestable.

Reference conditions: POS device — Android 11, 4 GB RAM, quad-core 1.8 GHz, 10-inch 1280×800. Network — 5 Mbps, 80 ms RTT. Tenant — 30 branches, 400 orders per branch per day, 2,500 stock items, 800 menu items.

ID	Requirement	Target
NFR-PERF-001	POS item add to render	≤ 100 ms p95
NFR-PERF-002	Payment finalisation (offline / online)	≤ 800 ms / 1,500 ms p95
NFR-PERF-003	POS cold start	≤ 6 s
NFR-PERF-004	Fired order to KDS display (LAN)	≤ 1 s p95
NFR-PERF-005	Stock level query, 3,000 items	≤ 500 ms p95
NFR-PERF-006	Recipe expansion + depletion, 30-line order	≤ 200 ms p95
NFR-PERF-010	Standard report, 31 days, one branch	≤ 2 s p95
NFR-PERF-011	Consolidated report, 100 branches, 31 days	≤ 5 s p95
NFR-PERF-012	Dashboard initial load	≤ 3 s p95
NFR-PERF-020	Sync 5,000 queued operations at 2 Mbps	≤ 5 min
NFR-PERF-021	Local order persistence	≤ 50 ms p95
NFR-PERF-030	API read endpoints	≤ 200 ms p95
NFR-PERF-031	API write endpoints	≤ 400 ms p95
NFR-PERF-032	Sync batch of 500 operations	≤ 3 s p95
NFR-PERF-040	Web dashboard Largest Contentful Paint	≤ 2.5 s
NFR-PERF-041	Web dashboard Interaction to Next Paint	≤ 200 ms

27.2 Scalability

ID	Requirement	Target
NFR-SCALE-001	Tenants per regional deployment	$\geq 10,000$
NFR-SCALE-002	Branches per tenant	≥ 500
NFR-SCALE-003	Concurrent active terminals per region	$\geq 50,000$
NFR-SCALE-004	Orders per second, peak, per region	≥ 500
NFR-SCALE-005	Menu items per brand	$\geq 5,000$
NFR-SCALE-006	Stock items per tenant	$\geq 20,000$
NFR-SCALE-007	Horizontal API scaling	Stateless; linear to ≥ 40 pods
NFR-SCALE-008	Read scaling	Replicas addable without application change

27.3 Availability and Reliability

ID	Requirement	Target
NFR-AVAIL-001	Cloud service uptime (monthly, excluding planned windows)	≥ 99.9%
NFR-AVAIL-002	Enterprise-tier uptime commitment	≥ 99.95%
NFR-AVAIL-003	POS sales availability (independent of cloud)	≥ 99.99%
NFR-AVAIL-004	Planned maintenance windows	Off-peak, ≥ 7 days notice, ≤ 4 h/month
NFR-AVAIL-005	Zero-downtime deployment	Required for all routine releases
NFR-REL-010	Committed sale loss	Zero, under any single-device failure
NFR-REL-011	Duplicate financial effect	Zero, guaranteed by idempotency
NFR-REL-012	Data durability	≥ 99.999999999% (managed storage guarantee)
NFR-REL-013	RPO / RTO	≤ 5 min / ≤ 60 min

27.4 Usability and Accessibility

ID	Requirement
NFR-USA-001	3-line order completable in ≤ 6 interactions
NFR-USA-002	Touch targets ≥ 48 × 48 dp with ≥ 8 dp separation
NFR-USA-003	Signup to first order ≤ 30 minutes median
NFR-USA-004	POS fully operable via images for non-reading users
NFR-USA-005	New cashier productive after ≤ 30 minutes of training
NFR-USA-006	KDS legible at 2 m on a 21" 1080p display
NFR-USA-007	Dashboard meets WCAG 2.1 Level AA
NFR-USA-008	Contrast ratio ≥ 4.5:1 for text, ≥ 3:1 for UI components
NFR-USA-009	Dashboard fully keyboard-navigable
NFR-USA-010	Every destructive action is confirmable or undoable
NFR-USA-011	Every error message states what happened and what to do next

27.5 Maintainability

ID	Requirement	Target
NFR-MAINT-001	Unit test coverage, domain layer	≥ 90%
NFR-MAINT-002	Unit test coverage, overall	≥ 75%
NFR-MAINT-003	Cyclomatic complexity per function	≤ 10 (≤ 15 with justification)
NFR-MAINT-004	Module boundary violations	Zero, enforced in CI
NFR-MAINT-005	Critical/high vulnerabilities in dependencies	Zero at release
NFR-MAINT-006	Public API documentation	Generated from code; drift impossible
NFR-MAINT-007	New engineer to first merged PR	≤ 5 working days
NFR-MAINT-008	Local environment startup	≤ 10 minutes from clone
NFR-MAINT-009	Architecture Decision Records	Required for every significant decision

27.6 Observability

ID	Requirement
NFR-OBS-001	All logs structured (JSON), with tenant, branch, correlation, and causation identifiers
NFR-OBS-002	Distributed tracing across API, workers, and database
NFR-OBS-003	RED metrics (Rate, Errors, Duration) per endpoint and per handler
NFR-OBS-004	Business metrics emitted: orders/min, sync backlog, fiscal failures, offline terminals
NFR-OBS-005	No PII or secrets in logs, enforced by a redaction layer with an allowlist
NFR-OBS-006	Alerts defined for every SLO breach with documented runbooks
NFR-OBS-007	Per-tenant health view available to support without database access

27.7 Portability and Compatibility

ID	Requirement
NFR-PORT-001	POS supported on Android 9+, iOS 15+, Windows 10+
NFR-PORT-002	KDS supported on Android 9+ and evergreen browsers

ID	Requirement
NFR-PORT-003	Dashboard supported on the two most recent versions of Chrome, Safari, Edge, and Firefox
NFR-PORT-004	Dashboard responsive from 360 px to 2560 px
NFR-PORT-005	Deployment portable across major cloud providers (no proprietary lock-in beyond managed Postgres, Redis, object storage)
NFR-PORT-006	ESC/POS printer support across the certified hardware matrix

Chapter 28 — Testing Strategy | استراتيجية الاختبار

28.1 Test Pyramid

	▲		
	/ \	Manual exploratory & UAT	~ 50
scenarios	/ \	E2E (Playwright, Patrol)	~ 180
tests	/ \	Contract & integration	~ 600
tests	/ \	Component / module	~1,400
tests	/ _____ \	Unit (domain-heavy)	~5,000
tests			

FR-QA-001 [M] — The domain layer SHALL be tested exclusively with unit tests requiring no database, no HTTP, and no framework.

FR-QA-002 [M] — Every functional requirement SHALL trace to at least one automated test, verified by the traceability matrix in Appendix B.

28.2 Test Categories

Category	Scope	Environment	Runs
Unit	Single class/function; domain logic	In-memory	Every commit
Component	One module through its contract	Ephemeral Postgres (testcontainers)	Every commit
Integration	Cross-module flows, events, transactions	Full stack, seeded data	Every PR
Contract	API request/response against OpenAPI	Schema validation	Every PR
Conformance	Client/server logic parity (21.9)	Both runtimes	Every PR

Category	Scope	Environment	Runs
Isolation	Cross-tenant leakage (FR-PLT-013)	Generated from schema	Every PR
E2E	Full user journeys	Staging	Nightly and pre-release
Offline	Partition simulation, sync, conflict	Orchestrated devices	Nightly
Performance	Load and soak against NFR targets	Production-like	Weekly and pre-release
Security	SAST, DAST, dependency scan, secret scan	CI	Every commit / nightly
Localisation	RTL layout, bidi, truncation, printing	Device matrix	Pre-release
Accessibility	WCAG 2.1 AA	Automated + manual	Pre-release
Disaster recovery	Restore drill	Isolated	Quarterly

28.3 Critical Test Scenarios

These scenarios are release-blocking. Failure of any prevents release regardless of other results.

ID	Scenario	Pass Criterion
CT-01	72-hour full offline operation, 500 orders, then sync	Zero loss, zero duplication, fiscal sequence intact
CT-02	Network partition mid-payment on integrated terminal	No double charge; last-transaction query resolves correctly
CT-03	Concurrent order edits on one table from two terminals	Converges per CRDT rules; no line lost
CT-04	Power loss during payment write	Order recoverable; no partial state
CT-05	Cross-tenant access attempt on every table	All attempts return zero rows
CT-06	Client/server conformance corpus	Byte-identical results on every case
CT-07	Recipe expansion with 5-level sub-recipes and modifiers	Depletion matches manual calculation exactly
CT-08	Stock count during active trading	Variance excludes concurrent sales
CT-09	Fiscal submission with	All documents submitted on

ID	Scenario	Pass Criterion
	authority downtime for 6 hours	recovery; nothing lost
CT-10	Device clock set 3 hours ahead	HLC ordering preserved; skew alerted; original timestamps retained
CT-11	Multi-currency consolidated report	Rates displayed; totals reconcile to source
CT-12	Money allocation across split bills	Sum of parts exactly equals whole in all cases
CT-13	Loyalty double-redemption attempt from two offline terminals	Detected on sync; policy applied; ledger consistent
CT-14	Sync backlog of 20,000 operations after extended outage	Completes; no timeout; no memory exhaustion
CT-15	Arabic receipt printing across the printer matrix	Correct joining, ordering, and no truncation

28.4 Test Data

FR-QA-010 [M] — The System SHALL provide reproducible seed datasets: a minimal single-branch café, a 12-branch chain, a multi-brand multi-country group, and a cloud kitchen with aggregators.

FR-QA-011 [M] — Test data SHALL be synthetic. Production data SHALL NOT be copied into non-production environments.

FR-QA-012 [M] — Where production-shaped data is required for performance testing, it SHALL be generated at production scale rather than copied, or anonymised through a documented, verified pipeline.

28.5 Quality Gates

Gate	Blocks
Any unit or component test failure	Merge
Coverage below threshold	Merge
Module boundary violation	Merge
Lint or type error	Merge
Critical or high dependency vulnerability	Merge
Secret detected in diff	Merge
Isolation suite failure	Merge
Conformance corpus divergence	Merge

Gate	Blocks
Migration exceeding lock budget	Merge
E2E failure	Release
Performance regression > 20%	Release
Accessibility regression	Release
Any CT scenario failure	Release

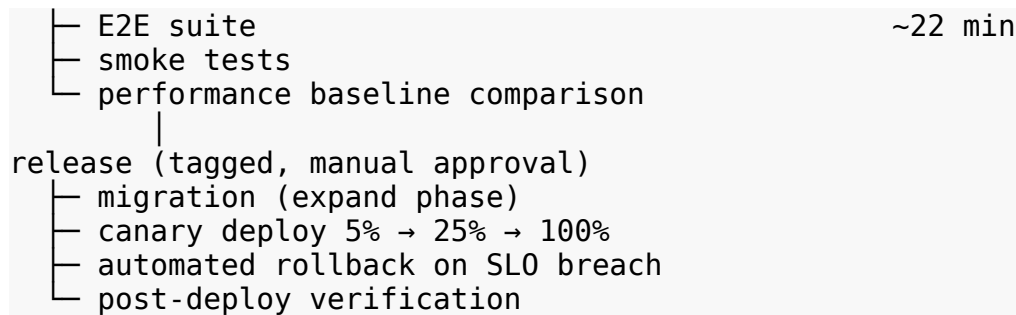
Chapter 29 — DevOps & Infrastructure | البنية التحتية والتشغيل

29.1 Environments

Environment	Purpose	Data	Access
Local	Development	Seed	Engineer
CI	Automated verification	Ephemeral	Automation
Development	Integration of in-progress work	Seed, reset nightly	Engineering
Staging	Pre-production verification, UAT	Production-shaped synthetic	Engineering, QA, Product
Sandbox	Customer and partner integration testing	Synthetic, tenant-isolated	External developers
Production	Live	Real	Restricted, audited, break-glass

29.2 CI/CD Pipeline

commit	
├ lint · typecheck · format	~1 min
├ unit tests (parallel shards)	~3 min
├ component tests (testcontainers)	~5 min
├ architecture boundary tests	~1 min
├ tenant isolation suite	~2 min
├ conformance corpus (server + client)	~2 min
├ SAST · dependency scan · secret scan	~3 min
├ build container images · sign · SBOM	~4 min
└─┘	
pull request	
├ integration tests	~8 min
├ contract tests against OpenAPI	~2 min
├ migration dry-run with lock timing	~3 min
└─ preview environment deployed	
└─┘	
merge to main	
└─ deploy to staging (automatic)	



FR-OPS-001 [M] — Deployments SHALL be zero-downtime and SHALL be rollback-capable within 5 minutes.

FR-OPS-002 [M] — Canary deployments SHALL automatically roll back on error-rate or latency SLO breach without human intervention.

FR-OPS-003 [M] — All infrastructure SHALL be defined as code (Terraform), reviewed, and version controlled. Manual production changes are prohibited except under a documented break-glass procedure.

FR-OPS-004 [M] — Container images SHALL be signed, scanned, and accompanied by a Software Bill of Materials.

FR-OPS-005 [M] — Feature flags SHALL gate risky changes, permitting per-tenant progressive rollout independent of deployment.

29.3 Release Management

FR-OPS-010 [M] — POS and KDS client releases SHALL support staged rollout by tenant, branch, and terminal.

FR-OPS-011 [M] — The client SHALL be able to operate against the previous API version for at least one release cycle, so that a client update failure never prevents trading.

FR-OPS-012 [M] — The System SHALL support forced client updates for security-critical releases, with the update deferrable until after the current shift closes.

Rationale: Forcing an update mid-service is the fastest way to have a customer stop trusting the product. Security-critical updates must still be deferrable to a shift boundary, with the deferral bounded.

FR-OPS-013 [M] — Release notes SHALL be published per release, in Arabic and English, and SHALL be visible in the product.

29.4 Monitoring and Incident Response

SLI	SLO	Alert
API availability	99.9% monthly	Page at 5-minute burn rate
API p95 latency	≤ 200 ms read, 400 ms write	Warn at 1.5×, page at 3×
Sync success rate	≥ 99.5%	Page below 98%

SLI	SLO	Alert
Sync backlog age	≤ 10 minutes p95	Page above 60 minutes
Fiscal submission success	≥ 99% within window	Page on repeated failure
Order processing errors	≤ 0.1%	Page above 0.5%
Database replication lag	≤ 10 s	Warn at 30 s, page at 120 s
Job queue depth	≤ 1,000	Warn at 5,000
Terminal offline rate	≤ 2%	Warn above 5%

FR-OPS-020 [M] — The System SHALL maintain an on-call rotation with documented escalation and runbooks for every paging alert.

FR-OPS-021 [M] — Incidents SHALL be classified by severity with defined response times:

Severity	Definition	Response	Resolution Target
SEV-1	Sales blocked for multiple tenants	15 min	4 h
SEV-2	Major feature unavailable; workaround exists	1 h	24 h
SEV-3	Minor feature degraded	4 h	5 days
SEV-4	Cosmetic or low impact	Next business day	Next release

FR-OPS-022 [M] — Every SEV-1 and SEV-2 SHALL produce a blameless post-incident review within 5 working days, with action items tracked to completion.

FR-OPS-023 [M] — A public status page SHALL report current and historical availability.

29.5 Cost Management

FR-OPS-030 [S] — Infrastructure cost SHALL be attributable per tenant, enabling gross-margin measurement per customer and detection of unprofitable accounts.

FR-OPS-031 [S] — The System SHALL alert when a single tenant's infrastructure consumption exceeds a configured multiple of their subscription revenue.